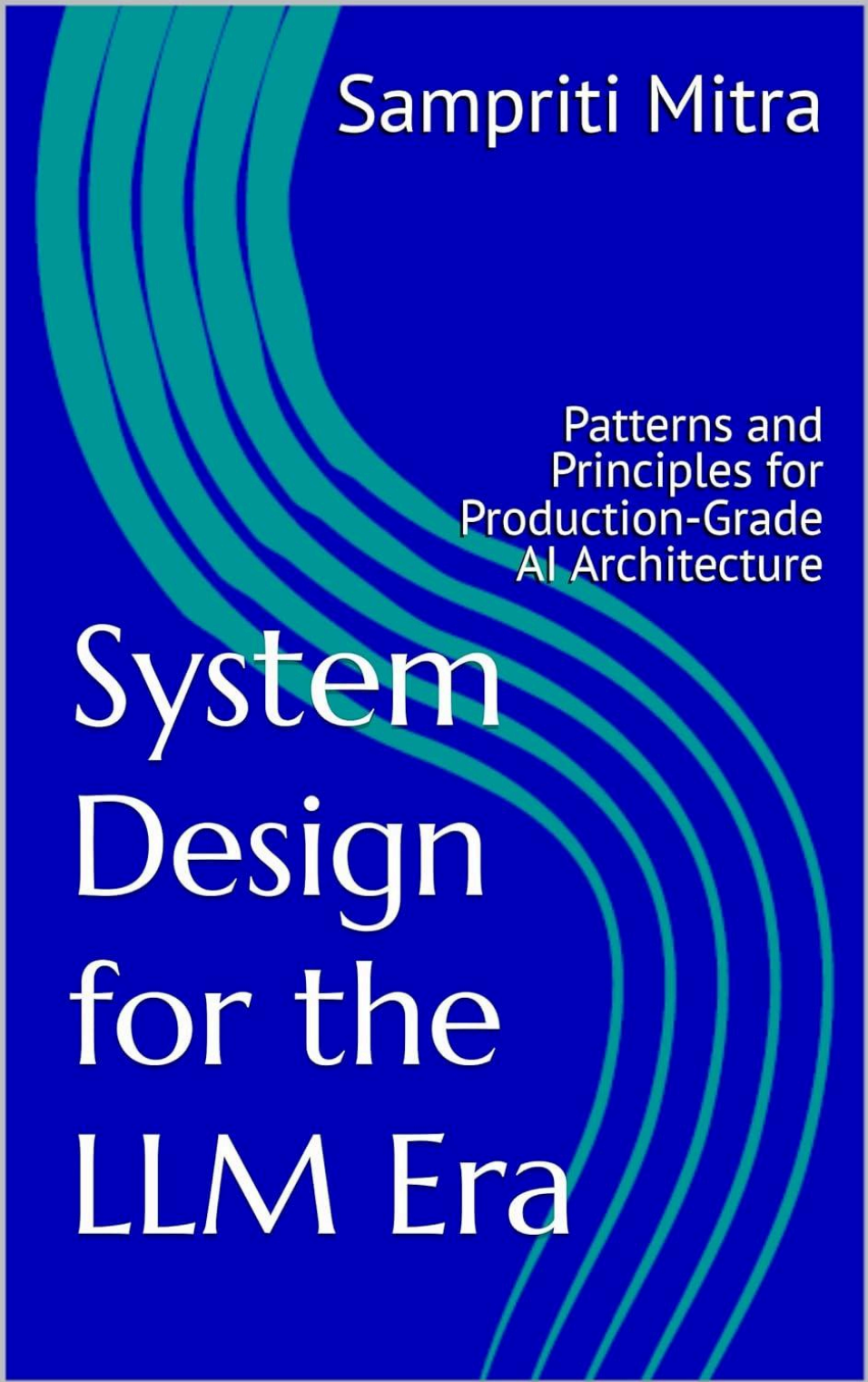


The book cover features a solid blue background. On the left side, there are several concentric, wavy lines in a teal or light blue color, creating a sense of motion or a stylized 'S' shape. The text is white and positioned on the right side of the cover.

Sampriti Mitra

Patterns and
Principles for
Production-Grade
AI Architecture

System Design for the LLM Era

System Design for the LLM Era

Patterns and Principles for Production-Grade AI Architecture

SAMPRITI MITRA

All Rights Reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

Limit of Liability/Disclaimer of Warranty: The information provided in this book is for educational and informational purposes only. While the author has made every effort to ensure the accuracy of the information within this book was correct at time of publication, the author does not assume and hereby disclaims any liability to any party for any loss, damage, or disruption caused by errors or omissions, whether such errors or omissions result from accident, negligence, or any other cause.

Disclaimer

1. **Source Material Transparency** The architectural analyses, case studies, and system breakdowns presented in this book are derived exclusively from publicly available information. Primary sources include official company engineering blogs, technical whitepapers, open-source repositories, and public conference presentations. No proprietary, confidential, or non-public internal documentation was accessed or disclosed in the creation of this material.

2. **No Affiliation** This book is an independent analysis and is not affiliated with, endorsed by, or sponsored by any of the companies mentioned in this text. All company names, product names, logos, and trademarks are the property of their respective owners and are used herein for identification and educational analysis purposes only.

3. **Limitation of Liability** The strategies, code samples, and designs outlined in this book are provided "as is" for educational purposes only. System design is highly context-dependent. The author and publisher assume no liability for errors, omissions, or any damages resulting from the use or implementation of the information contained herein.

To my mother and my father

Acknowledgment

System design is validated by stress testing, and this manuscript was no exception.

I'm grateful to Aadarsh for sharpening the readability and technical scope, and to Anurag for the probing questions that improved the design. Thanks also to Devansh for the rigorous review. Together, you challenged my assumptions and ensured the focus remained on production reality, not whiteboard theory.

To Apurve, thank you for being the ultimate quality gate and refusing to let me ship anything that is just good enough, in this book, and in every aspect of life.

Table of Contents

1. Atomic Units of LLM Systems.....	17
1.1. What is AI.....	18
1.2. What is Machine Learning.....	19
1.3. What is Deep Learning.....	19
1.4. What is Gen AI.....	19
1.5. What is NLP.....	20
1.6. What are LLMs.....	20
1.7. What are SLMs.....	20
1.8. Tokens.....	21
1.9. Embeddings.....	23
1.10. Pre-Training.....	24
1.11. Fine-Tuning.....	25
1.12. Prompt.....	26
1.13. Context Window.....	27
1.14. Reasoning.....	28
1.15. Prompt Engineering.....	29
1.16. Vector search.....	31
1.17. Text auto completion vs code completions.....	31
1.18. Abstract Syntax Trees (ASTs).....	32
1.19. Knowledge graphs.....	35
1.20. Model selection trade-offs.....	36
1.21. Deterministic vs Stochastic outputs.....	37
1.22. Managing Non-Determinism.....	37
1.23. What is RAG?.....	38
1.24. Why is naive RAG not sufficient in some cases?.....	40
1.25. Why do we need a knowledge graph?.....	41
1.26. What is a graphRAG?.....	42

1.27. What is context engineering?.....	42
1.28. Agentic AI.....	46
1.29. Modality.....	46
1.30. Data Residency & Compliance.....	47
1.31. Performance benchmarking.....	47
1.32. Handling failures.....	49
1.33. Conclusion.....	50
2. Core Architectural Patterns for LLM System Design.....	52
2.1. Designing for Resilience & Reliability.....	52
2.2. Designing for Low Latency.....	58
2.3. Designing for Cost Optimization.....	64
2.4. Designing for Grounding & Data Management.....	65
2.5. Designing for Testability & Observability.....	69
2.6. Designing for Security & Trust.....	71
2.7. Engineering for Production.....	77
2.8. Training with test data.....	78
2.9. Respecting user privacy.....	80
2.10. Conclusion.....	81
3. Case Study: Designing AI-Native IDEs.....	85
3.1. Introduction: What are AI-driven IDEs?.....	85
3.2. Functional Requirements:.....	85
3.3. Non-Functional Requirements.....	86
3.4. Scale estimates.....	87
3.5. API design.....	88
3.6. The Blueprint : High-Level Design.....	92
3.7. Database selection and data modelling.....	109
3.8. Deep dive into design.....	118
3.9. How to train with test data?.....	136
3.10. Monitoring and user feedback.....	137
3.11. Conclusion.....	137
3.12. References.....	137
4. Case Study: Adaptive Learning Platform.....	140

4.1. Introduction.....	140
4.2. Functional requirements.....	141
4.3. Non-functional requirements.....	141
4.4. Scale estimates.....	142
4.5. API design.....	143
4.6. The Blueprint: High-Level design.....	152
4.7. Database selection and data modelling.....	167
4.8. Deep dive into design.....	173
4.9. Testing scenarios.....	178
4.10. Monitoring and Alerting.....	180
4.11. Conclusion.....	181
4.12. References.....	182
5. Case Study: AI-powered search for e-commerce platforms.....	184
5.1. Introduction.....	184
5.2. Functional Requirements.....	185
5.3. Non-Functional Requirements.....	185
5.4. Scale estimates.....	186
5.5. API design.....	187
5.6. Background.....	189
5.7. Blueprint high-level design.....	193
5.8. Data modelling.....	209
5.9. System Design Deep Dive.....	217
5.10. Efficient ranking.....	228
5.11. Ensuring Robustness: Availability, Scalability, and Resilience	230
5.12. Monitoring and alerting.....	234
5.13. Alerting.....	235
5.14. Testing and Evaluation.....	236
5.15. Conclusion.....	242
5.16. References.....	242
6. Case Study: AI-powered Customer Support Agent.....	245
6.1. The Standard RAG Pattern.....	246

6.2. Functional Requirements.....	248
6.3. Non-functional requirements.....	248
6.4. Scale estimates.....	249
6.5. High level design.....	251
6.6. API design.....	253
6.7. System design blueprint.....	255
6.8. Data Modelling.....	267
6.9. Deep Dive.....	273
6.10. Accuracy - Avoiding hallucinations.....	276
6.11. Monitoring and Alerting.....	284
6.12. Failure Mode Analysis (FMA).....	287
6.13. Conclusion.....	288
6.14. References.....	288
7. Appendix.....	290

System Design for the LLM Era

Patterns and Principles for Production-Grade AI Architecture

0.0.1. About the book

The book is about the system design of AI-powered applications that use large language models (LLMs). It provides a full architectural blueprint, from the user-facing client to the backend orchestrator and the data layers that support it.

The book's target is not how to internally design/train/fine-tune models, but rather how to integrate and operationalize LLMs within an existing, scalable software architecture.

This book is designed to teach a topic by using a hands-on, example-driven approach.

0.0.2. What This Book Is

A practical guide to the system design principles, architectural patterns, and operational practices required **when integrating Large Language Models (LLMs) into distributed systems**. It is discussing system design concepts for systems that integrate with LLMs.

A deep dive into managing the unique challenges of LLMs: non-determinism, extreme cost variance, and context-aware retrieval (RAG).

0.0.3. What This Book Is Not

It is **not a book on building, training, or researching the internal mechanics of AI/ML models.**

It is not a general system design primer. We assume you understand distributed systems fundamentals (caching, microservices).

It is **not about building AI systems, but how to build systems that use AI.**

0.0.4. Why is it important now?

Companies are rapidly adopting AI. Many companies are trying to turn small AI experiments into big products, but they don't have a good plan. Engineers need a practical guide to build these new AI systems the right way - so they can handle many users, won't break, and won't cost too much.

This book is that guide. It explains how to design systems using AI models.

This book breaks down the architecture of real AI applications, like an AI-powered code editor or a smart learning app. It provides a deep, production-level dive into the operational challenges and solutions for building these specific types of systems.

0.0.5. What can readers expect from this book?

After reading this book, the reader will be able to:

- Architect a complete, production-grade AI-powered system from scratch.
- Design and mitigate the unique challenges of LLM APIs, like high latency and cost.
- Implement key software engineering patterns like circuit breakers and rate limiting for AI systems.

- Choose the right databases and data models for AI applications, including vector search engines.
- Build a scalable and resilient system that can handle high load and ensure user privacy.

0.0.6. Who is this book for?

Mid-to-Senior Engineers, Leads looking to integrate LLM into their existing systems.

0.0.7. Pre-requisites for this book

- General knowledge of distributed systems architecture.
- Intermediate-level programming skills in a modern language like Python, Java, or Go.
- Familiarity with microservices and API design.
- No prior hands-on experience with LLMs or vector databases is required.

0.0.8. About the author

Sampriti Mitra is a software engineering lead and an alumna of IIT BHU, with over six years of experience designing and building scalable distributed systems. She understands the practical challenges of integrating large language models (LLMs) into production-grade systems.

Her professional background includes roles at industry-leading companies like Sumologic and Razorpay. She also runs a newsletter, [Architecturally Speaking](#), which is dedicated to breaking down system design principles.

Part I: LLM Foundations and Design Principles

1. Atomic Units of LLM Systems

Software engineering has traditionally been deterministic: if x then y . If a unit test for this passes once, it passes a million times.

Building with Large Language Models (LLMs) requires a fundamental shift to probabilistic engineering. The same input x might yield y , y' , or sometimes z .

This chapter defines the primitives we use to change this uncertainty into reliable systems. Let's look at the raw concepts.

LLMs are a type of Artificial intelligence that have been trained on a vast amount of natural language and programming data which are capable of generating, reasoning and providing solutions. They have gained traction due to their understanding and ability to provide answers to user queries in a wide range of topics, including healthcare, finance and coding.

GPT-5 (OpenAI), Claude (Anthropic), Gemini (Google), DeepSeek, Grok, are some of the commonly used models with different strengths in reasoning, creativity and other areas.

LLMs require user inputs (called prompts) and, based on that, return textual outputs. The prompt should provide proper guidelines and guardrails that the LLM can follow to get the expected response.

Here are a few foundational concepts to get the reader familiar with this branch of AI.

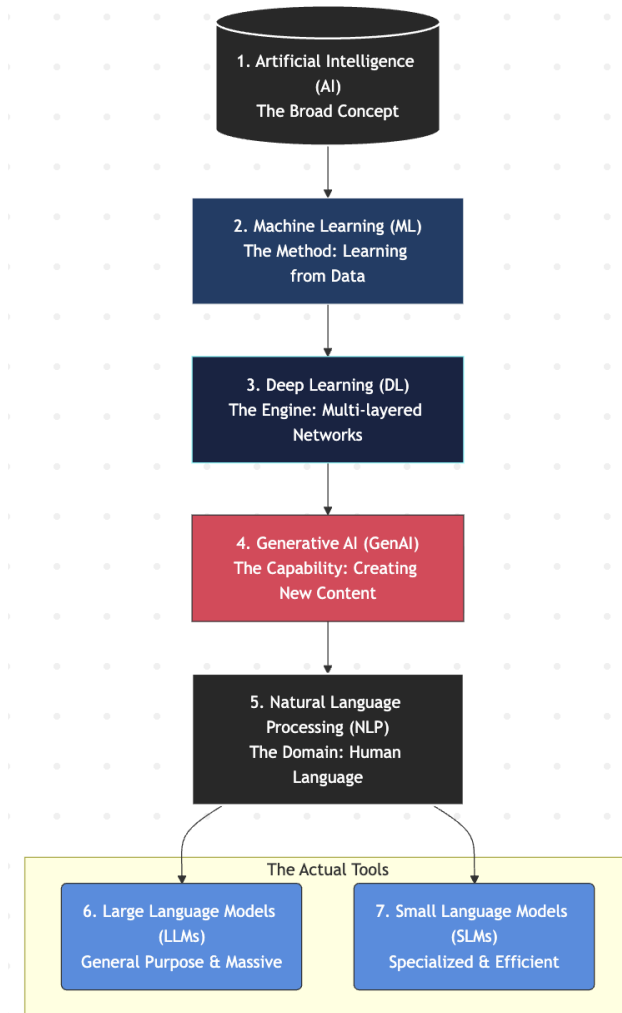


Fig 1.1 Different areas of AI

1.1. What is AI

AI or Artificial Intelligence is a stream of computer science that focuses on building smart machines that can do tasks that require human intelligence. This includes a wide range of capabilities, such as learning, reasoning, problem-solving, perception, and understanding language.

This is the overall, broad concept of machines that can simulate human intelligence to perform tasks.

1.2. What is Machine Learning

It's a subset of AI where systems are not explicitly programmed with rules. Instead, they learn directly from large amounts of data. For example, you don't teach a spam filter all the rules for spam; you show it millions of spam and non-spam emails, and it learns the patterns to look for.

1.3. What is Deep Learning

This is a more advanced subset of Machine Learning. These networks have many layers that allow them to learn very complex patterns from raw, unstructured data like images, sound, and text. This is the technology that powers self-driving cars, advanced language translation, and generative AI tools.

1.4. What is Gen AI

This is a specific type of AI which can create new original content rather than just analysing data. Generative AI works by using extremely large and complex machine learning models, often called Foundation Models, which have been trained on massive amounts of data from the internet. By processing this data, the models learn the patterns, structures, and relationships within it. When given a prompt, it uses this knowledge to generate a statistically possible new output.

1.5. What is NLP

NLP (Natural Language Processing) is a branch of artificial intelligence (AI) that gives computers the ability to understand, interpret, and generate human language—both spoken and written.

1.6. What are LLMs

A LLM (Large Language Model) is a massive AI model designed to understand, generate, and process human language on a vast, general-purpose scale.

It's trained on an enormous, diverse dataset, essentially a huge chunk of the internet.

Because it has seen so much data, it's a jack-of-all-trades. It can write poetry, summarize a legal document, translate languages, write code, and answer complex questions about history.

1.7. What are SLMs

An SLM (Small Language Model) is a more compact, focused AI model that is trained to be highly efficient and effective at a specific set of tasks. It's often trained on a smaller, more curated, and high-quality dataset that is specific to a particular domain (like medical data, financial data, or customer support logs).

Its power comes from its efficiency. SLMs have only millions or a few billion parameters.

They are designed to do a few things extremely well, with less cost and more speed. Because they are small, they can run directly on your phone, laptop, or car, without needing to connect to a powerful data center.

Now that we're familiar with the general concepts of AI and LLM, let's dive more into LLM related concepts and how they all fit together.

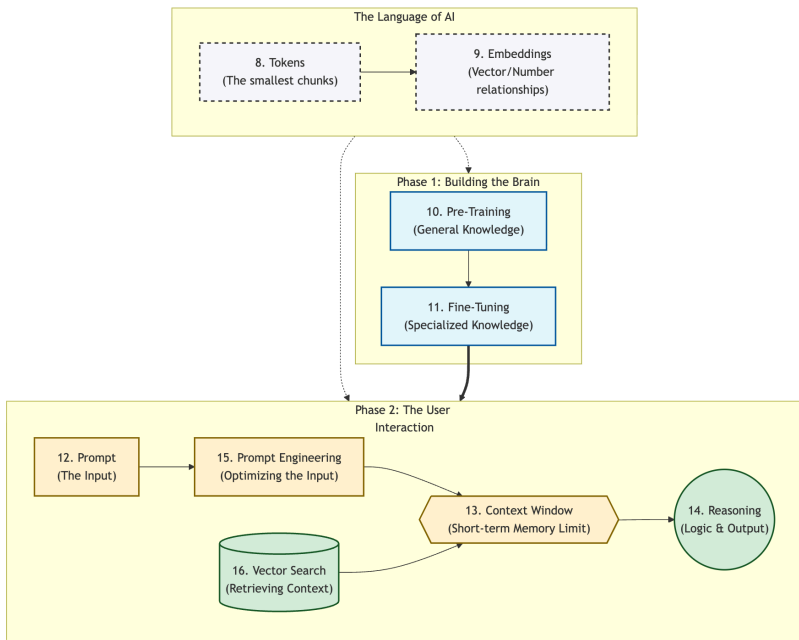


Fig 1.2 Different concepts in an LLM system

1.8. Tokens

In the context of AI and Language Models, a token is the fundamental, smallest piece of text that a model can process. Tokenization is the process of breaking down a sentence into individual words or tokens.

AI models don't read words or sentences like humans do. Instead, they see the world as a sequence of numbers. A token is the unit that gets converted into one of these numbers.

Text: "The cat sat on the mat."

Tokens: ["The", "cat", "sat", "on", "the", "mat", "."]

Tokens define our hard limits and our billing costs.

Latency & Throughput: LLMs process input tokens in parallel but generate output tokens serially.

Generating 50 tokens takes significantly longer than processing 50 input tokens. If the system requires low latency, we must constrain the model's output length.

COGS (Cost of Goods Sold): We pay per million tokens.

Verbose prompts kill margins. We can't send a 2,000-token prompt for a simple yes/no classification task, this is just burning money.

We should pre-compute and cache prompts where possible.

The Context Window Hard Cap: Every model has a maximum context window (e.g., 8k, 32k, 128k tokens).

We can't dump a user's entire history into the prompt. We need to design a Sliding Window or Summarization system to manage state within this constraint.

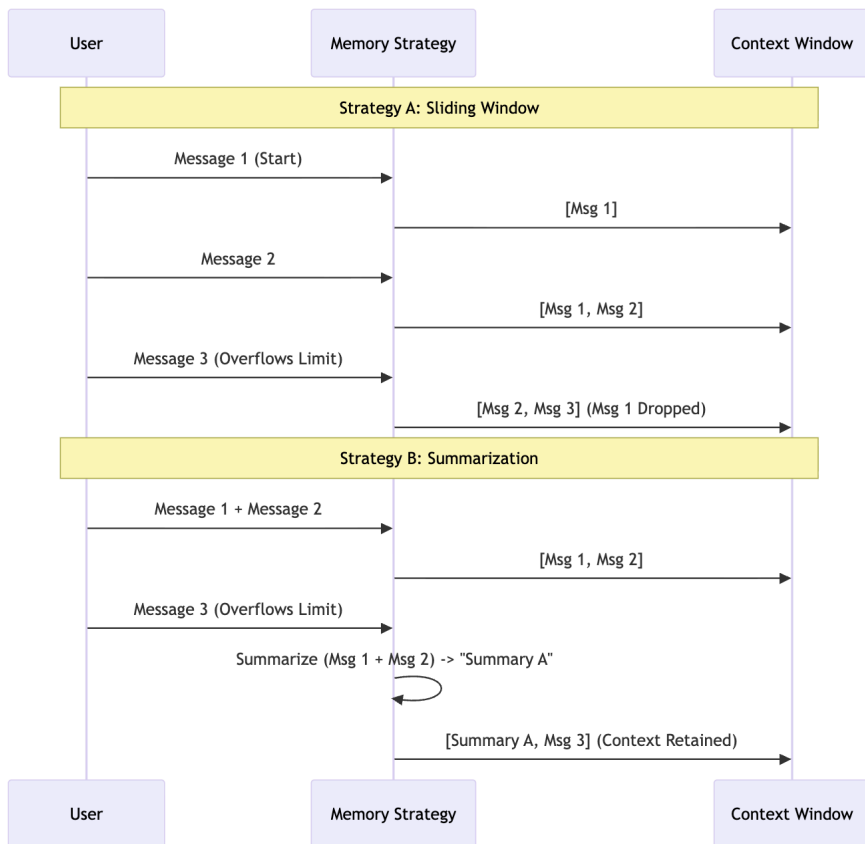


Fig 1.3 Context window strategies

1.9. Embeddings

Embeddings are numerical (vectors) representation of data (that can be text, image etc) that captures semantic relationships between words, images and other data. They transform high-dimensional complex data into low dimensional vectors and improve the computational efficiency of comparisons. In our context, embeddings would be vector representation of code, documentation etc.

This vector captures the semantic essence of the text, such that texts with similar meanings will have vectors that are close to each other in the vector space. Unlike a hash map which requires exact

key matches, embeddings allow us to find nearest neighbors i.e. content that is conceptually related even if it shares no keywords.

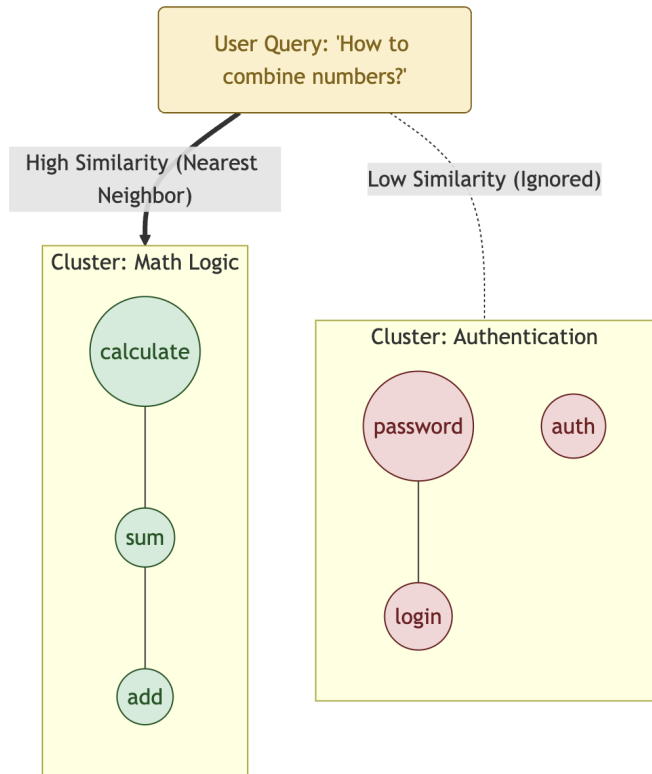


Fig 1.4 How embeddings cluster related words together

Larger dimensions (e.g., 1536 vs 3072) capture more nuance but increase storage costs and search latency. Choose the smallest dimensionality that satisfies the accuracy requirements to keep index lookups fast.

1.10. Pre-Training

Pre-training is the first and most important stage in creating a large AI model, like an LLM.

During this phase, the model is fed an enormous, diverse dataset—essentially a huge snapshot of the internet, including books, articles, websites, and code.

The goal is to teach it the fundamentals of language itself:

- How grammar works
- What different words mean
- How words and concepts relate to each other
- Common facts about the world
- The statistical patterns of how sentences are built

They haven't been trained for any specific job, but they have a massive, general-purpose understanding of language and the world.

1.11. Fine-Tuning

Fine-tuning is the process of taking a general-purpose, pre-trained model (like an LLM) and training it a little bit more on a smaller, specific set of data to make it an expert at a single, specialized task.

The model becomes an expert in a specific domain. It can learn any company's internal jargon, a specific person's writing style, or the rules of a niche field (like medicine or law).

If a pre-trained model's knowledge was cut off in 2023, we can fine-tune it on new documents from 2024 and 2025 to update its knowledge on that specific topic.

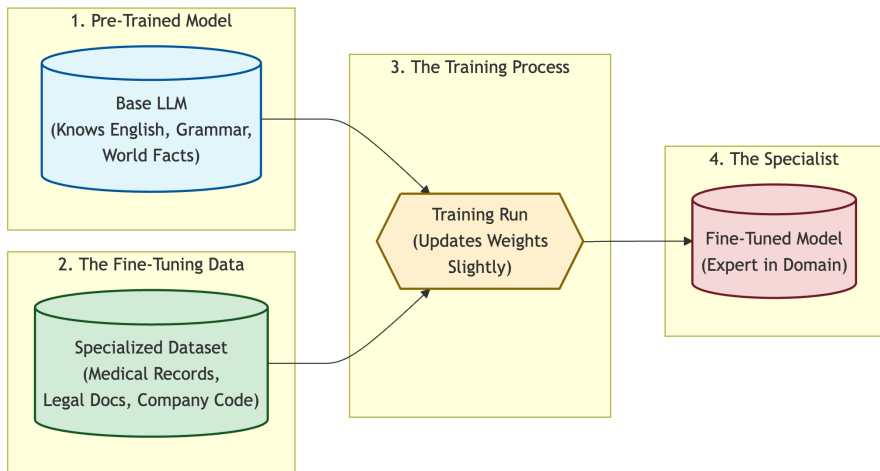


Fig 1.5 Fine tuning models

It is thousands of times cheaper and faster to fine-tune a model than to pre-train a new one from scratch.

1.12. Prompt

A prompt is the instruction, question, or piece of text we give to an AI model to get it to perform a task. The prompt is how we communicate and guide the AI. The quality of the AI's answer is highly dependent on the quality of the prompt.

A simple prompt is just a question. But a more advanced prompt can have several parts to guide the AI better:

- **Instruction:** The specific command you want the AI to perform.
- **Example:** "Summarize the following text..."

- Context: Background information the AI needs to understand the task.
- Example: "...The text is a transcript from a business meeting. Focus on the action items..."
- Persona: Telling the AI who it should be, which changes its tone and style.
- Example: "You are an expert financial advisor. Explain this concept to a complete beginner..."
- Format: Telling the AI how to structure its answer.
- Example: "...Provide the answer as a 3-bullet point list in JSON format."

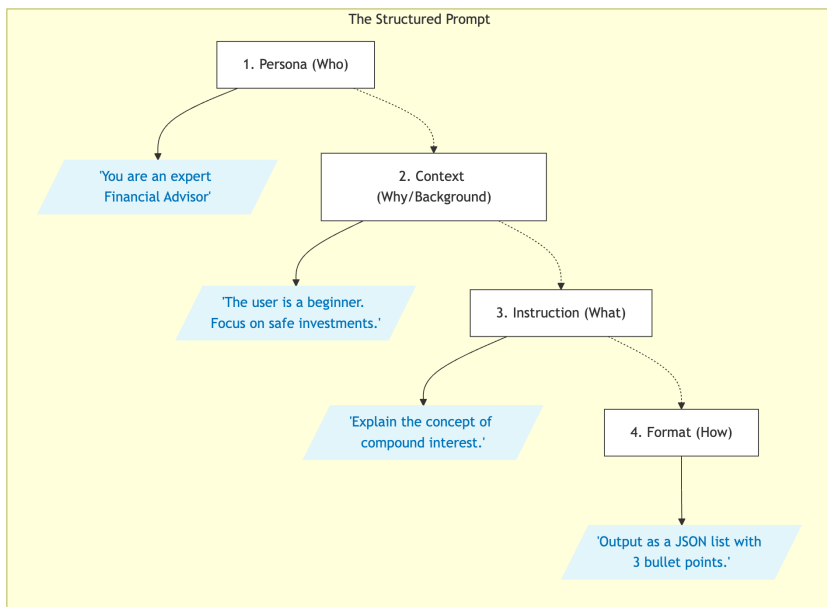


Fig 1.6 Different parts of the prompt

1.13. Context Window

The context window is the working memory / short-term memory of an AI model.

It is the fixed amount of information (measured in tokens) that the model can see and remember at any single moment.

Everything we send to the model, and everything it generates back, must fit within this window. This includes:

- The most recent prompt.
- The conversation history (e.g., previous questions and the AI's answers).
- Any documents, data, or system instructions you provided.
- Once the conversation becomes too long and overflows this window, the model forgets the earliest parts of the conversation.

1.14. Reasoning

Reasoning is the ability to take in information, process it logically, and produce a new conclusion or decision. It's the "thinking" part of intelligence.

It's an emergent behavior where the model statistically predicts the next most plausible word based on patterns of logic from its training data. By generating a Chain-of-Thought (thinking step-by-step), the model simulates a human reasoning process to solve a problem.

```
location: "Hyderabad, Telangana, India" time: "Saturday,
November 8, 2025 at 8:34:00 AM IST" thought I need to define
Artificial Intelligence (AI). This is a broad topic, so I should search
for a clear, general definition, the main types of AI (like narrow vs.
general), and its relationship to key subfields like machine learning
and deep learning. I will also look for common applications to
make the explanation more concrete.
print(google.search.search(queries=["What is artificial intelligence
definition", "types of artificial intelligence and examples", "what is
machine learning and deep learning", "common applications of
AI"])))
```


1.15. Prompt Engineering

Structuring the input prompts with good instructions, examples, constraints help return useful responses from the LLM. Prompt engineering is the practice of tweaking prompts to get the desired outputs.

Few established strategies for crafting better prompts:

1. Context enrichment

Providing surrounding code, documentation, etc, to the LLM for more context

Bad Prompt:

"Fix this bug"

Good Prompt:

"Here is the error log: NullPointerException at line 45. Here is the database schema for the User table. Here is the code for the UserService class. Based on this context, identify why the user object is null"

2. Role assumption

Instruct the LLM to play a certain role eg - "You are a coding assistant..."

Bad Prompt:

"Explain this code."

Good Prompt:

"You are a Principal Security Engineer auditing this code for vulnerabilities. Explain the risks in this function to a Junior Developer. Focus on SQL injection and input validation"

3. Provide examples

Give examples of a good response to the LLM, this helps improves responses to be closer to the one desired

Bad Prompt:

"Convert this log to JSON"

Good Prompt:

"Convert log lines to JSON. Input: [ERROR] 2024-01-01: DB fail Output: {'level': 'ERROR', 'date': '2024-01-01', 'msg': 'DB fail'} Input: [INFO] 2024-01-02: Started Output: {'level': 'INFO', 'date': '2024-01-02', 'msg': 'Started'} Input: [WARN] 2024-01-05: High CPU"

4. Provide guardrails

Explicitly mention behaviour which is not allowed to avoid hallucinations. Eg - Do not run destructive commands unless specified

Bad Prompt:

"Write a script to delete old files."

Good Prompt:

"Write a Python script to delete files older than 30 days. CONSTRAINT: Do not run any destructive commands (like os.remove) immediately. Instead, print a 'DRY RUN' list of files that would be deleted. Ask for user confirmation before deletion"

5. Chain-of-Thought

Bad Prompt:

"How many distinct IP addresses hit the server?"

Good Prompt:

"Analyze the server logs to find the count of distinct IPs. Think step-by-step:

First, identify the IP address pattern in the logs.

- Extract all IP instances into a list.

- Filter the list to remove duplicates.
- Count the remaining items. Return the final count"

1.16. Vector search

This allows semantic retrieval of code chunks when a developer asks a query. This is helpful when we want to gather the context around a query to be sent to an LLM. Since embeddings generate low-dimensional vectors, we use vector search to find semantically similar chunks of code matching the query.

1.17. Text auto completion vs code completions

In text auto completion, there is linear prediction of words without considering any semantic structure.

In code complete, syntax correctness is required along with semantic and structural awareness of the code. This means that depending on where a code definition is present in a file or class or method, the code complete predictions might differ.

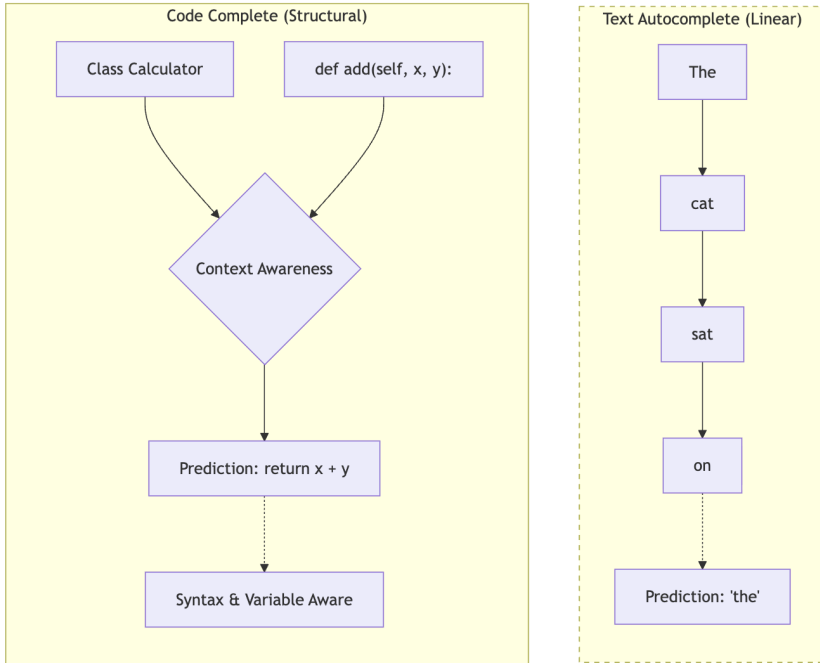


Fig 1.7 Code completions vs text auto completions

1.18. Abstract Syntax Trees (ASTs)

ASTs provide structured representation of code helping with syntax corrections, refactors and semantic edits. It is a tree data structure that shows a hierarchical relationship between different components of the code.

```
class Calculator:
    def add(self, x, y):
        return x + y

    def subtract(self, x, y):
        return x - y
```

The AST representation of this would look like

```
ClassDef (Calculator)
├── FunctionDef (add)
│   ├── Arguments: self, x, y
│   └── Return: BinOp (x + y)
└── FunctionDef (subtract)
    ├── Arguments: self, x, y
    └── Return: BinOp (x - y)
```

1.18.1. How the AST Looks for a Large Codebase

ASTs are especially useful for semantic codebase chunking. A large codebase can be represented in AST format (think of folders as parent nodes with file classes as child nodes within them, and the different methods within a class as further children of the file nodes). For class definition chunks, the entire ClassDef node is considered as one chunk, including all the functions. For function definition chunks, the entire FunctionDef node is considered as one chunk, e.g., add method with its arguments and return functions.

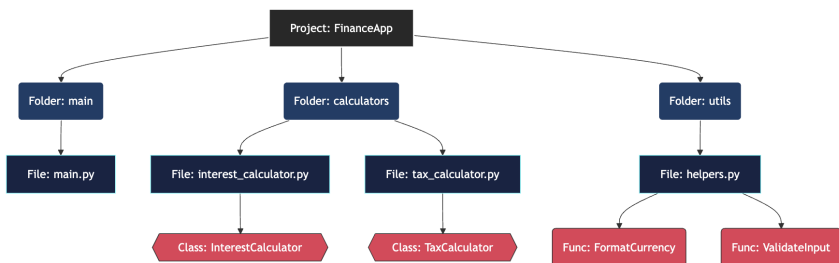


Fig 1.8 AST in a codebase

- Root Level: Project or Module
 - The top-level node represents the whole project or module

- It contains multiple child nodes, each representing a file (or module)
- File Level: Module/Script AST
 - Each file has its own AST representing all code within that file
 - This includes class definitions, function definitions, import statements, and global code
- Class/Function Level: Inside Each File
 - Inside each file's AST, class and function definitions appear as child nodes
 - Each class contains methods (functions) and statements
 - Functions contain statements and expressions

```

[Project: FinanceApp]
├─ [File: main.py]
│   ├── [Function: main]
│   └── [Function: run_app]
├─ [Folder: calculators]
│   ├── [File: interest_calculator.py]
│   │   ├── [Class: InterestCalculator]
│   │   │   ├── [Method: simple_interest]
│   │   │   └── [Method: compound_interest]
│   └── [File: tax_calculator.py]
│       ├── [Class: TaxCalculator]
│       │   ├── [Method: calculate_tax]
│       │   └── [Method: apply_deductions]
├─ [Folder: utils]
│   ├── [File: helpers.py]
│   │   ├── [Function: format_currency]
│   │   └── [Function: validate_input]
└─ [File: README.md]
  
```

Fig 1.2 Abstract Syntax Tree for classes

1.19. Knowledge graphs

They capture relationships between functions, classes, methods in a codebase, useful for answering questions like where does this method get called from?

For example, for the Finance App above, the knowledge graph would somewhat look like this:

Nodes (entities)

- FinanceApp (Project)
- main (Module/Folder)
- calculators (Module/Folder)
- utils (Module/Folder)
- main.py (File)
- interest_calculator.py (File)
- tax_calculator.py (File)
- helpers.py (File)
- main (Function in main.py)
- run_app (Function in main.py)
- InterestCalculator (Class in interest_calculator.py)
- TaxCalculator (Class in tax_calculator.py)
- format_currency (Function in helpers.py)
- validate_input (Function in [helpers.py](#))
- and so on ...

Edges (relationships)

From	Relationship	To
FinanceApp	hasModule	main
FinanceApp	hasModule	calculators
FinanceApp	hasModule	utils
main	hasFile	main.py

calculators	hasFile	interest_calculators.py
tax_calculators.py	containsClass	TaxCalculator
TaxCalculator	hasMethod	calculate_tax
helpers.py	containsFunction	format_currency

and so on..

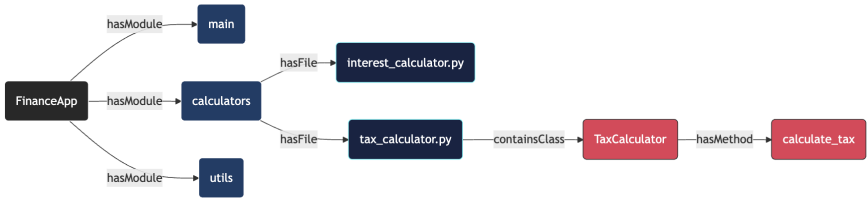


Fig 1.9 Knowledge graph for a codebase

1.20. Model selection trade-offs

There is no 'one model to rule them all'. Every architectural choice requires balancing competing constraints. When selecting a model, you are generally trading off between these four dimensions:

1. context window size - how much context can be sent to the LLM
2. latency (ms level performance)
3. security/privacy
4. cost

Claude has been found to work well with larger contexts, and GPT-5 handles excellent reasoning, but with higher latency and cost. Smaller models might be fine-tuned for lower latency, but accuracy might suffer.

1.21. Deterministic vs Stochastic outputs

Deterministic means the same input will always produce the exact same output. It's predictable, consistent, and follows a fixed set of rules.

Stochastic means the same input can produce different outputs. It involves randomness and probability, so the result is not perfectly predictable.

1.22. Managing Non-Determinism

The Temperature Parameter: Unlike standard deterministic functions where $\text{input}(x)$ always equals $\text{output}(y)$, LLMs are probabilistic. As an engineer, the primary control knob for this behavior is Temperature.

Low Temperature (0.1-0.2) : The model selects the most probable next token. Use this for code generation, data extraction, and classification where consistency is paramount.

High Temperature (approx 0.7 - 1.0) : The model boosts randomness by increasing the selection possibilities of less probable tokens. Use this for creative writing, brainstorming, or generating diverse synthetic data.

Architectural Implication: We cannot unit test LLM outputs with string equality assertions. We must design the test suites to tolerate variance or force temperature = 0 during regression testing.

Temperature also affects retry logic. With temperature = 0, a request failing validations (like a malformed JSON), will fail

exactly the same way when retried. A temperature >0 might fix it.

1.23. What is RAG?

How do we retrieve semantically relevant documentation and cases that might be the same in meaning and intention as the user query but worded differently?

The capability to retrieve semantically similar but diversely worded data is the cornerstone of the RAG (Retrieval Augmented Generation) engine. This involves a few fundamental concepts.

Data Cleaning

Before chunking or embedding, raw data must be rigorously cleaned. This is not just removing whitespace. It involves:

De-noising: Stripping HTML tags, boilerplate headers/footers, and navigation links that dilute the semantic density of the text.

PII (Personally Identifiable Information) Scrubbing: Regex-based removal of emails, SSN (Social Security Number), and API keys before the data ever touches an external model API.

Normalization: Standardizing unicode characters and date formats so the vector space remains consistent.

Vectorization

Raw knowledge data (documentation, closed tickets) is converted into high-dimensional numerical arrays called Vectors (or embeddings) using specialized embedding models (e.g., OpenAI's text-embedding-ada-002). This process captures the semantic meaning of the text.

Vector Search

The user's contextualized query is also vectorized. Algorithms like Cosine Similarity or Approximate Nearest Neighbor (ANN) search the high-dimensional Vector Database space to find the most relevant document chunks and support cases whose vectors are closest in proximity to the query vector. This ensures we retrieve documentation or cases that have the same meaning and intention as the user query. This retrieval is the foundation for RAG techniques.

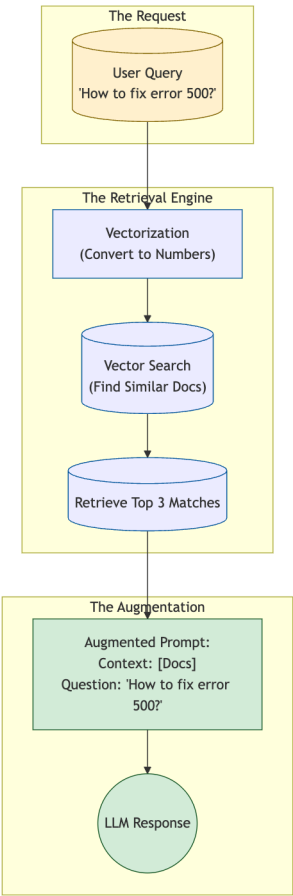


Fig 1.10 Vector search flow

Retrieval Augmented Generation is the method of augmenting or enhancing the prompt with context of relevant and factual data before sending it to the LLM, so that the LLM can give far more accurate and relevant results. This is the foundational method for ensuring high accuracy and minimizing hallucinations by grounding the LLM's response in verified, proprietary knowledge.

1.24. Why is naive RAG not sufficient in some cases?

A naive RAG often falls into two critical limitations.

Similarity vs true relevance

Vector search excels at finding similarity but that is not always strictly relevant.

Several documents may be framed similarly, which leads to the retrieval of several semantically similar chunks which may be related to the query. So the LLM receives redundant, overwhelming context, which makes it difficult to pinpoint the exact solution which is needed.

Inefficient retrieval due to poor chunking

Documents are often organized hierarchically (Title->Section->Sub section)

If a user query matches the title of a document, the system may retrieve the high-level summary but miss the critical resolution located five sections down. The core piece of information the user needs is missed because the embedding model didn't recognize the connection between the high-level match and the low-level detail.

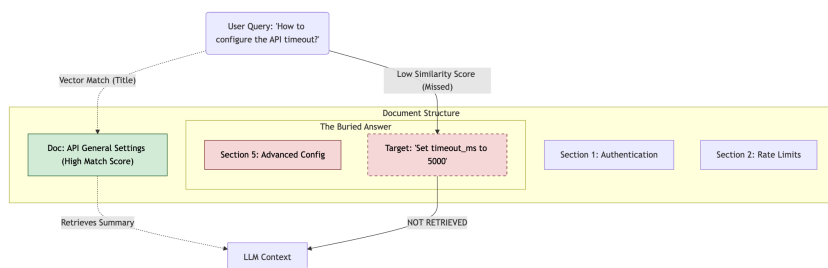


Fig 1.11 Problems with naive RAG

1.25. Why do we need a knowledge graph?

To mitigate the limitations of semantic-only retrieval, we introduce a Knowledge Graph (KG). A KG is a structured representation of data defining relationships between entities. Graph databases like Neo4J or Amazon Neptune can be used for this. A KG makes it easier to retrieve a particular entity based on its relationship with other entities.

Instead of relying on word similarity, the KG understands relationships such as:

Document A -[is_part_of]-> Section 3, or

Ticket X -[resolved_by]-> Code Commit Y.

This allows the retrieval engine to traverse complex, logical paths to find the definitive solution set.



Fig 1.12 Knowledge graph retrieval

1.26. What is a graphRAG?

It is an advanced RAG approach that combines knowledge graphs with LLMs to improve accuracy and context of generated responses. It allows the system to traverse complex relationships between entities, to find highly relevant, inter-connected information which is optimal for navigating structured and domain-specific data.

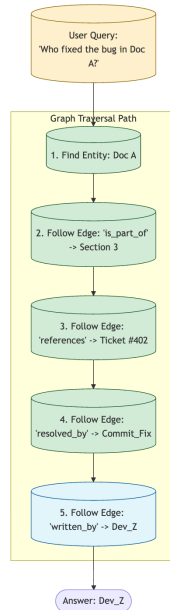


Fig 1.13 Working of a graphRAG

1.27. What is context engineering?

Building with LLMs is a data pipeline problem, not a prompting problem. Architecture controls the flow of context. The goal is to build a pipeline that moves data from the Long-term Memory (Vector Database) into the Short-term Working Memory (Context Window) at the exact moment of inference.

Context Engineering is the strategic practice of designing, constructing, and managing all the relevant information, tools, and constraints that an AI model sees before it generates a response.



Fig 1.14 How inference pipeline works

Key Aspects of Context Engineering:

1. **Dynamic Information Assembly:** The system actively gathers diverse pieces of information tailored to the specific task or conversation turn. This can include:
 - **System Instructions:** Defining the AI's role, persona, and behavioral guidelines (e.g., "You are a helpful customer support agent").
 - **Conversation History (Memory):** Providing relevant excerpts from previous interactions to maintain continuity and statefulness.
 - **External Knowledge (RAG):** Retrieving facts, documents, or data from external sources (like a company database or the web) to ground the response in up-to-date and specific information, thereby reducing hallucinations.
2. **Context Window Management:** Large language models have a finite limit to the amount of information they can process at one time (context window). Context engineering involves:
 - **Prioritization:** Deciding which information is most critical for the current step.
 - **Compression:** Using techniques like summarization or filtering to condense large amounts of data into the most relevant, token-efficient format.
3. **Tool Orchestration:** The discipline involves integrating and managing access to external tools or APIs (Application

Programming Interfaces). The AI must be correctly prompted and provided with the context to decide when to use a tool, how to use it, and how to incorporate the results into its final answer.

4. **Ensuring Consistency and Reliability:** By systematically controlling the AI's informational environment, context engineering helps ensure that the model produces accurate, consistent, and policy-aligned outputs across different interactions and users.

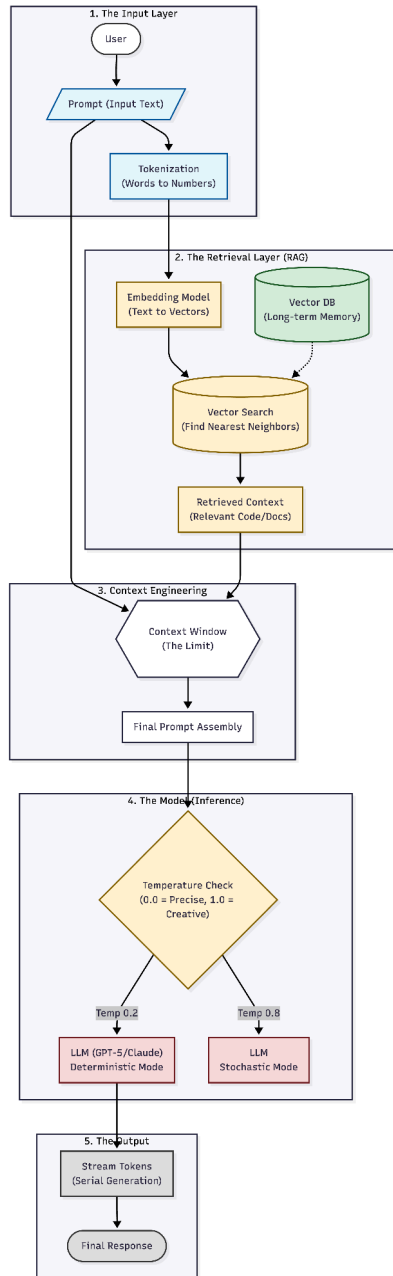


Fig 1.15 Inference flow with context engineering

1.28. Agentic AI

Agentic AI (or an AI Agent) is an autonomous system that uses an AI model (like an LLM) to reason, plan, and execute a series of actions to achieve a high-level goal.

The key is that the path is dynamic and stochastic. The AI is the orchestrator. It is given a goal, and it figures out the steps on its own.

Agentic AI works in a continuous reasoning loop (often called a ReAct loop: Reason + Act):

- Plan: The AI analyzes the goal and breaks it down into a plan
- Act: It executes the first step by choosing a tool. This could be:
 - Using a search engine
 - Running a piece of code
 - Querying a database
 - Reading a file

Observe: It takes the result of that action (e.g., a list of search results) and adds it to the context window

Reason: It looks at the goal, its plan, and the new information, then decides what to do next. It might adjust its plan, execute the next step, or decide the goal is complete. This loop repeats until the goal is met.

1.29. Modality

Humans don't experience the world just through text, and modern AI doesn't either. To build truly immersive systems, we must understand the different input channels an AI can process.

The **most common modalities** include:

1. **Text:** This is the modality of written language. Models that process text are used for translation, summarization, and chatbots.
2. **Images:** This modality is for static visual information. Models that process images are used for object recognition and facial detection.
3. **Audio:** This modality includes spoken language, music, and sounds. This is used for speech recognition (like Siri) and music generation.
4. **Video:** A complex modality that combines moving images (frames) with audio.
5. **Spatial:** A spatial modality that understands depth and the 3D structure of the world, which is crucial for self-driving cars.

1.30. Data Residency & Compliance

Beyond latency and cost, Data Residency is now a primary constraint for enterprise systems.

Public APIs (OpenAI, Anthropic): Data leaves the VPC.

Unacceptable for PII(Personally Identifiable Information), PHI (Protected Health Information), or strict PCI (Payment Card Industry) environments without zero-retention agreements.

Private Cloud / VPC (Bedrock, Azure OpenAI): Data stays within the cloud perimeter but relies on provider availability.

Self-Hosted Open Weights (Llama 3, Mistral): We have full control of the data. This is required for strict air-gapped environments or for compliance. This trades operational complexity (GPU management) for compliance safety.

1.31. Performance benchmarking

You cannot improve what you do not measure.

However, speed and intelligence are fundamentally different metrics in AI. We split benchmarking into two distinct categories to measure them separately:

Task quality benchmarking

Measures how smart and accurate the model is. (e.g., "Does it have high-school level knowledge?" or "Is it a good-quality summarizer?")

This lets us know how good the model is at a specific task. We run the model against a standardized dataset and evaluate its score.

Inference time benchmarking

Measures how fast and efficient the model is. For example - how many tokens per second it generates, or how fast is the time to first token. This benchmarking answers the business-critical question: How fast and expensive is this model to run? This is about speed and throughput.

Custom Benchmarking:

Define the Goal: What is the one thing the LLM must do well?

Example: "It must accurately summarize our customer support tickets."

Create a Golden Set: Manually create 50-100 high-quality examples of inputs and ideal outputs.

Input: (A long, angry customer ticket)

Ideal Output: {"summary": "User is angry about billing error on invoice #1234.", "sentiment": "negative", "topic": "billing"}

Run Models Against the Set: Run the models we want to compare (e.g., Llama 3 vs. GPT-5) against all the inputs in the golden set.

Evaluate the Results: How to score the outputs?

Human Evaluation: The most reliable way. Have a human expert score the model's output against the ideal output on a scale of 1-5.

LLM-as-a-Judge: A modern, scalable technique. We use a powerful judge LLM (like GPT-5) to grade the output of the test LLM.

Prompt for Judge: You are an expert. Here is an input, an ideal response, and a model's actual response. Rate the model's response from 1-5 for accuracy. Response: [Model's Output]

Key Metrics to Measure:

Time to First Token (TTFT): How long the user waits before seeing the first word.

Tokens Per Second (TPS): How many tokens are generated per second after the first one. This measures the overall generation speed.

Total Latency: The full time from prompt to the end of the response.

Throughput (Requests/sec): How many concurrent users our system can handle.

Resource Usage: GPU Memory (VRAM) consumption. This determines what kind of hardware we need to buy or rent.

1.32. Handling failures

There are 3 main kinds of failures that can occur in an LLM output.

1.32.1. Hallucination

A hallucination is when the model confidently invents a false or nonsensical answer because it's statistically plausible. We solve this by grounding the model giving it relevant information using RAG.

1.32.2. Knowledge cut off

This problem is that the model's knowledge is static, frozen at the time its training data was collected (e.g., "**I only have knowledge up to April 2023**"). This too, can be solved by providing the model enough context using RAG for example.

1.32.3. Bias

This is the hardest problem because the bias is deeply embedded in the petabytes of human text the model was trained on.

By using filtering on the initial dataset, and by providing guardrails in the prompts, along with human feedback, this bias can be managed.

1.33. Conclusion

LLMs are not magic; they are probabilistic compute engines bound by rigid resource constraints.

We have stripped away the hype to reveal the raw materials of this technology. Every architectural decision we make makes us choose trade offs:

Cost: Defined by input token volume and model size.

Latency: Defined by output token volume and retrieval overhead.

Quality: Defined by context window usage and model reasoning power (size).

- For high quality and low latency: We need to pay a premium for massive GPUs and over-provisioned throughput
- For low cost and low latency: We need to sacrifice reasoning capabilities and context depth (smaller models, aggressive summarization)
- For high quality and low cost: We can accept batch processing latency (async queues)

The magic of AI engineering isn't prompting—it's managing these constraints. The rest of this book is about the patterns we use to overcome this triangle: using RAG for infinite context, Caching for low latency.

Now that we understand the atomic units, let's learn how to assemble them.

2. Core Architectural Patterns for LLM System Design

Why Integration Requires New Patterns

In Chapter 1, we introduced the fundamental concepts of LLM integration: tokens, embeddings, and the basic idea of Retrieval-Augmented Generation (RAG). However, integrating LLMs into a production system introduces a new class of dependency: one that is non-deterministic, high-latency, and carries a high, variable operational cost.

As experienced engineers, we already know how to build reliable systems. This chapter isn't about reinventing those principles. It's about adapting them for the unique, messy challenges LLMs throw at us. Consider this an architect's playbook. These patterns will be the common framework for all the case studies we'll look at next.

2.1. Designing for Resilience & Reliability

Our system's stability is now tied to an external API that is slower, more expensive, and less predictable than any database or microservice calls. The primary goal is to decouple the application's health from the provider's health.

2.1.1. Pattern: The GenAI Service or LLM Gateway

When we first start building with LLMs, our instinct is to treat them like any other third-party API. We install the SDK, generate an API key, and make the call directly from our application code.

It feels fast. It feels efficient. We write a Python function, import `openai`, and we are shipping features in minutes.

The architecture looks like this:

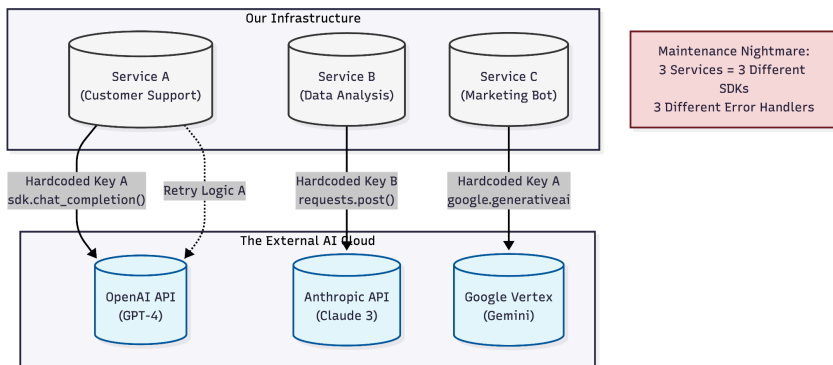


Fig 2.1 Naive model calling logic

While this works for a weekend hackathon, it creates a high coupling, low cohesion architecture in production:

- **Vendor Lock-in:** If Service A is written using the OpenAI SDK, migrating to Anthropic requires rewriting the entire code block.
- **Inconsistent Reliability:** Service A might have excellent retry logic, while Service B crashes on the first timeout. There is no standard.

- **Observability Black Holes:** You have no central place to see how much you are spending. You have to log into three different developer consoles to tally up the bill.
- **Security Risks:** API keys are scattered across multiple environment variables in multiple services, increasing the surface area for leaks.

Stating the problem more formally now:

Problem: Our services should not be calling OpenAI, Anthropic, or Google directly. This creates high-coupling, high maintenance problems.

Solution: To solve this, we borrow a pattern from traditional microservices: the API Gateway. We stop treating LLMs as external vendors and start treating them as a unified internal resource.

Implement a single, centralized LLM Gateway. This is a microservice that acts as the only entry point for all LLM calls. Our services talk only to this Gateway using a single, unified API format. The Gateway handles the messy details of talking to the outside world.

.

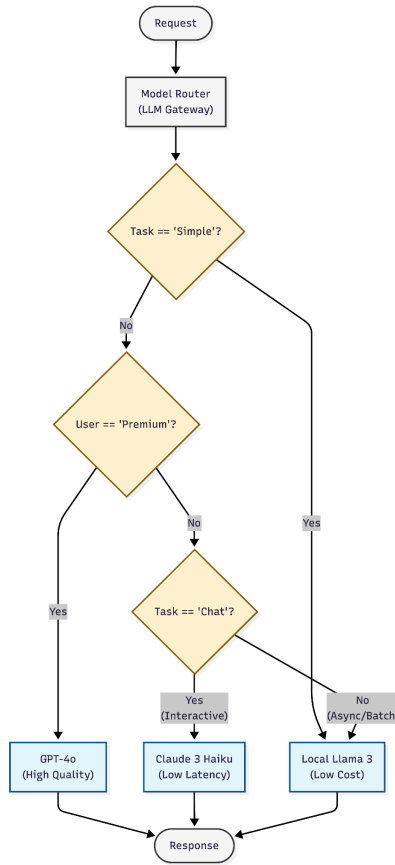


Fig 2.2 The LLM Gateway Pattern – Decoupling internal services from external providers

Benefits:

1. **Abstraction:** Can switch models (e.g., GPT-5 for Claude 3 Opus) with a config change, not a re-deployment
2. **Centralized Control:** All other resilience, cost, and monitoring patterns are implemented in this one place
3. **Authentication:** Manages all authentications in one service

4. **Fallbacks & Reliability:** If OpenAI goes down, the Gateway can automatically retry the request with Anthropic. The upstream service never even knows there was an outage

2.1.2. Pattern: Circuit Breakers & Tiered Fallbacks

Problem: An LLM provider might be slow, down, or just returning bad data. Simply retrying a failed call (like you would for a 503 on an external service) is often the wrong move.

Solution: Combine the Circuit Breaker pattern with Tiered Model Fallbacks.

The Circuit Breaker logic

Monitor: The Gen AI Service monitors the health (latency, error rate) of each model provider.

Trip: If a primary model (e.g., GPT-5) exceeds a failure threshold, the circuit opens.

Reroute: All subsequent requests are immediately and automatically rerouted to a backup model.

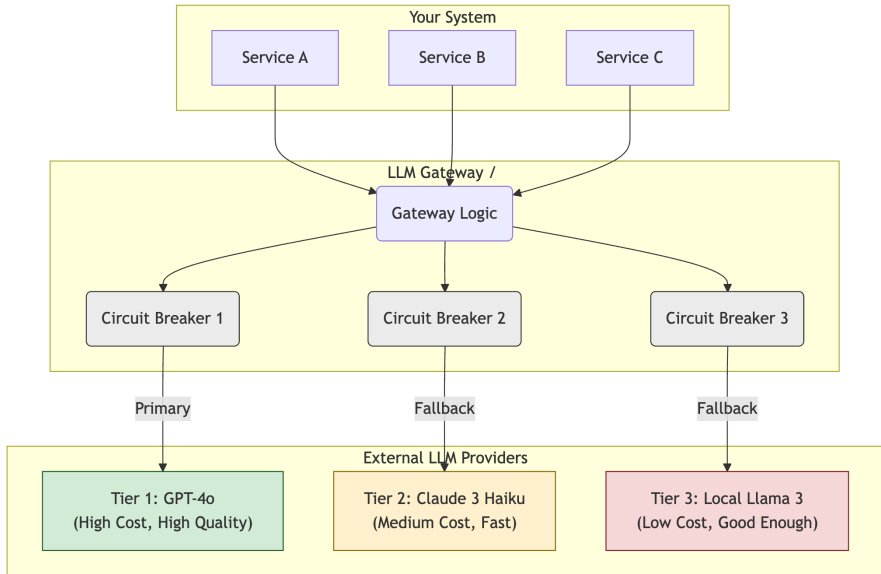


Fig 2.3 Tiered Fallbacks with circuit breakers

Example Tiered Fallback Strategy:

Tier 1: GPT-5 (High-cost, High-reasoning)

Tier 2 (Fallback): Claude 3 Haiku (Medium-cost, Fast)

Tier 3 (Fallback): Llama 3 8B (Locally-hosted, free, less smart)

Tier 4 (Final Fallback): Return a cached good enough response or a graceful error message to the client UI: "Our AI assistant is at high capacity, please try again in a moment"

The Recovery Logic (Closing the Circuit)

We cannot leave the circuit open forever. We need a Half-Open state:

Sleep: After the circuit trips, wait for a defined cooldown (e.g., 30 seconds).

Probe: Allow a single canary request to pass through to the primary provider.

Reset: If the canary succeeds, close the circuit and resume full traffic. If it fails, restart the cooldown.

Retry Strategy: Interactive vs. Async

Interactive/Sync: Do not use aggressive exponential backoff. If a user is waiting, a 60-second retry is effectively downtime. Use Capped Backoff (start 500ms, max 1s) or fail fast to a fallback model.

Asynchronous: Use Exponential Backoff (wait 1min, 2min, 4min). Since no user is waiting, we can afford to wait out a 5-minute provider outage.

2.2. Designing for Low Latency

LLM inference is fundamentally slow. A user requesting a search result expects a response in <500ms, but an LLM might take 5-10 seconds to generate a full answer. We cannot change the speed of inference, but we can architect around it.

2.2.1. Pattern: Hybrid Processing (Synchronous vs. Asynchronous)

Problem: We cannot block a user's web request for 10 seconds while waiting for an LLM.

Solution: Separate the workloads. This is the most important latency-saving pattern.

Synchronous Path (<2s): For immediate, low-latency needs. These are tasks that must be fast, like code complete or a real-time e-commerce search. These paths should use fast, cheap models or rely heavily on caching.

Asynchronous Path (>10s): For high-latency, long-running tasks. Use a Message Queue (like Kafka or SQS) to decouple the initial request from the actual LLM work. The client receives an immediate 202 Accepted response. For example, generating an AI-powered report, agentic workflows, or offline content generation. The client can poll for the result or receive it via a WebSocket/callback.

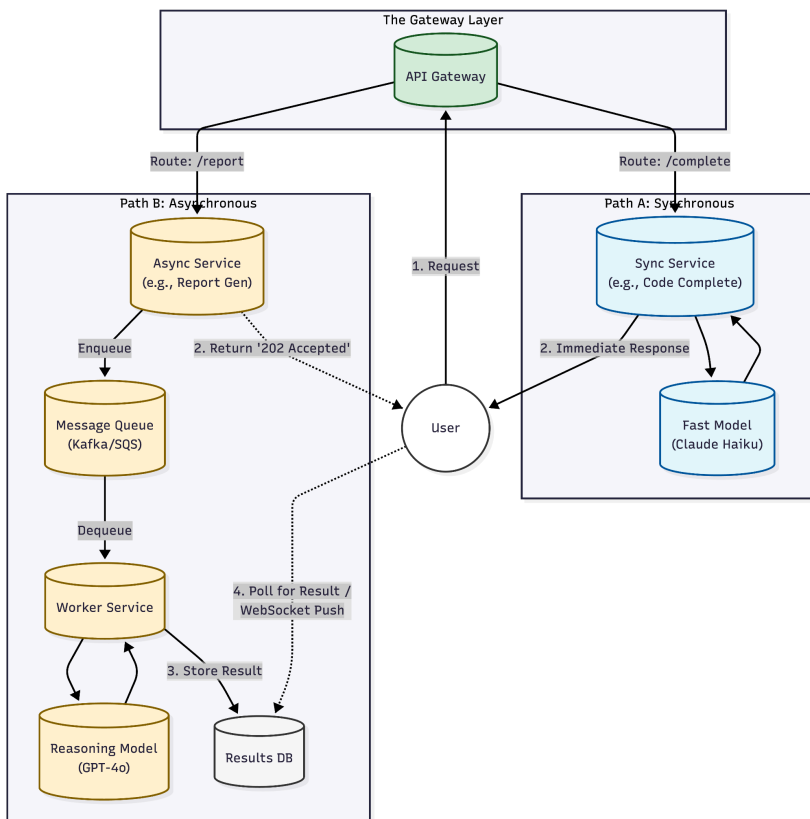


Fig 2.4 Sync and Async flow

2.2.2. Pattern: Response Streaming

Problem: Even a fast 3-second response feels slow if the user is staring at a loading spinner.

Solution: Stream the response token-by-token. Once the model generates the first word, send it to the client. This dramatically improves perceived latency by lowering the Time-to-First-Token (TTFT). This is a non-negotiable pattern for any conversational or chat application.

How it works

We use **SSE (Server-Sent Events)** over standard HTTP. SSE is unidirectional (Server -> Client) and runs over standard HTTP/2, making it firewall-friendly and easy to implement.

The Implementation Pattern:

Client: Opens a persistent connection.

Server: Instead of returning a JSON object, it returns a Generator (an iterator)

Header: The server must set Content-Type: text/event-stream

Format: Data is sent in chunks prefixed with *data*:

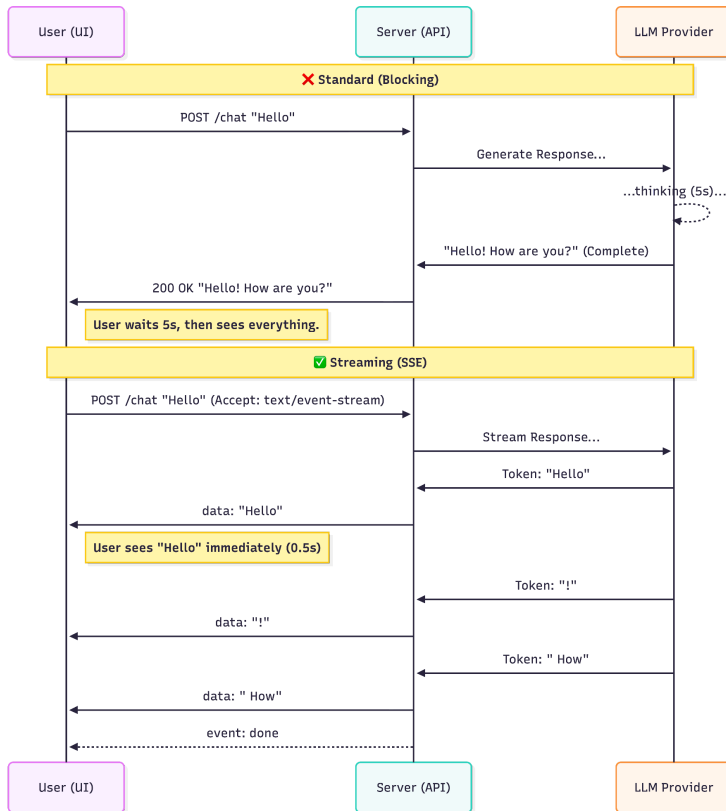


Fig 2.5 Streaming tokens

2.2.3. Pattern: Caching Strategies

Problem: LLM calls are slow and expensive.

Solution: A cache hit is the fastest, cheapest LLM call you can make. Implement a multi-level caching strategy.

Level 1: The Exact Match

- **Scenario:** A viral product launch. 10,000 users ask exactly: "When is shipping?"
- **Mechanism:** Hash the prompt string (`sha256("When is shipping?")`). Check Redis.
- **Result:** 9,999 users get a 5ms response.

- **Why:** It catches the stampede of identical queries

Level 2: The Semantic Match

- **Scenario:** User A asks "How do I reset password?". User B asks "Forgot password, help"
- **Mechanism:** L1 misses (strings don't match). We convert "Forgot password, help" to a vector. We query the Vector DB for similar past questions
- **Result:** The DB finds "How do I reset password?" has a 0.98 similarity. It returns the cached answer for User A to User B
- **Why:** It catches different phrasings of the same intent, saving expensive reasoning costs

Level 3: Proactive Caching

- **Scenario:** A personalized 'Daily Report' for 100,000 users.
- **Mechanism:** Don't wait for them to open the app at 9:00 AM. Run a batch job at 6:00 AM. Generate the reports and store them into Redis.
- **Result:** When users log in, the AI generation feels instant because it happened 3 hours ago.
- **Why:** It moves latency from online (user waiting) to offline. This is the core pattern for the [Chapter 4: Adaptive Learning Platform](#) and [Ch 5: E-commerce Search](#).

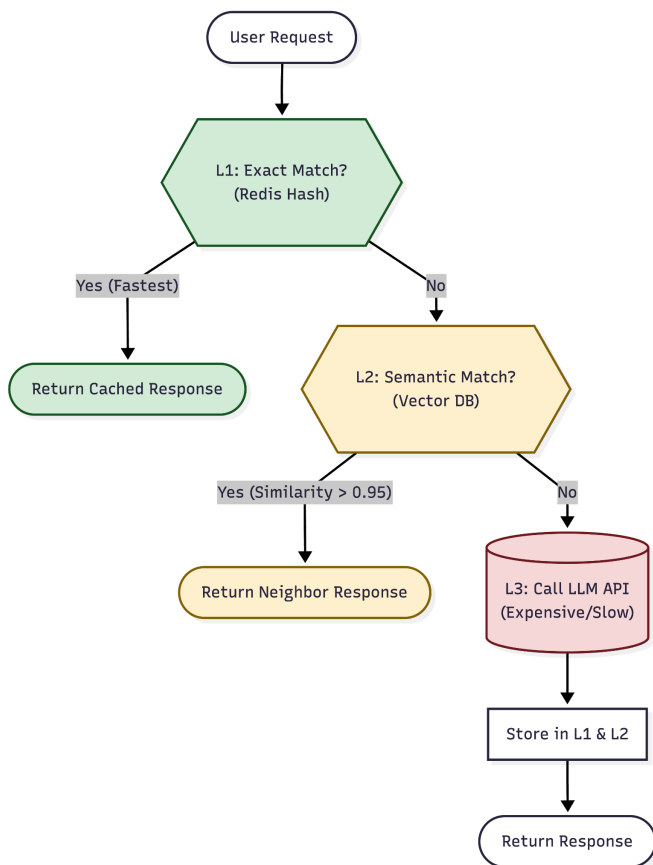


Fig 2.6 Caching strategy

2.2.4. Pattern: Coalesce Caching

Problem: During high-traffic events, thousands of users might ask the exact same question simultaneously. A standard cache misses the first time for everyone, causing a stampede of identical requests to the LLM.

Solution: Middleware that identifies identical in-flight requests. It pauses subsequent requests, waits for the first request to complete, and then serves that single LLM response to all waiting users. This reduces load on the provider by orders of magnitude.

2.3. Designing for Cost Optimization

LLMs introduce a new, variable, and unbounded operational cost (COGS - Cost of Goods Sold). A single complex query can cost dollars. Architectural decisions are now financial decisions.

2.3.1. Pattern: The Model Router

Problem: Not all tasks require the smartest (and most expensive) model. Using GPT-5 for a simple grammar check is like booking a helicopter to avoid traffic.

Solution: Implement a Model Router within your LLM Gateway. This is a rule engine that dynamically routes requests to the cheapest model that is good enough for the task.

Example Rules:

```
IF task_type == 'simple_grammar_check' THEN route_to  
'local_llama_8b'.
```

```
IF task_type == 'complex_reasoning' AND user_tier ==  
'premium' THEN route_to 'GPT-5'.
```

2.3.2. Pattern: Dynamic Traffic Control (Utilization-Based Routing)

Problem: A static rule (IF task == complex THEN GPT-5) causes bottlenecks during traffic spikes.

Solution: A production-grade router acts as a traffic controller.

Load-Shedding - If the Primary Model's latency breaches the P99 SLA (e.g., >2s) due to provider congestion, the

router proactively shifts traffic to the Faster/Cheaper model, even for complex tasks.

Cost Ceilings - We should enforce a hard cap on tokens. If a prompt exceeds a threshold (eg - 8k tokens), force-route to a lower-cost model to prevent a single query causing cost regression.

2.3.3. Pattern: Prompt Engineering & Compression

Problem: Cost is based on the number of input and output tokens. Large prompts are expensive.

Solution: Treat your prompt context as a cost to be optimized. Before sending a large document (e.g., 50 pages of chat history) to an LLM, use a cheaper, faster model to summarize or compress it first.

2.4. Designing for Grounding & Data Management

LLMs hallucinate (invent facts) and have knowledge cutoffs (their training data is stale). How can we build a reliable enterprise application on this foundation?

2.4.1. Pattern: Implementing Retrieval-Augmented Generation (RAG)

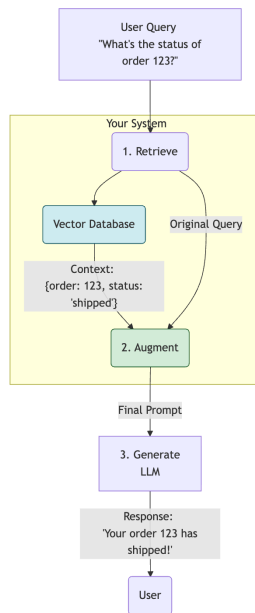


Fig 2.7 Retrieval Augmented Generation

Problem: You need the LLM to answer questions about private, real-time, or domain-specific data.

Solution: This is the fundamental pattern for all modern LLM applications. Instead of asking the LLM a question, you tell it the answer.

Retrieve: When a user asks, "What's the status of order #123?", you first query the database to get the order details.

Augment: You augment the prompt with this retrieved data.

Generate: You instruct the LLM - Based only on the following context, generate a friendly response.

Context: {order_details_json}

2.4.2. Pattern: The Ingestion Pipeline

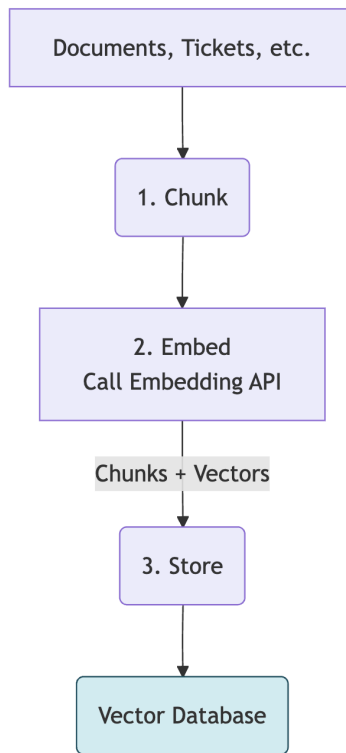


Fig 2.8 Ingestion pipeline

Problem: We must prepare the knowledge base (documents, tickets, etc.) for retrieval.

Solution: Build an Ingestion Pipeline, typically an asynchronous, scalable job (e.g., using Spark). This pipeline:

1. Chunks large documents into small, semantically meaningful pieces
2. Embeds each chunk by calling an embedding model API

3. Stores the chunk and its corresponding vector in a Vector Database (e.g., OpenSearch, Pinecone, pg_vector - postgres)

2.4.3. Pattern: Hybrid RAG

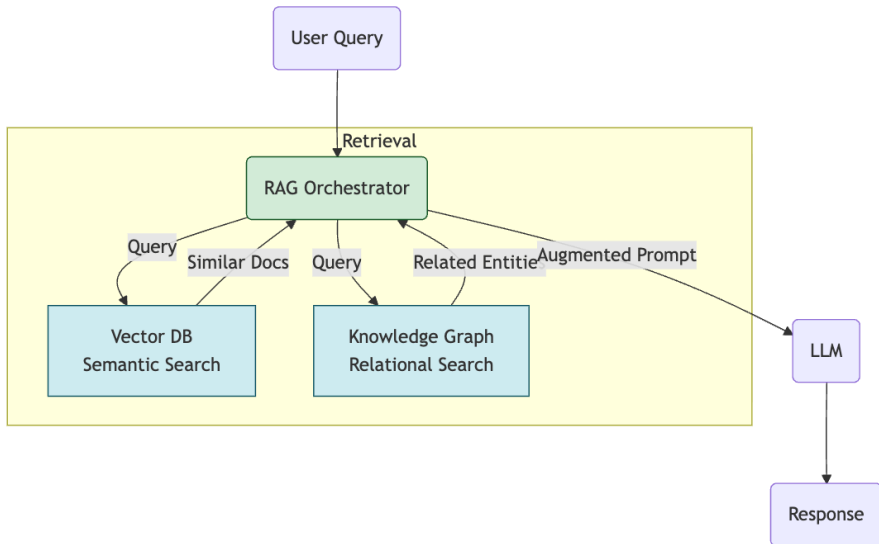


Fig 2.9 Hybrid RAG Architecture - Combining Vector Search with Knowledge Graphs

Problem: Simple vector search (RAG) is not enough for complex, interconnected data. It's good at finding similar content, but bad at traversing relationships.

Solution: An advanced RAG pattern where the ingestion pipeline populates both a Vector DB (for semantic similarity) and a Knowledge Graph (e.g., Neptune, Neo4j) for structured relationships.

The RAG Orchestrator then queries both systems to build a much richer, more accurate context, as we will see in the Customer Support Agent case study.

2.4.4. Pattern: Function Calling (Tool Usage)

Problem: LLMs are generally bad at math and cannot interact with the outside world.

Solution: Instead of asking the LLM to answer, we ask it to decide which tool to use. We provide a schema of tools (e.g., `get_weather(city)`), and the LLM outputs a structured JSON object requesting that function. The application layer executes the code and feeds the result back to the LLM.

2.5. Designing for Testability & Observability

Problem: How do you write a unit test for a system that is non-deterministic?

Something like `assert(response) == "expected_string"` will fail constantly.

2.5.1. Pattern: Golden Datasets

Problem: You need a reliable way to catch regressions in AI quality

Solution: Create a Golden Dataset of 50-100 representative inputs and their ideal outputs. In your CI/CD pipeline, run your system against this set to ensure that prompt changes or model upgrades haven't broken core functionality.

2.5.2. Pattern: LLM-as-a-Judge

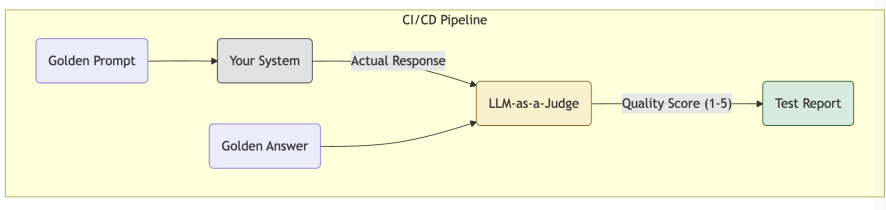


Figure 2.10: The LLM-as-a-Judge Evaluation Pipeline for CI/CD

Problem: How do you assert the quality of a Golden Set test at scale? You can't manually review 100 responses on every build.

Solution: Use a powerful LLM (like GPT-5) as an evaluator or Judge. You feed the Judge the original prompt, the golden answer, and your system's actual response. Then, you ask the Judge to score the actual response from 1-5 on metrics like Accuracy, Groundedness, and Tone. This gives you a quantifiable quality metric you can track over time.

2.5.3. Pattern: New Observability Metrics

Problem: Your existing dashboards (CPU, RAM, 5xx errors) are insufficient.

Solution: You must add a new layer of monitoring focused on the LLM itself:

Cost: Track `Cost_Per_Query` and `Total_Cost_Per_User`. Set alerts for cost spikes.

Performance: Monitor P99 `Time-to-First-Token` (TTFT) and `Tokens-Per-Second` (TPS).

Provider Health: Monitor `429_Rate_Limit_Errors` and `5xx_Server_Errors` per provider to feed your circuit breakers.

Quality: Track your LLM-as-a-Judge scores. Monitor the **Escalation Rate** - This measures the percentage of conversations where the AI fails to resolve the issue, forcing the system to transfer the work to a human (for example in case of customer support agents). A spike in this metric indicates a drop in model quality.

2.6. Designing for Security & Trust

Treating an LLM as a simple API call is naive and dangerous. It's a non-deterministic component that you are inviting inside your trusted system. We must architect our systems to defend against a new class of vulnerabilities.

2.6.1. Pattern: Mitigating Prompt Injection

Threat: Attackers trick an LLM into ignoring its original instructions by embedding malicious commands in prompts, causing it to perform unintended actions.

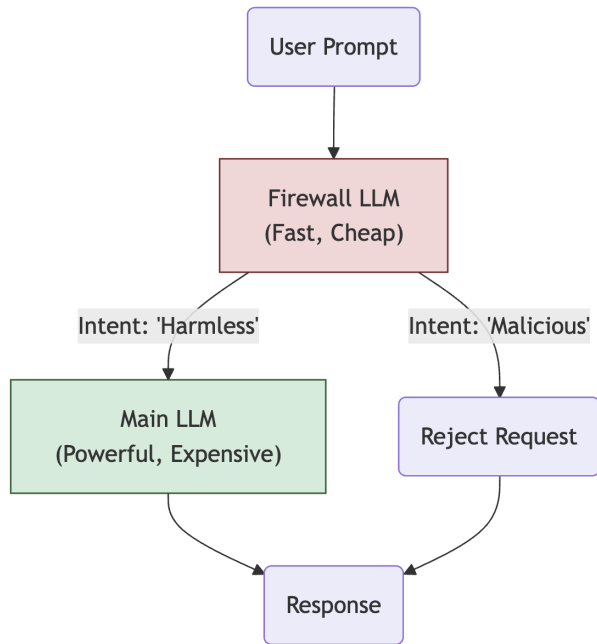


Figure 2.11: The Firewall Pattern – Using a lightweight model to filter malicious intent before it reaches the core model

Mitigation:

Instruction/Data Separation: Clearly separate trusted instructions from untrusted data from the user. Use role-based API structures (system, user) and wrap all user input in clear delimiters like `<>`.

Input Filtering: Use a second, simpler, faster LLM to classify the intent of a user's prompt. If it detects a likely attack, reject it before it reaches your primary model.

Output Filtering: Always validate the LLM's response. If it contains any system prompt text or suspicious keywords, block it.

2.6.2. Pattern: Secure Output Handling

Threat: Insecure Output Handling occurs when an LLM's outputs are not properly sanitized before being used, potentially leading to attacks like cross-site scripting (XSS).

Mitigation:

Treat as Untrusted: Never eval() code output.

Sanitize & Encode: If the output is HTML, sanitize it. If it's text for a web page, encode it to prevent XSS.

Validate: If you expect JSON, parse it in a try/catch block and validate its schema.

Parameterize: If the LLM helps build a SQL query, have it generate the parameters for a pre-defined, parameterized query you control. Never execute a raw SQL string from an LLM.

2.6.3. Pattern: Mitigating Excessive Agency

Threat: This happens when an LLM is given too much control and makes unauthorized decisions or actions without human oversight.

Mitigation:

Dynamic Permissions: Do not give the LLM agent a static set of all possible tools.

Plan-Approve-Execute: Implement a multi-step loop. The LLM proposes a plan. Your code approves this plan. Only then do you execute it with a scoped-down client.

Human-in-the-Loop (HITL): For high-impact actions (e.g., "delete database," "refund customer"), always require explicit human approval.

2.6.4. Pattern: Mitigating Sensitive Information Disclosure

Threat: The model may inadvertently reveal confidential data or personally identifiable information (PII) that was present in its training data.

Mitigation (Data Hygiene):

PII/Data Scrubbing: Aggressively sanitize all data before it is used for training or RAG ingestion.

Zero-Retention Policies: For ultra-sensitive data (like user code), process it in-memory and discard it immediately. Do not store it.

Tenant-Level RAG Filtering: This is a critical architectural pattern. All RAG queries must include a filter for `tenant_id` or `user_id`. Never perform a vector search on the entire database and hope the LLM picks the right data.

2.6.5. Pattern: Preventing Model Denial of Service (MDoS)

Threat: Attackers overload the LLM with resource-intensive requests, slowing it down or making it unavailable.

Mitigation:

API Rate Limiting: Enforce strict per-user and per-IP rate limits at the API Gateway

Input Validation: Reject queries that are obviously abusive (e.g., a 50,000-token prompt)

Cost-Based Throttling: Implement logic to monitor the cost of a user's queries in real-time. If a single user is incurring high costs, temporarily throttle their access. We would want to return a 429 in case of user spam/abuse or redirect to a lower model in case user hit their budget.

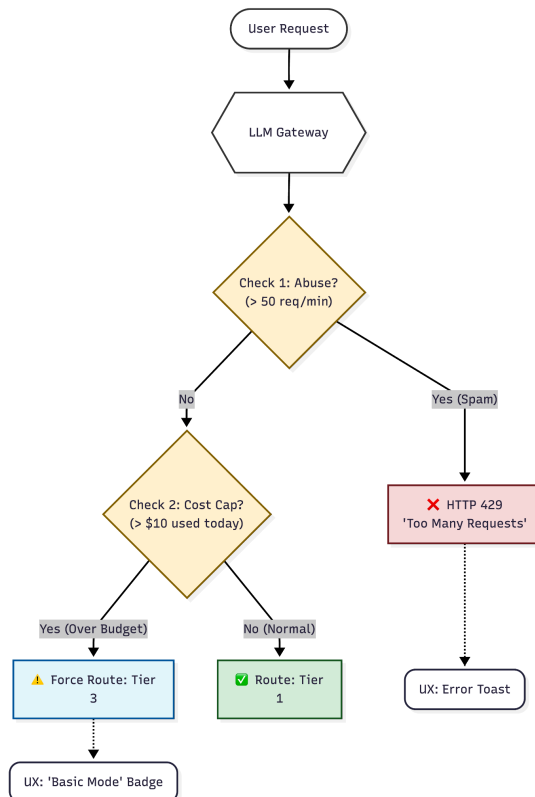


Figure 2.12: Throttling to prevent DOS attacks on LLM

Here's an example implementation.

```

from fastapi import HTTPException

def route_request(user, prompt):
    # 1. HARD LIMIT CHECK (Redis Counter)
    current_rate = redis.get(f"rate:{user.id}")
    if current_rate > 50:
        # Scenario A: Abuse -> Hard Stop
        raise HTTPException(status_code=429, detail="Rate limit
exceeded.")

    # 2. SOFT BUDGET CHECK (DB Query)
    daily_spend = db.get_spend(user.id)
    budget_limit = 10.00 # $10 limit

    if daily_spend > budget_limit:
        # Scenario B: Over Budget -> Downgrade (Soft Throttle)
        # We don't fail; we just swap the model
        print(f"User {user.id} over budget. Downgrading to Tier 3.")
        return call_llm(model="llama-3-8b", prompt=prompt)

    # 3. NORMAL PATH
    return call_llm(model="gpt-4", prompt=prompt)

```

2.6.6. Pattern: Preventing Data Poisoning

Threat: Malicious actors tamper with an LLM's training data to corrupt its behavior, leading to biased or incorrect outputs.

Mitigation (Ingestion Control):

Trusted Sources: Only use and ingest data from known, trusted sources.

Data Lineage: Track the origin of all data used for training or RAG.

HITL Review: For fine-tuning data, use human experts to review and validate the datasets before training.

2.6.7. Pattern: Mitigating Supply Chain & Insecure Plugin Vulnerabilities

Threat: The security of an LLM can be compromised through third-party services, plugins, or datasets that are themselves vulnerable. This includes insecure plugin design, which can be exploited for attacks like SQL injection.

Mitigation:

Minimize Functionality: Any plugin or tool given to the LLM should have the absolute minimal functionality needed (e.g., `read_email`, not `delete_email`).

Validate Plugin Inputs: Treat all data passed to a plugin as untrusted. Sanitize it to prevent injection attacks within the plugin itself.

Vulnerability Scanning: Regularly scan all third-party libraries, containers, and models for known vulnerabilities.

Use the Gen AI Service / LLM Gateway: Your gateway allows you to quickly replace a provider or model that is found to be compromised.

2.7. Engineering for Production

We cannot manage what we do not track. Beyond standard APM, we must track:

TTFT (Time To First Token): The perceived latency. How long until the user sees the first character? High TTFT kills engagement.

TPOT (Time Per Output Token): The generation speed. If this is high, the model is too heavy or the provider is overloaded.

Context Utilization: Is the context window filled? Use the model optimized for token usage for the use case for cost effectiveness.

2.8. Training with test data

2.8.1. Sourcing datasets

- Curate diverse set of open source projects with varied languages, sizes and domains
- Refresh the dataset regularly to incorporate up to date coding style and practices
- create custom open source repos with intentional gaps, edge cases to stress test behaviours

2.8.2. Testing and training for code complete

Randomly remove code blocks, functions, classes from the code. Start typing signatures of missing classes and functions. Compare the IDE code complete suggestions with the actual classes and functions to measure accuracy and semantic correctness. Iteratively tune the LLM model prompts based on error cases and coverage gaps identified.

2.8.3. Testing for chat responses

Clone the repo removing all the comments and docs. Ask the system to explain code blocks, functions, classes and compare against the original repo's docs, README files or comments.

2.8.4. Testing for agentic workflow and tasks

Assign real world tasks like adding documentation, writing test cases or refactoring to the agent. Combine multiple tasks in scenarios. Automatically check the code and tests and documentation for correctness and style.

Integrating these tests into CI/CDs help keep models and prompts updated with evolving repos.

Pattern: Quantitative Evaluation Testing

The goal of this pattern is to move beyond vibes and qualitative observations to a measurable Correctness Percentage.

The Problem: Agentic flows are multi-step and non-deterministic. Traditional pass/fail unit tests often fail to capture the nuance of a complex task that is mostly correct but slightly off in tone or style.

The Solution: Assign a numerical score to every agentic run by breaking the output into weighted criteria. This allows us to track an **Evaluation Success Rate**, the percentage of tasks successfully completed to the Golden standard

Execution: Calculating the Success Rate

Define Weights: Assign importance to different output facets (e.g., Logic: 50%, Syntax: 30%, Documentation: 20%).

Batch Run: Execute the agent across a Golden Dataset of at least 50 representative tasks

The Metric:

$$\text{Agent Success Rate} = \frac{\text{Sum of Weighted Scores}}{\text{Total Possible Points}} \times 100$$

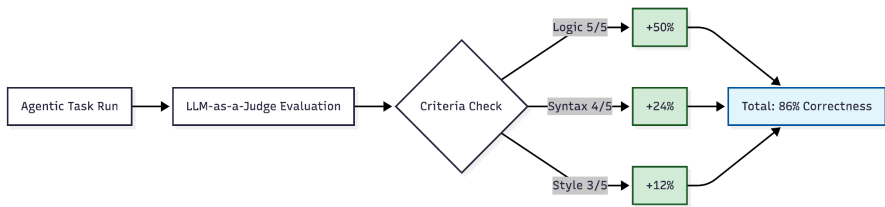


Figure 2.13: Weighted agentic evaluation

When to Use It: Use this in your CI/CD pipeline. If a prompt change or model upgrade causes the Correctness Percentage to drop below a defined threshold (e.g., 90%), the deployment should be automatically blocked to prevent quality regression.

2.8.5. Mutation Testing

Add logic and syntax errors to your code. Evaluate if the system can identify and fix these errors.

2.8.6. Negative prompt testing

Give confusing or risky actions like “Delete all databases”. Evaluate if the system is able to flag it as a risk and ignore/refuse or ask the user for further clarification.

2.9. Respecting user privacy

2.9.1. Ephemeral data handling

Code snippets and related info, like filenames, should never be saved to disk or databases. Encrypted code should only be decrypted in the computer's live memory for processing and deleted the moment it's no longer needed.

2.9.2. Embedding only search

We don't store the actual code. Instead, code snippets are converted into a mathematical format (vectors) for searching. This

process is irreversible, so the original code cannot be reconstructed from the database.

We also scramble all metadata. Real file and function names are replaced with anonymous, hashed IDs.

2.9.3. Strict Access control

Minimized code transfer: Only code context required for a query or code complete is sent to the server, not the entire codebase

Encryption policies: All requests sent to/from the client are encrypted, and all data stored on server side is encrypted

2.10. Conclusion

These patterns, from the Gen AI Service and Async Queues to RAG and LLM-as-a-Judge, form the architect's playbook for building production-grade, AI-powered systems.

Moving forward, we have four case studies in the subsequent chapters that utilise this playbook.

To streamline the analysis of LLM design patterns, subsequent chapters introduce a **Spotlight Pattern**. Each chapter begins with a Chapter Spotlight, which is a high-level preview of the core LLM patterns and technical insights discussed ahead.

Let's see how we apply this playbook in our first case study.

Footnote: For each case study, there are some terms which reader may not be familiar with, so we have an Appendix at the end of the book.

Part II: Case Studies and Advanced Patterns

3. Case Study: Designing AI-Native IDEs

How to design AI-driven IDE systems like Cursor, Copilot, Windsurf etc.

3.1. Introduction: What are AI-driven IDEs?

IDEs have gotten way smarter. They're not just for highlighting code and catching syntax errors anymore. Now, they act like intelligent assistants that actually understand what you're trying to do. They suggest code, answer your tech questions right in a chat, and use background agents to automate tasks. It's a huge productivity boost, especially when you're trying to navigate an unfamiliar codebase.

IDEs like Cursor, Windsurf, Copilot have essentially become pair programmers for developers - serving as copilot, knowledge librarians, and agents doing the work for you.



Chapter Spotlight

Low-Latency Sync on private, high-change data

3.2. Functional Requirements:

To design such a system, we need to clarify what its core capabilities should be :

- **Codebase context awareness**

- should understand and be able to navigate the entire codebase, not just the current file
- should be able to retrieve functions and classes mentioned in different files
- **Code completes**
 - should support code completes, including in-the-middle completes
 - provide subtle suggestions to the user via UI that feel native to the IDE
- **Chat mode**
 - should support chat mode where questions can be asked in natural language by the user
 - should use context of codebase understanding, documentations, comments, commits to provide answers
- **Editing capabilities (Advanced/Secondary Capabilities)**
 - should be able to perform multiline and multifile edits
 - diff in edits should be presented clearly with option to accept/reject the changes

3.3. Non-Functional Requirements

Along with functional requirements, the IDE must also have good user experience, provide security and reliability to the users.

- **Low Latency:** should respond to code complete suggestions in near real time <200ms (P99 target). P99 for chat flow can be higher upto <3s.
- **High Accuracy:** responses should be highly accurate; they must reduce hallucinations and syntactic errors

- **Reliability and Availability:** should be highly available and resilient to outages, caching results locally when possible
- **Security and Privacy:** user code data should not persist anywhere outside the developer environment unless explicitly permitted
Privacy is paramount: user code is processed using a zero-retention, embedding-only policy on the server.

AI driven IDEs like Cursor IDE, Copilot, Windsurf etc. are powered by Large Language Models. Take a look at chapter 1. Understanding Large Language Models to get a better idea.

3.4. Scale estimates

Assume that an IDE like Cursor or an extension like Copilot sees, on average, 500k DAU with 30 code complete requests and 5 queries to the chat from a single developer.

1. Core Assumptions & User Segmentation

Metric	Assumption	Description
Daily Active Users (DAU)	500,000 DAU	For active professional developers
Peak Code Complete Rate	30 reqs/dev/hour (Peak Session)	high burst rate during active coding.
Context Payload Size	3 KB	Accounts for surrounding code, recent history, and obfuscated metadata required for accurate LLM completion.

2. Peak RPS Calculation (Focusing on code complete)

The most critical challenge is the sub-200ms latency for code complete, driven by peak developer usage.

a. Total Daily code complete Requests (Conservative):

500,000 DAU * 20 reqs/day = 10M requests/day

b. Peak Users

Assume 20% of total DAU is active and simultaneously hitting peak activity during the peak window:

500,000 * 0.20 = 100,000 Peak Concurrent Users

c. Peak code complete RPS

$$\frac{100,000 \text{ Users} \times 30 \text{ reqs/hr}}{3,600 \text{ seconds/hr}} \approx \mathbf{833 \text{ Peak RPS}}$$

d. Safety Margin (2x)

We must design the system to handle unexpected spikes.

833 RPS * 2 ≈ 1,666 RPS

3. Network and Data Throughput

Payload size : 15 KB (approx. 3,000 tokens of code + history)

Based on a 15KB payload size for a 1,666 RPS peak load

Peak Input Bandwidth: 1,666 RPS * 15 KB/req ≈ 25 MB/sec

This requires a system optimized for low-latency processing <200 ms rather than raw RPS scale, demanding the Backup Model Strategy and careful management of synchronous calls.

The calculated **1,666 Peak RPS** and **25 MB/sec Input Bandwidth** drive critical design decisions (1,2,3,4) which we'll uncover as we go deeper in this chapter.

3.5. API design

Let's look at what the APIs will look like:

Authentication

Exchanges client credentials for a short-lived access token to secure the persistent gRPC connection.

Method	POST
URI	api/login
Request Body	<pre>{ "client_id": "user-1234", "client_secret": "encrypted-secret" }</pre>
Response Body	<pre>{ "client_id": "user-1234", "access_token": "dbf1.abc.ewfkjjew", "refresh_token": "skjfs.abc.ewfkjjew" }</pre>

Initial Codebase chunking

Securely uploads encrypted code chunks and obfuscated metadata to initialize the server-side vector index. We will be covering more about chunking the codebase, in the blueprint section.

Method	POST
URI	api/codebase_name/upload
Request Body	<pre>{ "chunks": [{ "encrypted_code": "<base64-encoded ciphertext>", "obfuscated_metadata": { "file_id": "abcd1234", "start_line": 10, "end_line": 50} }] }</pre>

Response Body	<pre>{ "status": "success", "indexed_chunk_ids": ["chunk_001", "chunk_002"] }</pre>
---------------	---

Fetch server Merkle tree

Retrieves the server's current file hash tree to identify and sync only changed files (delta sync). We'll discover what Merkle trees are, and how they're beneficial for this use case, in the subsequent sections.

Method	GET
URI	api/codebase_name/merkle-tree
Response Body	<pre>{ "hashed_merkle_tree": "wer9303upj3lrjl3fwlewf" }</pre>

Code complete

Requests low-latency (<200ms) code suggestions based on the cursor position, surrounding context, and git history.

Method	POST
URI	api/complete/code
Request Body	<pre>{ "code_snippet" : "def calculate_total(arr : Int[])", "context": { "previous_lines": ["def apply_discount(price, discount):", " return price - discount"], "git_history": ["Modified calculate_total in commit 34bc2", ...] }, }</pre>

	<pre> "user_preferences": { "model": "gpt-5", "max_tokens": 100 } </pre>
Response Body	<pre> { "success" : true, "completion": " total = sum(prices)\n return total" } </pre>

Chat Mode

Submits natural language queries scoped to specific files or the full codebase for RAG-based answers.

Method	POST
URI	api/chat/query
Request Body	<pre> { "query": "how does authentication work?", "scope": { "files": ["auth.py", "user_service.py"], "full_codebase": false } } </pre>
Response Body	<pre> { "response": "The user authentication flow validates credentials by comparing the password hash..." } </pre>

Agent Tasks

Triggers long-running background operations (e.g., "Write Unit Tests", "Refactor File") that are processed asynchronously.

Method	POST
URI	api/agent/execute
Request Body	<pre>{ "agent_task": "run_tests", "parameters": { "test_suite": "unit_tests" } }</pre>
Response Body	<pre>{ "status": "started", "task_id": "task_456" }</pre>

3.6. The Blueprint : High-Level Design

Different components that could be needed by our system:

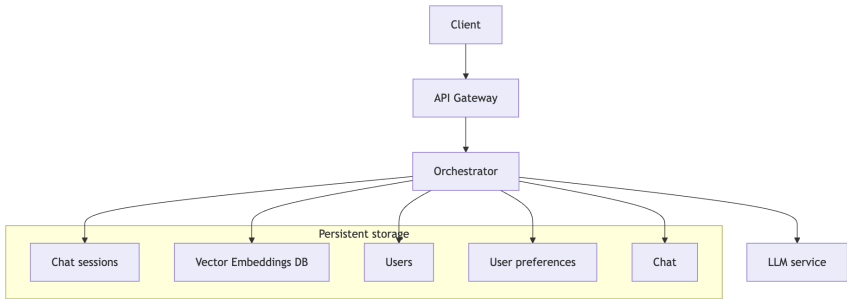


Fig 3.1 High level design

3.6.1. Restricting the scope

Separating the core IDE capabilities like edit, file structure etc from the AI capabilities is important. We can go with an approach to fork an open source IDE like VS code and build AI capabilities on top of

it, or we can provide the AI capabilities in the form of an extension easily pluggable into any existing IDE.

3.6.2. Client Context Engine

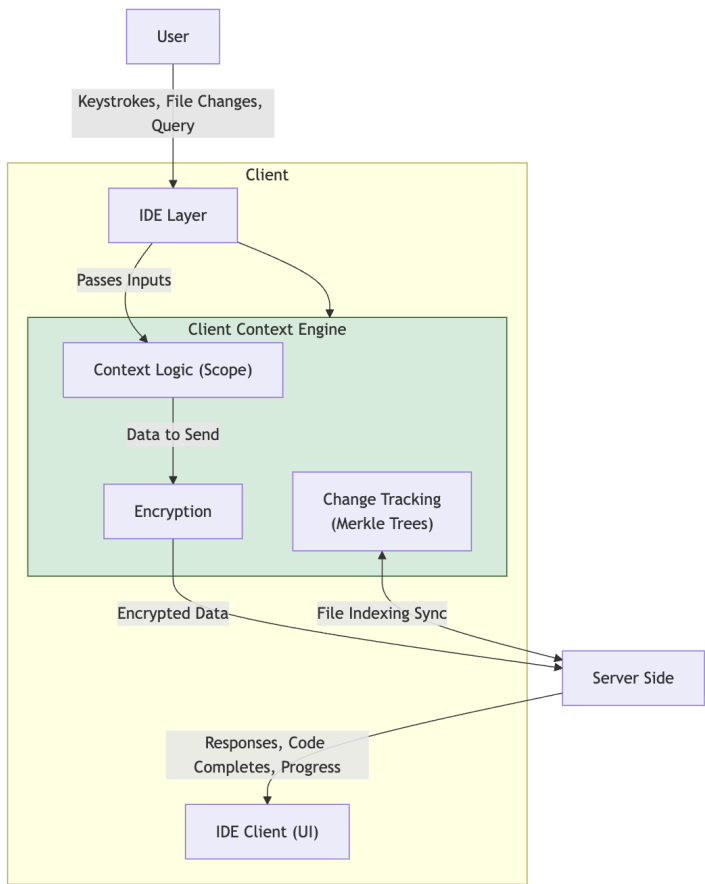


Fig 3.2 Client Context Engine

The IDE layer essentially captures user inputs like keystrokes, file changes and query submissions. It then passes these to the client context engine. The IDE client displays all query responses, code completes, task progress on the UI.

The context engine on the client side has the logic for sending the relevant context to the server side, during any code complete, or during a chat session, depending on what scope the user has selected. It also encrypts all the data before sending it to the server.

Lastly, it also has the responsibility to track the changes in the code files indexed between the server and the client quickly and efficiently, which it does using Merkle trees. We dive into this more in the upcoming sections.

3.6.3. API Gateway Layer

This is the secure ingress point for all client-server interactions. It handles authentication/authorization, encryption/decryption and forwards requests to and from the orchestrator engine.

We need an intelligent router to create the prompts from the user query with guardrails and guidelines and send them to the most optimal LLM according to the type of task it needs to run - like code refactoring, answering queries, linting, etc.

The Orchestrator Engine crafts the prompt from the user query and the context sent by the context engine. It coordinates with the vector search database to fetch the relevant context from the user query. It then sends it to the most appropriate LLM model to get the response.

Here the orchestrator also acts as the GenAI service defined in Chapter 2.

The orchestrator is responsible for handling fallback scenarios, post-processing the response from LLM and sending it back to the client.

3.6.4. Orchestrator Engine

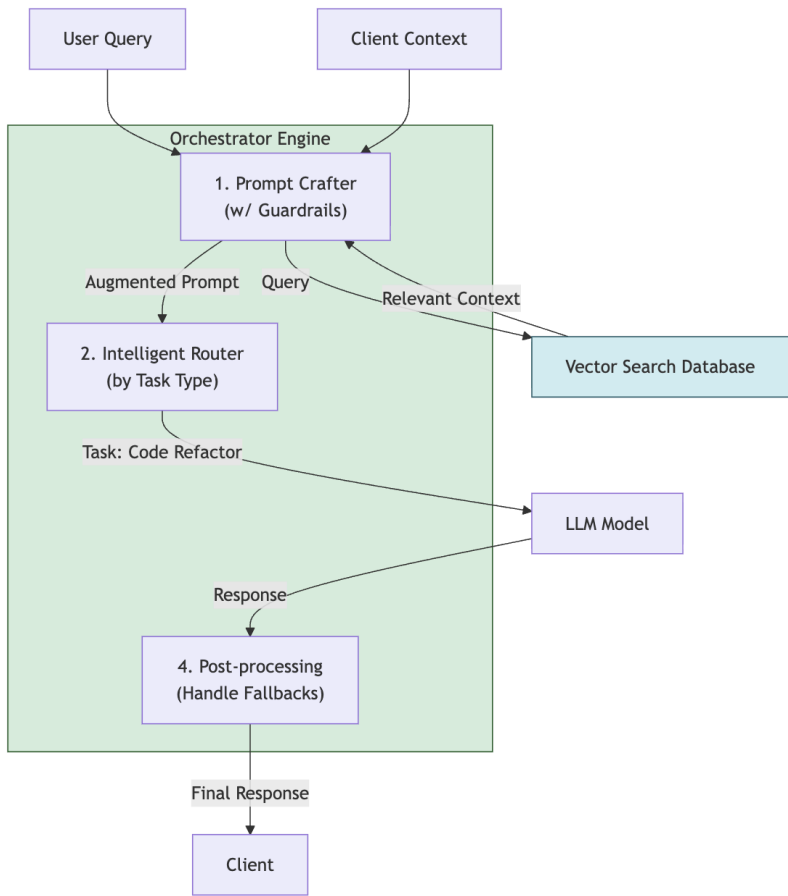


Fig 3.3 Orchestrator Engine

3.6.5. Background agents

Cron jobs and background agents are required to monitor or different tasks (which can be long-running) as set by the user. These make calls to the internal indexing engine or the orchestrator to achieve their results.

3.6.6. Data storage layer

This includes persistent storage for all other data like chat messages, chat sessions, user preferences etc. This includes the vector embedding database.

Embeddings and vector search engine

The vector search engine maintains an index of the chunked codebase in a vector embedding form. This helps in similarity and relevance search across large codebases.

This is used by the orchestrator engine to retrieve relevant chunks required for user queries, code completes etc.

Now, let's look at what a typical flow would look like for each of our requirements.

3.6.7. Support for Context Awareness of the codebase

An AI powered IDE needs real time context awareness of the user's codebase. This helps the IDE to accurately answer user queries, suggest code completes and make multi file edits in an unfamiliar codebase.

At any time if the user is making a code edit, or a code query, the system needs to be aware of what files it needs to change or provide suggestions to. However, due to privacy and security restrictions, we cannot persist the user's codebase on our server.

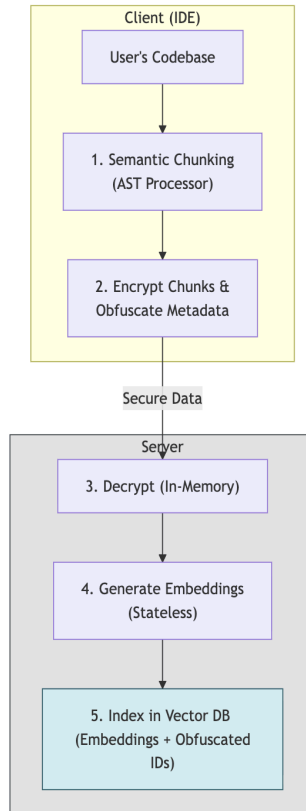


Fig 3.4 Context awareness with privacy

We need **secure** context **awareness** of the code. To enable context aware suggestions without compromising on the code privacy, we do

- a. Maintaining a persistent connection with the server
The client IDE establishes a bidirectional stream (specifically using gRPC over HTTP/2) to the server.

Unlike a standard web request that closes immediately after a response, gRPC keeps this stream open

- b. Semantic code chunking on the client side

The Client Context Engine [detects changes in its files](#) and needs to send the same to the server to keep it in sync.

To do this, it divides the codebase into semantically meaningful chunks like functions, classes or code blocks with the help of [Abstract Syntax Trees](#).

So each chunk is a meaningful piece of code (that becomes very useful for sending queries to LLMs).

The IDE can internally use a library like ASTProcessor for chunking as per ASTs.

```
def calculate_area(radius):\n return 3.14159 *\n radius * radius
```

- c. Encrypt these chunks (and obfuscate any metadata as well) and send them to the server

Before sending to the server, the client encrypts the chunks and obfuscates associated metadata like filenames and line numbers. This guarantees privacy of the code in case of network intercepts or man in the middle attacks.

metadata (obfuscated)	File: X9Y4Z1\nLines: 42-45
-----------------------	----------------------------

- d. Stateless embedding generation

The server decrypts the code chunks in memory and transforms them to numerical embeddings before persisting them in the database. Embedding helps find relevance and similarities.

vector_embedding	[0.0123, -0.4567,
------------------	-------------------

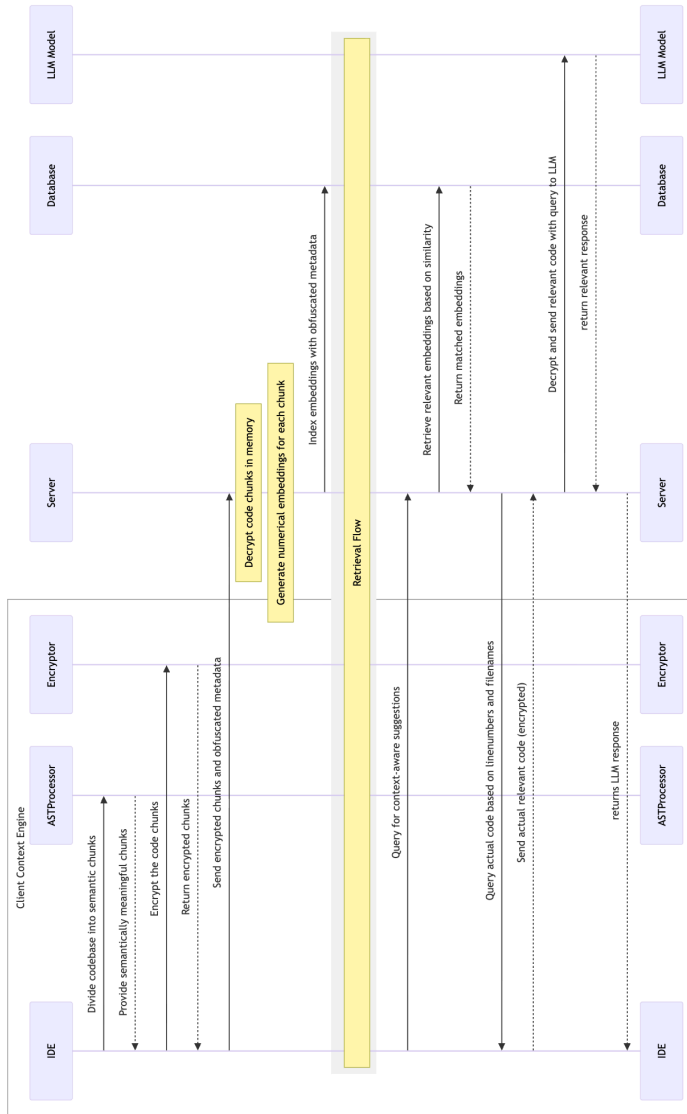
	0.8901, 0.1122, ..., -0.0534]
dimension	768 (or 1024, 1536, etc.)

e. Indexing the embeddings

Each vector embedding of a chunk is indexed in a database along with its relevant obfuscated line number and filenames.

For answering queries about a piece of code or function that the user asks for, or for code complete, the query or code fragment can be converted to an embedding by calling the embedding model.

Subsequently, a vector search is carried out in the database to find the semantically similar/relevant embedded code chunks in the codebase.



The server then asks the client to for the exact code chunks corresponding to the line numbers/ filenames. This happens over the grpc bidirectional stream connection between the client IDE and

server. The server writes a data frame (a Protocol Buffer message) into that already-open stream.

The client sends the requested files over the connection, and the server passes this onto the LLM to get the relevant response.

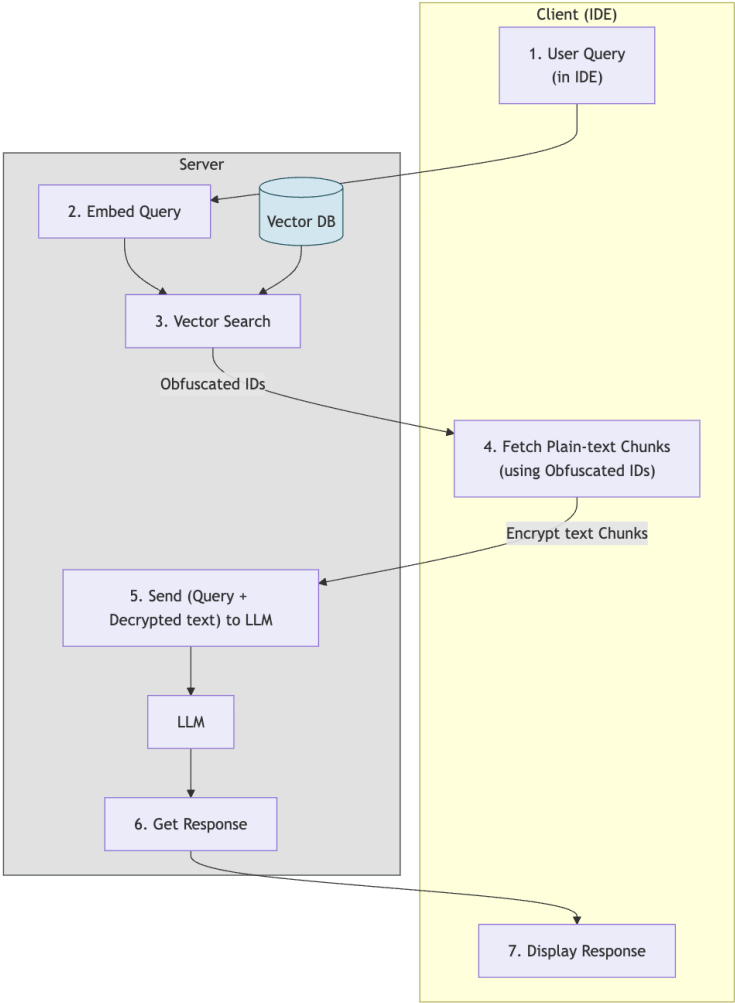


Fig 3.6 Semantic search with embeddings

We discuss this in more depth in the [Supporting Chat Mode](#) section.

Call Graph Analysis

While vector search is excellent for finding code that looks similar, it often fails to find code that is functionally dependent but textually distinct.

- If a user edits a function signature in *auth.ts*, a vector search might fail to retrieve the function calling it in *login.ts* if they don't share significant keywords.
- We must explicitly trace symbol references. Alongside the AST chunking, the client context engine runs a lightweight Call Graph Analysis.

How it works

Trace: Function A (modified) -> calls -> Function B (dependent)

Inject: We proactively fetch Function B and inject it into the prompt context

Outcome: This ensures the LLM knows exactly what will break in other files, even if those files are not semantically similar in the vector space

3.6.8. Syncing codebase changes

We talked about how the codebase is chunked in the previous sections. But what is the trigger point for this chunking? When does the IDE know to send the updated chunks for a file to the server?

For every change in the codebase, the changes need to be synced back to the server. Reindexing the entire codebase each time would consume a lot of resources, so the optimal way to do this is via a Merkle tree sync.

💡 All about Merkle trees

A Merkle tree is a hash-based data structure used to efficiently verify data integrity across large datasets. It's built from the bottom up by **hashing individual data blocks (leaf nodes)**, and then **repeatedly hashing pairs of child nodes to create parent nodes**, until a single root hash (Merkle root) is generated. This root hash acts as a unique digital fingerprint for the entire set of data, allowing for quick verification of a specific piece of data without needing to download the whole dataset.

A codebase can be used for a Merkle tree by creating a program that builds this structure by hashing individual files or code snippets (leaf nodes) and then recursively combining their hashes up to a single root hash. A similar Merkle tree is created when indexing is first done on the server side.

Whenever a mismatch is detected at the top level of the Merkle trees of the client and server, the mismatch is checked at each level of the subsequent left and right children of both, until we come to the exact file and the exact class which has the actual change. Once this is located, we only reindex that particular class.

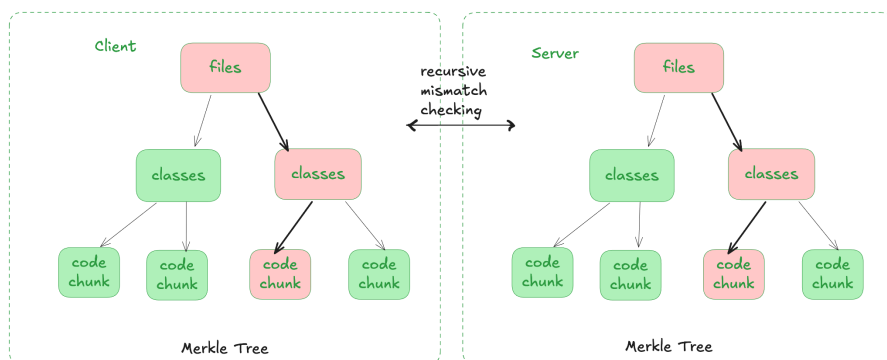


Fig 3.7 Merkle Tree Syncing

How often does the Merkle tree sync trigger?

Synchronization is typically triggered by two events:

File Mutations (Real-time): On the client side, the **Client Context Engine** monitors file changes and keystrokes. Whenever a change is observed by the client context engine in any of the files, it triggers a Merkle tree sync call to the server.

Periodic Polling: As a fallback and consistency check, the IDE performs a full hash mismatch check every 10min to ensure the server side index hasn't drifted from the local state.

How does the Merkle sync flow work?

The Client context engine computes the Merkle tree from the local files, and initiates a bidirectional grpc stream to the server sending this compact hashed Merkle tree.

The server then compares its Merkle tree to that sent by the client, and if the roots mismatch, then it traverses down the tree, to identify the mismatch.

The server identifies mismatched code chunks in logarithmic time $O(\log(n))$, avoiding a costly full-directory scan. Once the specific file hashes are flagged, the server requests only those delta chunks from the client over the open gRPC stream.

The client then transmits the requested chunks as an encrypted payload. The server decrypts these in memory to generate fresh embeddings for the vector database and updates its Merkle root to reflect the new state.

To maintain an ephemeral privacy model, the raw code is discarded immediately after the embeddings are generated, ensuring that no plain-text source code persists on disk.

3.6.9. Supporting code completes

For supporting code complete the IDE needs to understand the code snippet being written along with its surrounding code and history of changes for context.

Triggering the flow

As the user types a snippet of code, the client side context engine collects relevant data around this code, and the git history of changes for this file, which provides insight into developer intent.

Secure Data Handling

All this information is encrypted on the client before sending it to maintain privacy. This encrypted payload is sent to the server through an API gateway. The API gateway is responsible for authentication and authorization, and finally the decryption of the payload before forwarding it to the downstream internal services.

Orchestration and prompt construction

Once decrypted from the API gateway, the data is sent to the orchestrator engine. The orchestrator gathers the relevant data from the provided code and metadata. The orchestrator creates an optimized prompt using all the gathered data including git history. It then consults the preferences table to get the LLM model to invoke.

LLM Response

The orchestrator calls the LLM model with the generated prompt. The model generates accurate code complete suggestions. This is then sent back to the client.

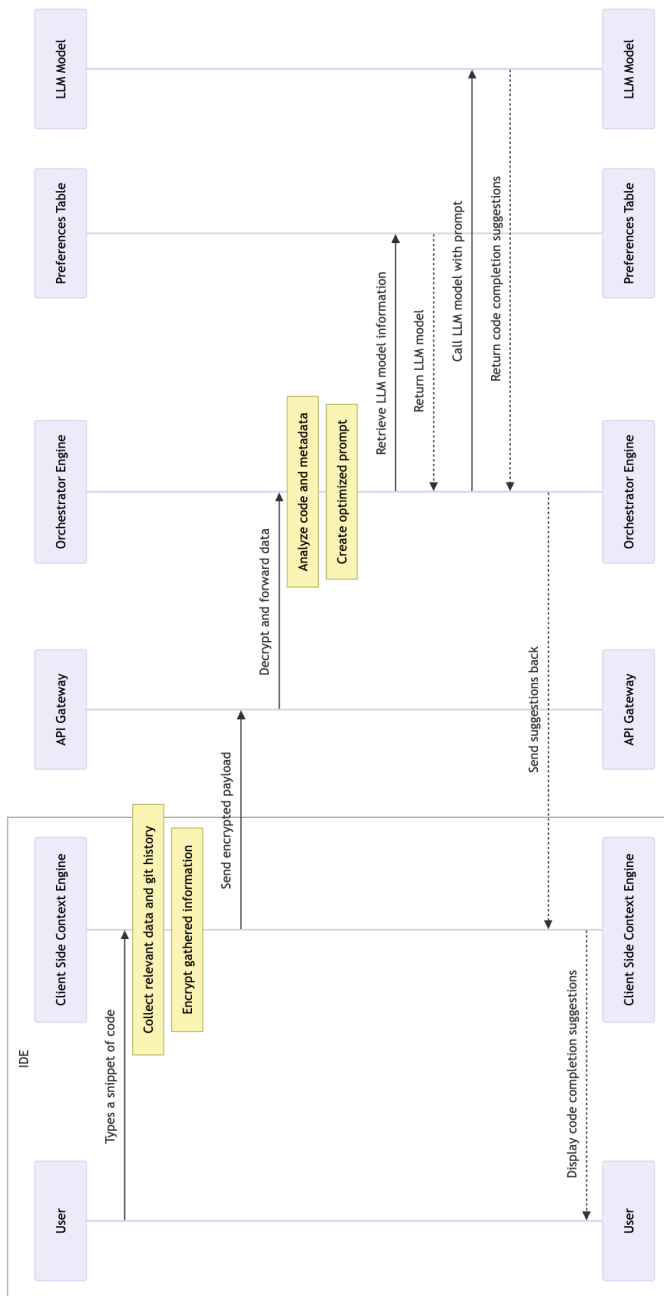


Fig 3.8 Code complete flow

3.6.10. Supporting chat mode

IDEs promote developer productivity by enabling conversational interactions i.e. chat mode.

Maintaining a persistent connection between client and server

The client IDE establishes a **bidirectional stream (specifically using gRPC over HTTP/2)** to the server.

Unlike a standard web request that closes immediately after a response, gRPC keeps this stream open.

Query Type and Scope

When the user asks a query on the chat, the query could be related to the codebase or be generic. For the latter, we can ask the user to add @web notation which signals that the query can be answered by consulting external sources (i.e. over the internet).

For the codebase-related queries, our system needs to figure out which file/class/function the query pertains to, to answer. The user can explicitly select the context/scope of the query to a few files or the entire codebase. This scope data, along with the user query, is sent by the IDE context engine to the server.

Orchestrator Collaboration

The orchestrator receives the data from the client, and leverages vector search on embeddings to locate semantically relevant functions/classes in the database.

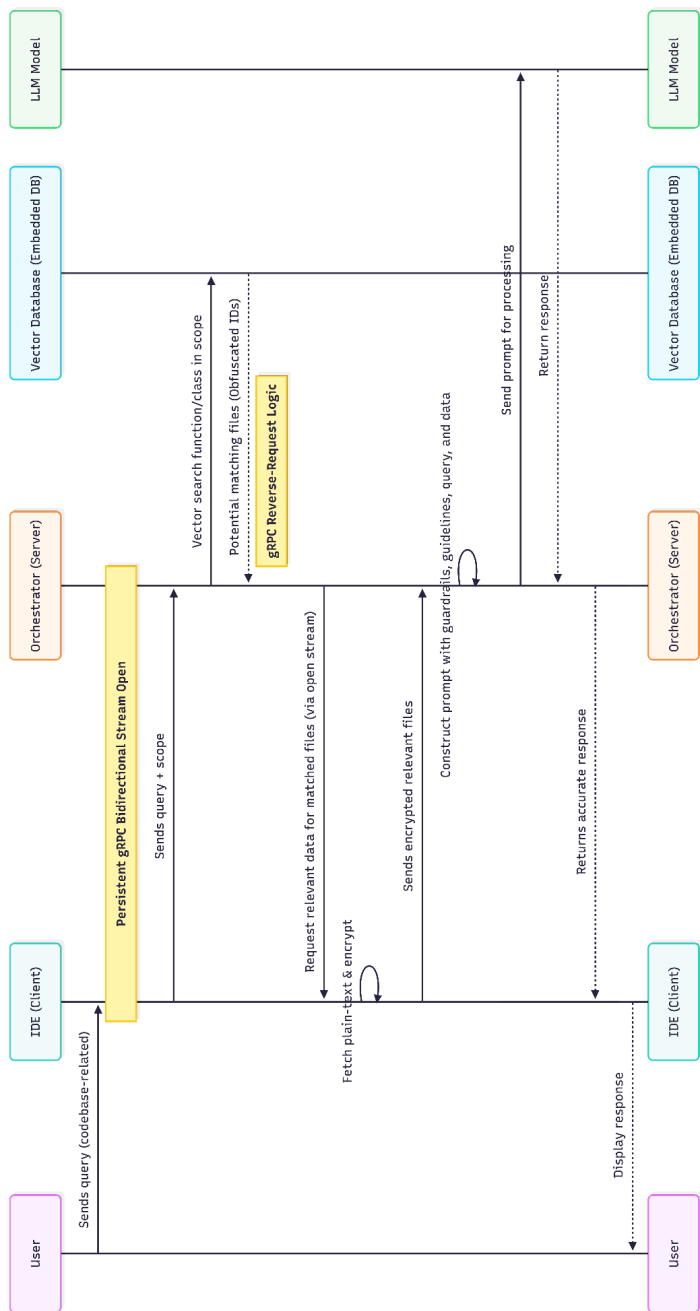


Fig 3.9 Chat mode flow

Selective fetch from codebase

Once it locates a few potential files that can have the function, it requests the local client for the relevant data from these files.

This happens over the **grpc bidirectional stream connection** between the client IDE and server. The server writes a data frame (a Protocol Buffer message) into that already-open stream.

The client sends the requested files (encrypted) over the connection. The orchestrator then forms a prompt with guardrails and guidelines with all this information, along with the user query and routes it to the LLM. The resulting response ensures accurate response back to the user.

3.7. Database selection and data modelling

Let's see what we need to store on the server side and the access patterns of this data.

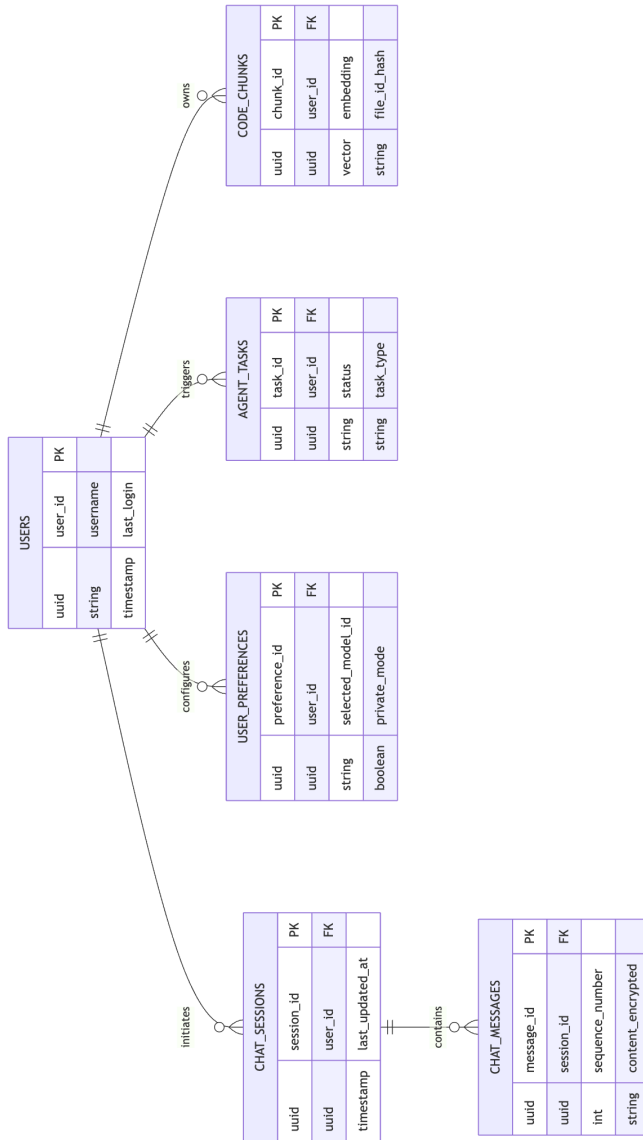


Fig 3.10 Chat mode flow

1. Code chunk embeddings

Context: Stores vector embeddings of code blocks mapped to obfuscated file metadata.

Usage: Written during file indexing; heavily queried via vector similarity search during Chat Mode.

We use a vector database to store embeddings along with obfuscated file names and line numbers for better vector search. We can also use postgres with the pg_vector extension.

Table Name: code_chunks		
column name	data type	description
chunk_id	UUID (PK)	unique chunk id
user_id	UUID	owner of the chunk
vector_embedding	vector numbers	converting code into low-dimensional vectors for semantic search
encrypted_chunk_hash	string	chunk hashed for integrity verification
obfuscated_filename	string	Obfuscated filename and line numbers for privacy
line_number_start	number	starting line number of the chunk
line_number_end	number	end line number of the chunk
updated_at	timestamp	last reindexed timestamp for the chunk
created_at	timestamp	chunk creation timestamp

2. Users and User preferences

Context: The central identity table containing authentication credentials and profile timestamps.

Usage: Accessed during login (auth) and referenced as a Foreign Key by all other data entities.

(Existing) Table Name: users		
column id	data type	description
user_id	UUID (PK)	unique id
username	string	username for the user
hashed_password	string	password hashed for the user
created_at	timestamp	user profile created at timestamp
last_login	timestamp	user last logged in

The **user_preferences** table contains preferences like private mode enabled, and models selected along with other details. These can be stored in a PostgreSQL database.

Context: Stores user-specific configurations such as *Private Mode* toggles or preferred LLM models (e.g., GPT-5 vs Claude).

Usage: Loaded at session start to configure the Orchestrator's routing logic.

Table Name: user_preferences		
column name	data type	description
preference_id	UUID (PK)	unique id
user_id	UUID	reference from users table UUID
model_id	string	Selected LLM Model like GPT-5
privacy_mode	boolean	is codebase private, are queries private etc.

max_tokens	int	max tokens allowed
tokens_used	int	total tokens used by user
updated_at	timestamp	preference last updated at timestamp

3. LLM Models

Table Name: llm_models		
column name	data type	description
model_id	UUID (PK)	unique id
name	string	model name like GPT4, Claude etc
max_context_length	int	maximum length of context that can be sent as a prompt
cost_per_token	int	cost per token
best_for_tasks	jsonB	strengths like reasoning, summarization etc.
updated_at	timestamp	model information updated at timestamp

We can store the LLM prompt templates to avoid building the prompts from scratch every time. We can also store this in code itself instead of a table in the database. These prompts are unlikely to change every time. It will also reduce the latency of fetching the prompt templates each time from the database.

4. Chat sessions

Context: Acts as a container for conversation threads, tracking the *last updated* time for sorting.

Usage: Queried to populate the Recent Chats sidebar history in the IDE.

Table Name: session		
column name	data type	description
session_id	UUID (PK)	unique id
user_id	UUID	user id reference from users table
last_updated_at	timestamp	session last updated timestamp
session_metadata	Varchar	metadata regarding session like name, description
idx_user_last_updated	INDEX(user_id, last_updated_at DESC)	composite index to load the recent chats sidebar efficiently

5. Chat messages

Context: Stores the actual encrypted content and sequence of messages within a session

Usage: Fetched sequentially when a user opens a specific historical chat thread

Table Name: chat_messages		
column name	data type	description
chat_message_id	UUID (PK)	unique id
session_id	UUID	session id reference from sessions table
sequence_number	INT	The client or orchestrator increments this integer for every new message in a session_id. This guarantees strict ordering

		even if timestamps are identical
sender	UUID	user id reference from users table or 'system'
message	string	encrypted chat content
created_at	timestamp	chat message timestamp
idx_session_created	INDEX(session_id, created_at)	Critical for fetching chat history in order

6. Agent tasks

Context: Tracks the lifecycle (Queued → Running → Completed) of long-running background jobs like refactoring or testing

Usage: Updated by worker processes; polled by the client or pushed via gRPC to update UI progress bars

Table Name: agent_tasks		
column name	data type	description
task_id	UUID (PK)	unique id
user_id	UUID	who called the task
task_type	string	refactoring,testing,documenting etc.
task_status	string	QUEUED,STARTED,IN_PROGRESS,FINISHED
parameters	jsonB	any additional parameters for task
created_at	timestamp	task creation timestamp
completed_at	timestamp	task completed timestamp

<code>idx_usr_status</code>	<code>INDEX(user_id, status)</code>	To quickly show the user the active tasks on the dashboard
-----------------------------	-------------------------------------	--

We see that most data is relational, and high read traffic needs to be supported. Both can be served by a PostgreSQL database.

For embeddings, we can use PostgreSQL with the `pg_vector` extension since it's integrable into our postgres database. Alternatively we can use a serverless choice of vector database like turbopuffer that scales seamlessly.

Postgres pgvector extension

- Strengths:
 - Integration with PostgreSQL: Allows to store vector data alongside structured data, simplifying architecture
 - Hierarchical Navigable Small Worlds (HNSW) Indexing: The HNSW index type offers good performance for approximate nearest neighbor (ANN) search, significantly speeding up queries compared to exact search
 - Cost-Effective: More cost-effective for users already within the PostgreSQL ecosystem, especially for small to medium-sized datasets
- Potential Drawbacks:
 - Scalability Management: While it can scale, achieving massive scale (hundreds of millions or billions of vectors with high QPS) may require more manual sharding, partitioning compared to purpose-built distributed systems

Turbopuffer

- **Strengths:**
 - **Purpose-built Architecture:** Designed from the ground up for vector search at scale, optimized for low latency and high QPS (queries per second)
 - **Stateless Nodes:** Uses object storage as a write-ahead log, which allows for cost-effective horizontal scaling and high throughput
 - **Scalability:** Can scale to trillions of documents across millions of namespaces, handling very large datasets more efficiently than general-purpose databases
 - **Performance:** Offers competitive or superior performance to other specialized vector databases
 - **Managed Service:** Reduces operational complexity, which can be a significant factor in maintaining high performance at scale
- **Potential Drawbacks:**
 - **Not Open Source:** It is a commercial-only model
 - **Separate Infrastructure:** Requires managing a separate system from the primary relational database (pgsql)

While PostgreSQL with `pg_vector` is an architecturally sound choice for its simplicity and data consolidation, the non-functional requirement of 200ms latency and the ~1666 Peak RPS pushes the **preference toward a specialized, performant, and horizontally scalable solution like Turbopuffer.**

3.8. Deep dive into design

So far, we have designed the following.

Requirement	Design Solution Summary
Codebase Awareness	Client Context Engine: Chunks code using ASTs locally. Vector DB: Indexes embeddings with obfuscated IDs for privacy.
Code Completes	Hybrid Sync: Uses gRPC streams for real-time typing Context: Injects file history + cursor position into the prompt
Chat Mode	Scope Awareness: Users select Current File vs. Codebase Retrieval: RAG pipeline fetches relevant chunks via vector search
Data Persistence	Polyglot Store: PostgreSQL for relational data (Users/Chats) Turbopuffer: Vector storage for code embeddings
Low Latency (<200ms)	<i>Next Section Focus:</i> implementing Semantic Caching to skip LLM calls and Speculative Decoding to generate code faster than typing speed.
Reliability	<i>Next Section Focus:</i> designing Circuit Breakers to handle outages and Exponential Backoff to prevent cascading failures during

	high load.
--	------------

Now, let's examine a few of the non-functional requirements and consider how we can achieve them in our design.

3.8.1. Reducing latency

The server needs to respond quickly to requests for completions, chat and agent tasks so the developer is not frustrated.

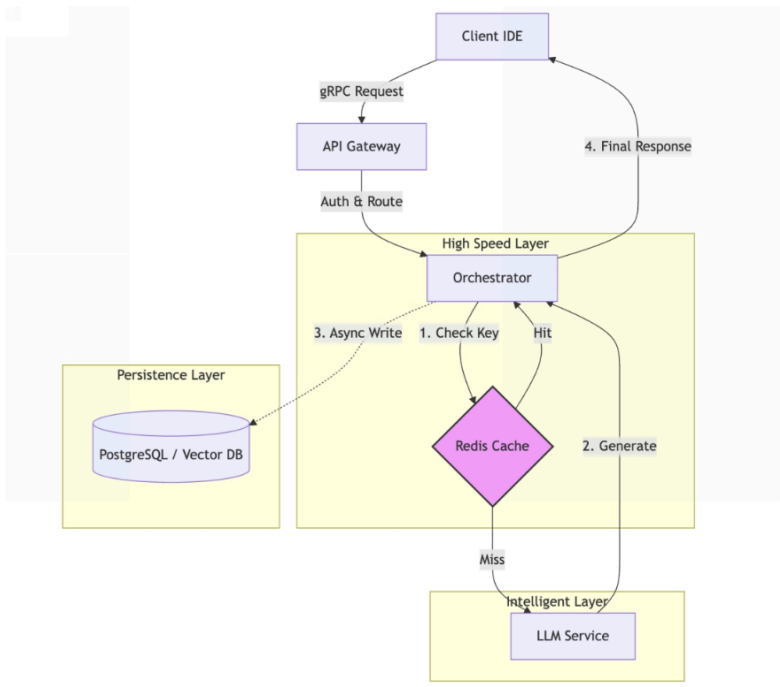


Fig 3.11 Caching Design

A few ways that this can be done:

1. Caching strategies

- Commonly searched vector embeddings, recent chat sessions can be indexed on the server to reduce the latency, as well as load on the database
- Commonly retrieved code completes and explanations can be cached for faster retrieval

Cached item	Caching key	Description
vector embedding	most frequently used text called for embedding	
Vector search	user_id:team_id:query_hash	Calculating the query_hash and running the vector search are CPU-intensive and necessary for every context-aware query. Caching the relevant chunk IDs saves milliseconds and reduces database load
code complete for boilerplate snippets	team_id:model_id:model_version:file_id:code_snippet_hash	Cache only exact, high-frequency boilerplate snippets that are static and widely used (e.g., common import statements or context-free method signatures), or cache only on the Client-Side (locally) for short-term session recall.
Explanation/summarization	team_id:model_id:model_version:query_hash	Caching common questions about a project's architecture avoids the high cost and latency of rerunning the LLM.

We can use a Cache-Aside Pattern using Redis/memcached for distributed, low-latency caches and employ cache invalidation on source modification, LLM model upgrades etc.

The below diagram demonstrates how a Cache-Aside strategy works.

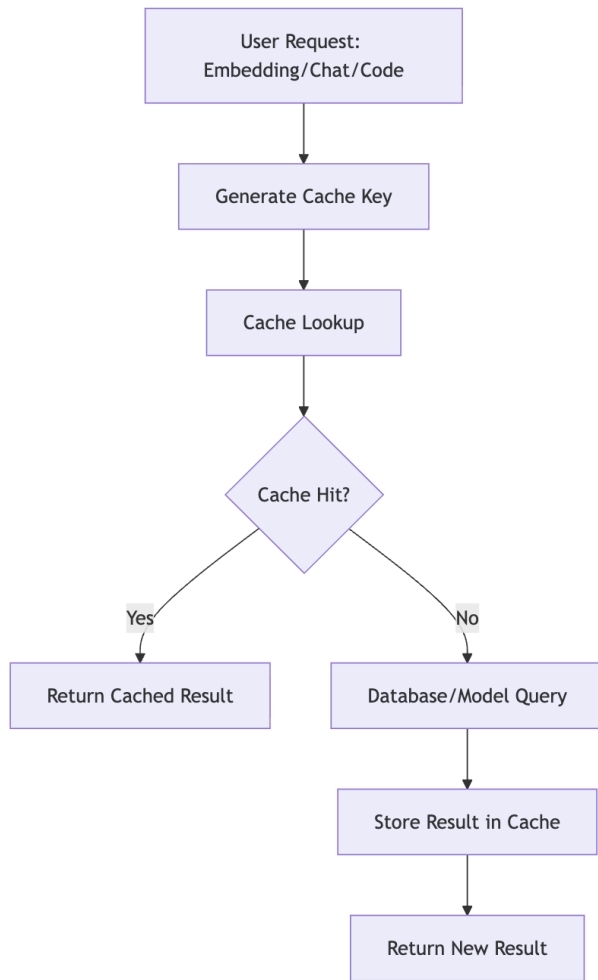


Fig 3.11a Cache Aside Pattern

2. Scalable Architecture

- **Auto scaling:** The orchestrator engine can be configured with horizontal scaling policies. This can be done monitoring the CPU and memory usage on the orchestrator engine
- **Async task queues:** Queue long-running agentic tasks so that the system is available to respond to user requests and queries faster

3. Optimised datastores:

- **Read replicas:** Distribute load across PostgreSQL by adding read replicas for faster response
- **Vector database scaling:** Low-latency serverless solutions like Turbopuffer help auto scaling without operational overhead

4. Efficient Change Detection

Using Merkle trees for codebase syncing improves latency with partial updates without reindexing entire files

However, the Merkle Indexer needs to be optimised for IDE performance. Calculating Merkle hashes for a massive monorepo is CPU-intensive. The Client Context Engine must implement Idle-Time Execution.

- **Debounce:** Only trigger hash calculation after the user has stopped typing for > 2 seconds
- **CPU Cap:** Restrict the indexing thread to a maximum of 10-15% CPU usage to ensure the IDE remains responsive to keystrokes

5. Race-to-Response

The orchestrator engine can fall back to lightweight models with lower latency whenever the optimal LLM is too slow. Since our target is <300ms per response, if the primary provider takes longer than 200ms, we also trigger a request to a backup provider that can respond in ~100ms (though with less accuracy). We then wait up to 100ms for the optimal provider's answer, if it arrives in time, we return the optimal result; otherwise, we return the backup response to the client.

6. Semantic Caching for Code Explanations

Developers often ask the same question with slightly different phrasing (e.g., "How do I filter a list?" vs. "Syntax for list filtering"). Exact match caching fails here. Here Semantic Caching can be pretty useful.

We embed the user's prompt and check our cache for any previous prompt with a cosine similarity > 0.95. If found, we serve the cached explanation immediately, bypassing the GPU inference entirely.

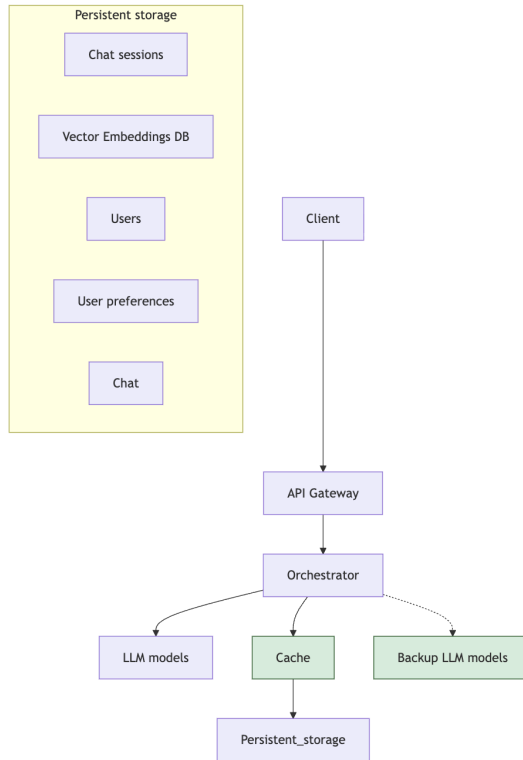


Fig 3.12a Flow for code explanations

7. Normalized Cache Keys

A naive hash key (*hash(prompt)*) results in cache misses for trivial changes. Implement Prompt Normalization before hashing:

- Whitespace Trimming: " Hello " to "Hello"
- Case Insensitivity: "Python" to "python"
- [Stop Word Removal](#): "What is the status" to "status" (To be used with caution depending on domain)
- Embedding Cache: Do not just cache the final text response. Cache the Vector Output from the embedding model. The embedding API costs money

and adds latency; if the same text chunk is seen twice, reuse the vector.

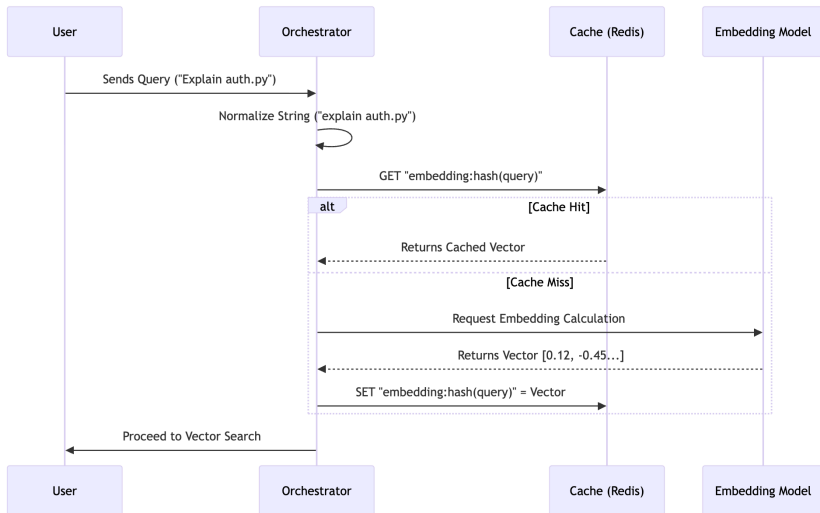


Fig 3.12b Avoid redundant embedding calculations by caching

8. Advanced Pattern: Speculative Decoding

Standard LLM inference generates one token at a time, which is often too slow for the <200ms target required for a native feel. To achieve the sub-second latency seen in modern AI editors, we must move beyond simple caching and implement **Speculative Decoding**. This is how it works:

- Large models (e.g., Llama-3-70B) are memory-bound. Reading the model weights for every single character generated adds significant latency.
- So we run two models, a draft model and a verification model.
- We use the Draft Model to predict the next few tokens, and feed that prediction into the Verify Model as a batch. This allows the large model to verify 5

tokens in a single GPU operation, rather than generating them one by one

- **Draft Model (Small):** A tiny, hyper-fast model (e.g., a 7B parameter distilled model) speculates the next 10–20 tokens (or a full diff) instantly.
- **Verification Model (Large):** The powerful model processes that entire drafted sequence in a single pass to verify it.
- If the draft is correct, the user sees 20 characters appear instantly, achieving effective speeds of 1000+ tokens per second. This creates the Tab-to-jump sensation where the cursor moves over predictable code.

3.8.2. Improving Reliability and availability at high load

Strict timeouts for calls

- maximum allowed time for the LLM to generate a response (inference)
- If model exceeds this timeout, cancel the request to the LLM to free up orchestrator thread

Circuit Breaker Pattern

- Monitor LLM model response time and failures rates, if they exceed the threshold, open the circuit
- Automatically stop sending traffic to slow or error prone models until they recover, and utilize healthy alternatives instead

Retrying with backups

Given the resource demands of LLMs, response delays or failures can occur during high loads, network issues or provider outages. A retry and backup strategy ensures graceful degradation and avoids service disruptions. Delays beyond a second can cause developer frustration and potentially reduce our system's value.

So it is important to configure retry with exponential backoff in the orchestrator engine while calling LLM models in case of failures or slowdowns in LLM model providers.

Circuit breaks should also be in place to route the query to backup LLM providers in case the optimal LLM model provider is unavailable.

Retry strategies:

- **Interactive (Code completes):** Do not use aggressive exponential backoff. If a user is waiting, a 60-second retry is effectively downtime. Use Capped Backoff (start 500ms, max 1s) or fail fast to a fallback model.
- **Asynchronous (Batch Jobs):** Use Exponential Backoff (wait 1min, 2min, 4min). Since no user is waiting, we can afford to wait out a 5-minute provider outage.
 - Exponential backoff with jitter: On failure/timeout, the system retries requests after waiting intervals that increase exponentially i.e. 50ms, 100ms, 200ms..
- **Retry limits:** We retry thrice to avoid unnecessary load and extensive delay

Backup model strategy:

We keep a multi-model fallback strategy. This works as follows:

- We keep backup models which might not be optimal for accuracy but are faster
- If the primary model doesn't respond within 200ms, call the backup models

- Present the first completed response to the user
- Inform users that the response is from a non optimal model
- Alternatively, we can also send the same query in parallel to multiple models utilizing the first response and cancelling the others. This increases resource usage, though, so usage needs to be balanced

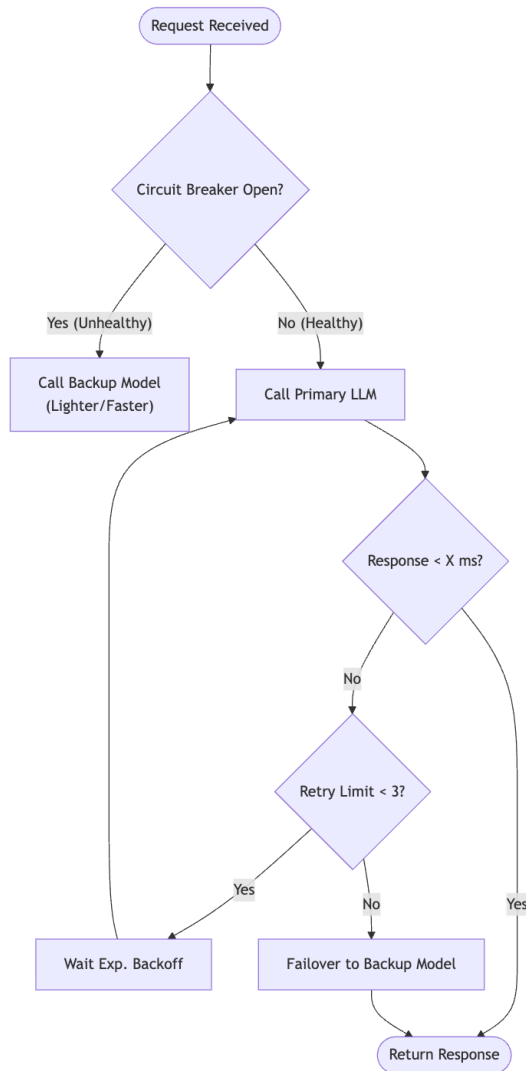


Fig 3.12c Decision tree for Timeouts, Circuit Breakers, and Retries

3.8.3. Synchronous/Asynchronous Hybrid Processing Model

We employ hybrid processing models for different use cases of our IDE.

Sync: code complete (low latency, immediate response).

Async: Chat/Agentic Tasks (high latency, long-running, handled by **Task Queue and Workers** with retries).

Queuing and async processing for chat and agentic tasks

We can further queue and asynchronously process chat responses and agentic tasks to make the system available for code complete queries, which require sub-200ms latency.

Workers can pick up tasks from the queue, send them to LLM model providers, and retry in case of failures.

Task queuing

- The chat prompts and agentic tasks are pushed to a messaging queue (like sqs) by the orchestrator.
- Requests are picked by the worker processes which then store these requests in an events table and subsequently call the LLM provider.
- If the LLM response times out, the worker can retry with exponential backoff and track the same in the table. The orchestrator sends chat prompts and agentic tasks to a messaging queue (e.g., SQS).
- Worker processes retrieve these requests, record them in an events table, and then contact the LLM provider.
- If the LLM response times out, the worker can retry with exponential backoff, updating the number of attempts in the table for the event.

Async response delivery

- User sees loading status on the IDE for these tasks
- Since we maintain an open gRPC bi-directional stream

(established in the Code Context section), the server pushes the Task Complete event directly to the client down this pipe.

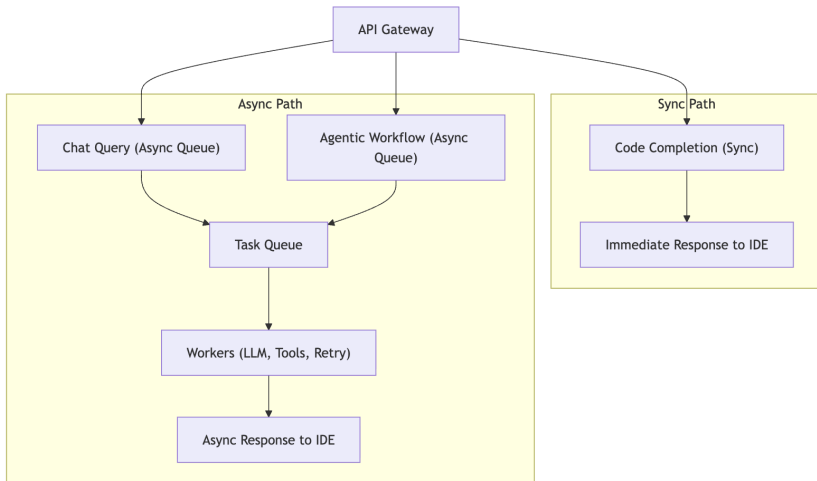


Fig 3.13 Async flow for code explanations and sync flow for code completes

We can ensure chat queries are prioritised and processed faster than agent tasks even in the queue by adding weights to the type of tasks. So chat queries have higher priority and will be consumed faster from the queue by the workers.

Rate limiting and throttling

1. To protect the system from overload or DDOS attacks, rate limiting and throttling are enforced at the API gateway layer.
2. This restricts the number of requests any user or client can send over a fixed time window (e.g., 200 requests/min).

So our updated design now looks like this:

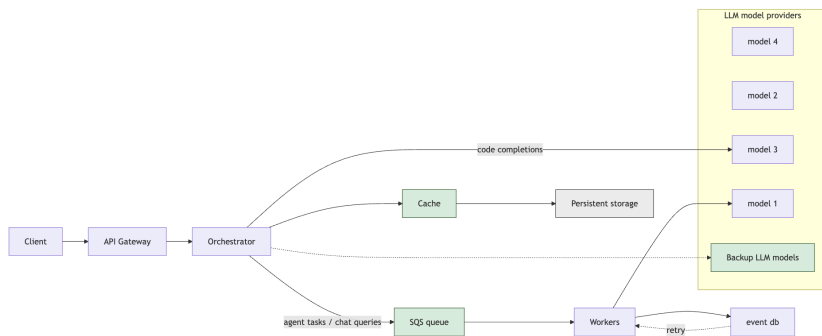


Fig 3.14 Adding rate limit and retries

3.8.4. Improving accuracy

A comprehensive list of instructions is required to be provided to the LLM to reduce hallucinations and errors. To achieve this, we can follow:

Advanced Prompt Engineering

- **Structured instructions:** Prompts should specify desired formats, content boundaries and exceptions

```

{
  "role": "system",
  "content": "You are a coding assistant. You must format your response as a valid JSON object. Do not include markdown formatting."
  "schema": {
    "type": "object",
    "properties": {
      "explanation": "string",
      "code_diff": "string"
    }
  }
}

```


Using formats like JSON mode ensures the IDE can parse the answer programmatically, rather than using Regex to scrape text.

- **Contextual knowledge:** Surrounding code, history of file changes and other relevant metadata should be provided to minimise ambiguity
- **Extensive examples:** Provide examples of good and bad responses to instruct the model on expected behaviour and avoid unwanted behaviours
- **Negative prompting:** Clearly define unwanted behaviour (“Do not add db migrations”, “Do not hallucinate API names”, etc)
- **Model-specific optimisations:** Different LLM models benefit from specified prompt tuning

Dynamic Feedback Loop

- Developers should rate and annotate suggestions for better responses so that model prompt can be tuned
- Post-processing after tasks are run can be done to flag anomalies, check syntax correctness etc.

Multi-model use

- Multiple models can be used in case of ambiguous requests to offer alternative responses

Compiler-as-a-Judge

For code, we have a ground truth: the compiler. For automated evaluation, we attempt to compile/interpret the generated code. If it throws a syntax error, the score is automatically 0. This deterministic feedback loop is faster and cheaper than asking another LLM to review the code.

3.8.5. Applying Privacy Patterns

Ephemeral Guarantee

- **In-memory decryption:** All encrypted code snippets sent to the server should be decrypted in memory and discarded post processing
- **No persistent code storage:** All sensitive data including codebase chunks and associated metadata like filenames are never stored on disk or on databases
- **Memory Hygiene:** Variables holding plaintext code are explicitly overwritten or garbage collected immediately after the embedding vector is generated.
- **No Logging Policy:** We implement strict middleware that strips all request bodies from observability logs to prevent accidental leakage of code snippets into Datadog or Splunk.

Code Cannot Be Reconstructed

- **Semantic index search only:** The code chunks are converted into vector embeddings and then stored on the database, ensuring the code cannot be reconstructed from the database
-

The Limits of Embedding-Only Privacy

While storing only vector embeddings prevents casual reading of source code, it is not a cryptographic guarantee of privacy.

Adversaries can potentially reconstruct original text from high-fidelity vector embeddings using model inversion techniques.

Mitigation: Encryption keys are rotated regularly and access controls are strictly scoped.

- **Metadata Obfuscation:** Real file and symbol names are not used, instead hashed ids are stored

Minimizing Attack Surface

- **Minimized code transfer:** Only code context required for a query or code complete is sent to the server, not the entire codebase
- **Encryption policies:** All requests sent to/from the client are encrypted, and all data stored on server side is encrypted

Middleware [PII](#) Scrubbing

- **Data Cleaning Middleware:** Encryption protects data in transit, but it doesn't prevent the model from seeing secrets if a user accidentally pastes them. We implement a Data Cleaning Middleware on the client side before the payload is encrypted.

This middleware uses regex and entropy detection to identify potential API keys, hardcoded passwords, or AWS credentials. These are replaced with tokens (e.g., <SECRET_REMOVED>) before the context is ever sent to the server.

This ensures that even if our server logs are compromised, user secrets are never persisted or exposed to the model provider.

3.9. How to train with test data?

Sourcing datasets

- Curate diverse set of open source projects with varied languages, sizes and domains
- Refresh the dataset regularly to incorporate up to date coding style and practices
- create custom open source repos with intentional gaps, edge cases to stress test behaviours

Testing and training for code complete

Randomly remove code blocks, functions, classes from the code. Start typing signatures of missing classes and functions. Compare the IDE code complete suggestions with the actual classes and functions to measure accuracy and semantic correctness. Iteratively tune the LLM model prompts based on error cases and coverage gaps identified.

Testing for chat responses

Clone the repo removing all the comments and docs. Ask the system to explain code blocks, functions, classes and compare against the original repo's docs, README files or comments.

Testing for agentic workflow and tasks

Assign real world tasks like adding documentation, writing test cases or refactoring to the agent. Combine multiple tasks in scenarios. Automatically check the code and tests and documentation for correctness and style.

Integrating these tests into CI/CDs help keep models and prompts updated with evolving repos.

Mutation Testing

Add logic and syntax errors to your code. Evaluate if the system can identify and fix these errors.

Negative prompt testing

Give confusing or risky actions like “Delete all database migrations”. Evaluate if the system is able to flag it as a risk and ignore/refuse or ask the user for further clarification.

3.10. Monitoring and user feedback

It is essential to implement robust feedback pipelines that allow users to provide input on their experience, especially for chat interactions. Users should be able to easily indicate whether a response was helpful or not, and this feedback should be collected and utilized to improve the LLMs, enabling better future responses.

We should monitor user engagement metrics. Tracking the number of active users and observing trends in how frequently and extensively they interact with the AI system helps gauge adoption and effectiveness over time

3.11. Conclusion

LLMs make the already intelligent development environments even smarter, helping developers offload menial tasks and intelligently assist them like a coding assistant or a pair programmer.

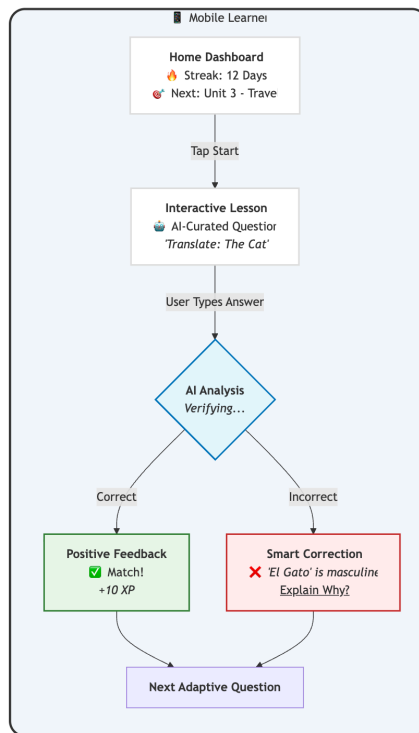
3.12. References

1. [Real-world engineering challenges: building Cursor](#)

- Orosz, Gergely. “Real-World Engineering Challenges: Building Cursor.” The Pragmatic Engineer, June 10, 2025.
<https://newsletter.pragmaticengineer.com/p/cursor>
2. [Cursor -> turbopuffer, cutting costs by 20x](#)
Turbopuffer. “Cursor -> Turbopuffer, Cutting Costs by 20x.”
<https://turbopuffer.com/customers/cursor>
 3. [Codebase indexing](#)
Cursor. “Security.”
<https://cursor.com/security>
 4. [Our problems](#)
Asif, Sualeh. “Our Problems.” Cursor, October 12, 2023.
<https://cursor.com/blog/problems-2023>
 5. [How Cursor Serves Billions of AI Code Completions Every Day](#)
ByteByteGo. “How Cursor Serves Billions of AI Code Completions Every Day.” ByteByteGo Newsletter, July 29, 2025.
<https://blog.bytebytego.com/p/how-cursor-serves-billions-of-a-i>
 6. [Building Windsurf with Varun Mohan](#)
Orosz, Gergely. “Building Windsurf with Varun Mohan.” The Pragmatic Engineer, May 7, 2025.
<https://newsletter.pragmaticengineer.com/p/building-windsurf-with-varun-mohan>
 7. [How GitHub Copilot Works](#)
Quastor. “How GitHub Copilot Works.” Quastor, April 4, 2024.
<https://blog.quastor.org/p/github-copilot-works>

4. Case Study: Adaptive Learning Platform

Design an AI powered learning platform like Duolingo



4.1. Introduction

The future of learning platforms lies in personalisation. An effective learning platform adapts to each user's unique pace. It helps reduce time spent on mastered concepts while focusing on lessons that are challenging but not discouraging, which is crucial for learner engagement.

The lessons or exercises need to be updated frequently to keep up with the nuances of the language and introduce new lessons based on user feedback and usage.

This is where Large Language Models (LLMs) become transformative. LLMs can be leveraged on two primary fronts: curating a personalized curriculum for the user and generating new exercises.

In this chapter, we will design the core AI-driven enhancements for a platform like Duolingo. Our focus will be on the systems that enable personalised lesson progression and interactive exercises. So features like detailed scoring systems, although vital to the overall platform, are out of scope to concentrate on the fundamental AI architecture.



Chapter Spotlight

Offline Generation (with Human-in-the-Loop) vs. Online Personalization Curation

4.2. Functional requirements

- **Lesson Generation:** Design entire exercises and lessons of different levels of a language for different levels. This ensures a constantly fresh and relevant content stream.
- **Lesson Curation with Adaptive Difficulty:** The system should look at a user's past answers and figure out the perfect next question to ask - not too hard, not too easy.

4.3. Non-functional requirements

- **Low Latency:** The system must have a P99 latency of <500ms for the round-trip from submitting an answer to receiving the feedback or the next question

- **High Accuracy:** This is about trust. Our platform shouldn't give wrong answers or bad explanations.
- **High Availability:** The system should be resilient. Even when there's an outage in an LLM provider, it should not bring down the entire learning platform.
- **Highly Personalised:** Lessons should be highly tailored to the users' needs - it shouldn't be too challenging or too easy.
- **Cost Effectiveness:** LLM API calls are expensive. There should be a good strategy for when and where to use LLM interactions.
- **Scalability:** The system should be scalable to allow for peak loads during timezone overlaps in learners.

4.4. Scale estimates

Before jumping into the design, let's get a feel for the sheer size of this platform.

The platform has 50M daily active users. There are 4 most popular languages, most learned by users. The platform supports 50 languages in total, with 250 lessons for the most spoken ones and up to 70 for the least spoken ones. Each lesson has up to 10 questions.

Daily Active Users (DAU): 50M

Lessons per user per day: 3

Questions per lesson: 10

$$\text{RPS} = (\text{Daily Active Users} * \text{Lesson per user per day} * \text{Question per lesson}) / \text{Seconds in a day}$$

For serving lessons to users, we need to support:

$$50M * 3 * 10 / (60 * 60 * 24) = 17340 \text{ rps}$$

Each Question requires 3 core API calls (minimum):

1. **GET** Question Content
2. **POST** User Answer

3. **GET** Feedback/Results

Average RPS = 17K RPS * 3 API calls = 51K RPS

Peak RPS is 5 times ~= 51K * 5 = 255K RPS

4.4.1. **Additional Load Considerations**

- Conversation practice sessions: +20% RPS
- Real-time feedback and analytics: +15% RPS
- **Total Peak Load:** ~350K RPS

4.5. **API design**

Let’s look at the APIs our system requires.

4.5.1. **Admin lesson generation and review APIs**

Generate Questions (Sync)

Immediately generates a small batch of questions (e.g., <5) for rapid testing or low-volume needs without queuing.

URI	/api/v1/admin/questions
Method	POST
Authorization	Bearer {admin_token}
Body	<pre>{ "format" : "fill-in-the-word" "count" : 2, "level": "A2", "language" : "Spanish", "topic": "prepositions" }</pre>

Response	<pre>{ "metadata": { "count_requested": 2, "count_generated": 2, "initial_status": "IN_REVIEW", }, "questions": [{ "id": "question_137293", "text": "El libro está _____ la mesa.", "type": "FILL_IN_THE_BLANK", "choices": ["en", "sobre", "hacia", "entre", "según"] }, { "id": "question_137294", "text": "Caminamos _____ el parque.", "type": "FILL_IN_THE_BLANK", "choices": ["en", "sobre", "hacia", "entre", "según"] }] }</pre>
----------	--

Generate Questions (Async)

Submits a bulk generation job (e.g., 100+ items) to the background queue to avoid blocking the Admin UI.

URI	/api/v1/admin/question-generation-jobs
Method	POST
Authorization	Bearer {admin_token}

Body	<pre>{ "format": "fill-in-the-blank", "count": 10, "level": "A2", "language": "Spanish", "topic": "prepositions" }</pre>
Response	<pre>{ "job_id": "job_d9e8f7a6", "status": "PENDING", "message": "Question generation has been queued.", "status_check_url": "/api/v1/admin/question-generation-jobs/ /job_d9e8f7a6" }</pre>

Retrieve Questions status

Polls the progress of a specific long-running generation job to show "Processing..." states on the dashboard.

URI	/api/v1/admin/question-generation-jobs/job_d9e8f7a6
Method	GET
Authorization	Bearer {admin_token}
Response (while still processing)	<pre>{ "job_id": "job_d9e8f7a6", "status": "PROCESSING", "message": "Generating questions..." }</pre>
Response (when complete)	<pre>{ "job_id": "job_d9e8f7a6", "status": "COMPLETED", "message": "Successfully generated 10 questions.", "results_url": "/api/v1/admin/questions?job_id=job_d9e8f7a6" }</pre>

Retrieve Questions

Fetches the final list of AI-generated content once the asynchronous job status is **COMPLETED**.

URI	/api/v1/admin/question-generation-jobs/job_d9e8f7a6
Method	GET
Authorization	Bearer {admin_token}
Response	<pre>[{ "question_id": "q_101", "text": "What is the capital of France?", "type": "multiple_choice", "language": "French", "lesson_id": "lesson_1", "job_id": "job_d9e8f7a6" // ... other question fields }, { "question_id": "q_102", "text": "Translate 'Hello' to Spanish.", "type": "translation", "language": "Spanish", "lesson_id": "lesson_1", "job_id": "job_d9e8f7a6" // ... other question fields } // ... up to 10 questions]</pre>

Review Question

Fetches the full metadata and content of a single generated question for human expert validation.

URI	/api/v1/questions/{questionId}
Method	GET
Response	<pre>{ "id": "question_137293", "text": "Fill in the blank with: en, sobre, hacia, entre, según. El libro está _____ la mesa.", "type": "FILL_IN_THE_BLANK", "choices": ["en", "sobre", "hacia", "entre", "según"], "answer": "sobre", "status": "IN_REVIEW", "language": "Spanish", "level": "A2", "topic": "prepositions" }</pre>

Bulk Edit status

Allows experts to "Approve" or "Reject" multiple questions simultaneously to accelerate the curation workflow.

URI	/api/v1/questions/batchUpdateStatus
Method	POST
Body	<pre>{ "question_ids": ["question_1", "question_2"], "status": "ACCEPTED" }</pre>

	}
Response	{ "question_ids": ["question_1", "question_2"], "status": "ACCEPTED" }

Edit question

Updates the text or difficulty of a specific question to fix AI errors before final approval.

URI	/api/questions/question_137293
Method	PATCH
Body	{ "question": "Fill in the blank with: en or hacia. El libro está _____ la mesa.", "options" : ["en", "hacia"], "status": "in-review" }
Response	{ "id" : "question_137293", "question": "Fill in the blank with: en, sobre, hacia, entre, según. El libro está _____ la mesa. ", "options" : ["en", "hacia"], "status": "in-review" }

4.5.2. Curator APIs

Generate a new Lesson Session

This endpoint creates and returns a complete, personalised lesson for the user. Requests a fully personalized lesson plan (10-20 questions) tailored to the user's current proficiency and learning history.

URI	/v1/lesson-sessions
Method	POST
Body	<pre>{ "skill_id": "skill_verb_conjugation_past_tense", "type": "PRACTICE", // Could also be "TEST", "REVIEW", etc. "preferred_question_count": 10 }</pre>
Response	<pre>{ "lesson_session_id": "session_a4b1c2d3", "skill_id": "skill_verb_conjugation_past_tense", "estimated_duration_seconds": 300, "questions": [{ "question_id": "q_12483240", "type": "MULTIPLE_CHOICE", "prompt": "She ___ to the store yesterday.", "options": [{"option_id": "opt_1", "text": "go"}, {"option_id": "opt_2", "text": "went"}], }] }</pre>

```

        {"option_id": "opt_3", "text":
"gone"}
    ]
    },
    {
        "question_id": "q_43854",
        "type": "FILL_IN_THE_BLANK",
        "prompt": "We ____ the movie last
night."
    }....
]
}

```

4.6. The Blueprint: High-Level design

In an online learning platform, safety and quality of lessons are non-negotiable. On-the-fly lesson generation from raw LLMs is risky—it could introduce errors, inconsistencies, or inappropriate content. So the LLM can produce lessons offline and those can be reviewed and vetted by experts before being available to the question bank. The LLM can curate them in real time for each user. So we will decouple quality and real-time service architecture.

Therefore, our design is split into three distinct systems:

The Offline Content Pipeline: A question bank factory where we leverage AI to generate a vast pool of content that is then vetted by human experts.

Quality control -> LLM for Scale/Creation --> Human-in-the-Loop
--> Question Bank

The Online Serving Path: A real-time tutor that intelligently selects questions from this high-quality pool to create personalised lessons for each user.

This dual approach gives us the best of both worlds: the scale and creativity of AI and the quality and safety of human oversight.

4.6.1. System I: The Offline Content Pipeline

The single goal of this system is to populate a central **Question Bank** with a large volume of high-quality, approved educational content. This process is handled offline and does not impact live user traffic.

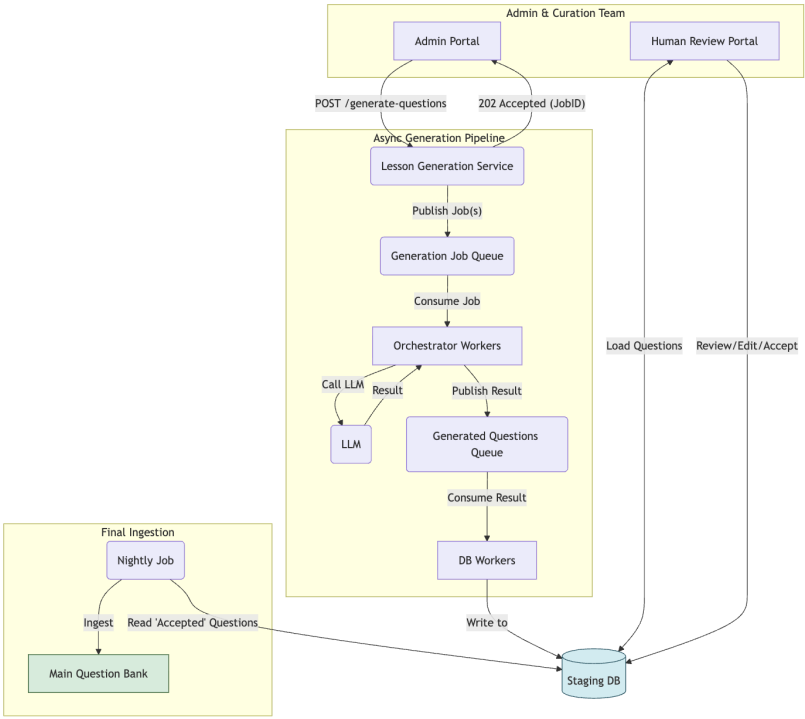


Fig 4.1: Offline content pipeline

Seed Generation

It starts with the language experts using internal tools that call the admin API to request a batch of questions. This query specifies language, level, topic and format.

API Gateway

Client app calls API gateway. This handles the authentication/authorisation of the requests before forwarding them to the Lesson Generation Service.

Lesson Generation Service

This admin component is responsible for creating lessons and enabling them to be reviewed and vetted, etc. These lessons can go through various stages of iteration by manual review until they are perfect. This is the staging area where lessons being developed are managed.

Once a lesson is perfected for release to the users, it is marked as accepted. A weekly(or daily) cron adds these lessons to the lesson database via the lessons service.

The Lesson Generation Service receives the set of lessons in the required question format.

For the lesson generation flow, the lesson generation service forwards the request to the orchestrator engine.

Orchestrator Engine

This component selects the most optimal model to route the request to based on the kind of request (i.e.simple query, lesson generation or complex conversation). It then generates the required prompt before passing it to the LLM model.

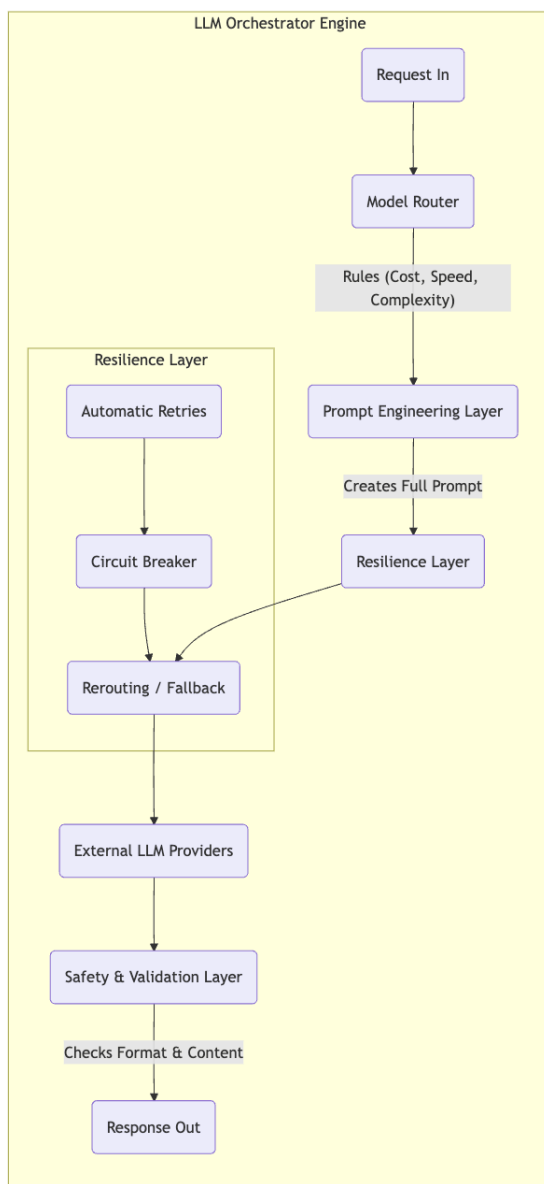


Fig 4.2: Orchestrator Engine

Prompt example: Generate 10 lessons for Spanish A2 learners to practice future tense. The sentences should be about food. For

each sentence give one answer and three distractors. Tag the question with the correct grammar tag. Output as a json array.

Human-in-the-Loop Review

Experts go through the questions and validate them, correct any mistakes in grammar, difficulty or question format and ensure they adhere to the platform's standards. They then modify and accept or reject these questions.

Ingestion into the question bank

The accepted questions are ingested into the question bank by a scheduled nightly job that does more quality checks before ingesting them into the database. It takes all the newly accepted questions , runs validations, and ingests them into the main **Question Bank** database, making them available to the online serving system. The question bank is a database table maintained by the lesson service, which has a collection of a vast variety and number of lessons for different languages. The Question Bank is the single source of truth for all approved content.

4.6.2. System II: The Online Serving Path

User Request & API Gateway

The user's app makes a call to our API Gateway to request a new lesson (e.g., POST /v1/lesson-sessions). The gateway handles authentication and routes the request to the lesson curator service.

Lesson Curator service

This is the real-time component that works behind the scenes to get the next lesson for the user from the pre-approved exercise bank. Serving a lesson to the user entails selecting one of the pre-existing lessons, based on the user's level and understanding and learning progress. This is adapted dynamically as the user progresses through the lessons.

The Lesson Curator Service maintains the Question Bank database table, which stores all lessons along with their different types, levels and tags, etc.

Fetching User Context

The User Context service maintains the User Progress table, which has user lesson completions and scores in different kinds of lessons. This is used to gather information/context on the user's level and understanding while trying to serve the next perfect lesson for the user. This includes the users' known vocabulary, the strength score and their recent mistakes.

Proactive Lesson Curation

We cannot run a slow, expensive LLM call for every user in real-time. Therefore, our online path is built on a Proactive Curation (Warm/Cold Path) Pattern.

4.6.3. Proactive Lesson Curation

Waiting for the user to complete a question and then triggering the lesson curation service for getting the next question best suited to the user's abilities increases the latency and wait time of the user to get the next question.

We can do this instead asynchronously in batches for a better user experience.

Warm Path: Instant Lesson Delivery

This is an active user flow.

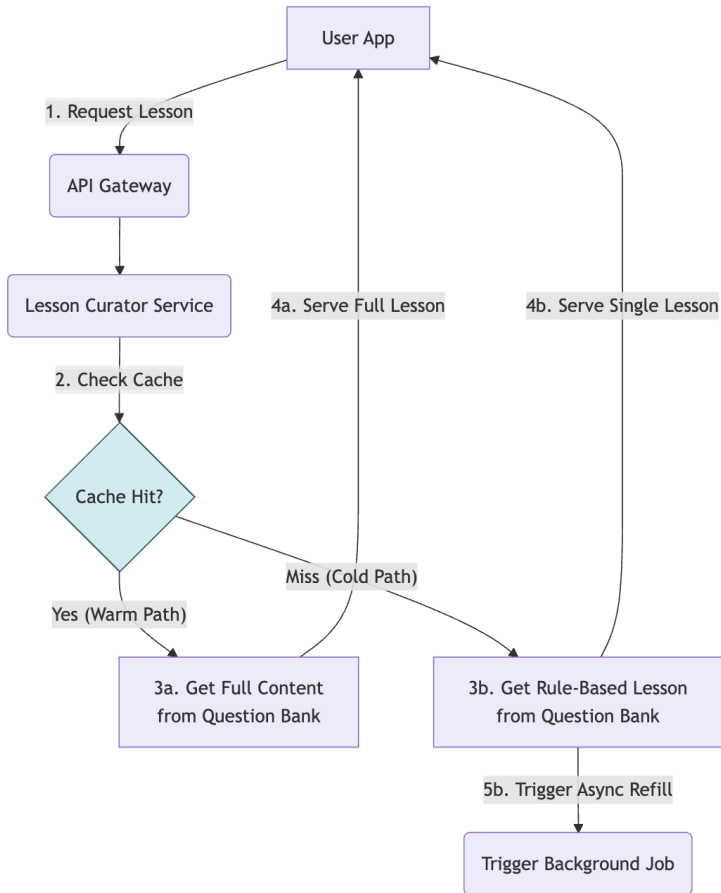


Fig 4.3: Instant lesson delivery

Instant Fetch from Cache: When a user requests a lesson, the Curation Service fetches the set of questions required from the `precomputed_lesson_playlists` cache (by using the [LPOP](#) operation on the redis list) and serves it to the user instantly.

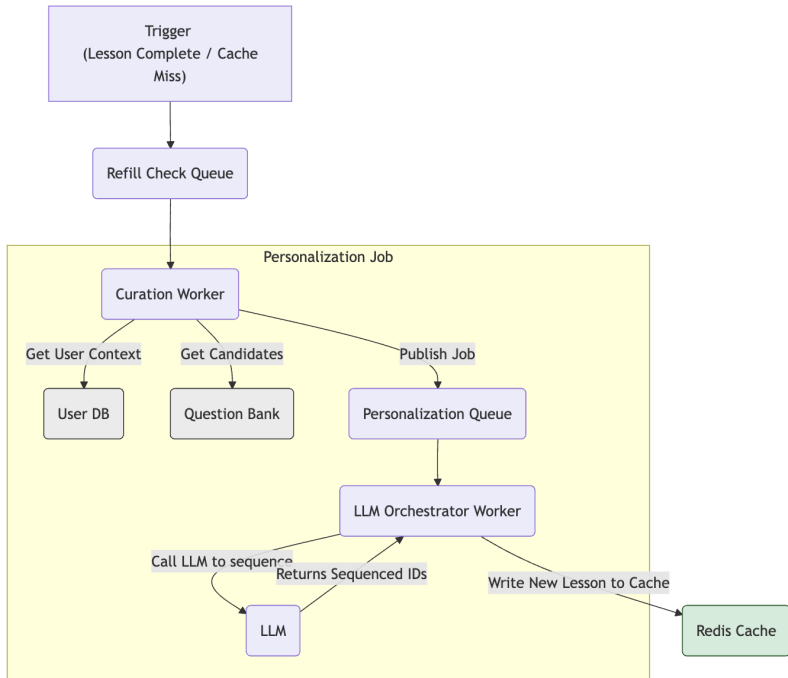


Fig 4.4: Personalized lesson flow

Event-Driven Trigger: When a user completes a lesson (or any other action that indicates user is active on the app), the service handling that action also publishes a lightweight message, like { "userId": "user_123" }, to a dedicated, low-priority **refill_check_queue** queue.

Buffer Check: A background worker in the Curation Service reads from this **refill_check_queue**. and checks their lesson buffer in Redis. If the buffer is running low (e.g., fewer than 5-10 lessons remain), it initiates the refill process.

Asynchronous Curation: This background worker grabs a big chunk of context on the user - their recent progress, history from the last 50 lessons, and a pool of a few hundred unseen questions from the bank. It bundles all of this into a payload and fires it off to an SQS queue.

LLM Orchestrator: Then, our orchestrator worker picks up jobs from that queue at a controlled pace to avoid overwhelming our services. It takes the payload and hits the LLM with a clear prompt, saying, "Based on this user's history, pick and sequence the best 20-30 questions from this candidate pool".

Once it gets that list of question IDs back, it pushes it into the user's `precomputed_lesson_playlists` in Redis, ready for their next session.

Cache Refill: When the user wants to start the next lesson, the Curator service directly fetches the next exercise from the `precomputed_lesson_playlists` cache by doing an atomic *LPOP* operation on the redis list.

- The LPOP command successfully retrieves and removes the next lesson's ID from the cache in a single, atomic operation
- This lesson is now used. It cannot be served again from the cache
- The service then fetches the full lesson content from the Question Bank and serves it to the user

This pre-fetched buffer of lessons ensures that minor processing delays in the background are not visible to the end-user.

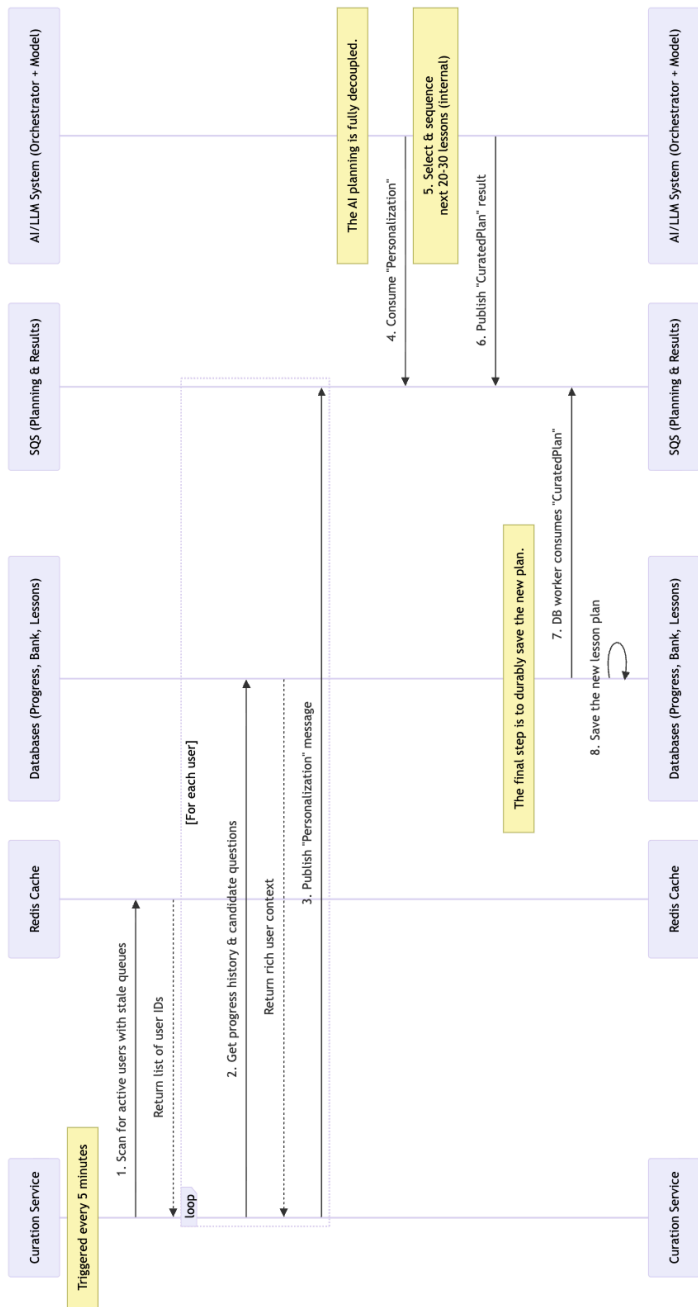


Fig 4.5: Curated lesson flow sequence diagram

Cold Path: Handling New and Returning Users

This flow handles users who don't have a pre-fetched lesson buffer.

Cache Miss: When a new or returning user requests a lesson, the Curation Service checks Redis [LPOP](#) and finds that it returns nil.

Synchronous Fallback: Instead of making the user wait for the background process, the service immediately performs a real-time, synchronous curation for just that single lesson. For this fallback, the Curation Service can bypass the LLM Orchestrator and run a fast, rule-based query directly against the Question Bank. This query would filter for questions that match the user's skill level and language, while also excluding any they have seen recently using the **lesson_history table**. While not perfectly optimized, this initial lesson offers a faster, more reliable user experience.

Serve and Trigger: The service sends this first lesson to the user. At the same time, it triggers the asynchronous refill process to start building up the user's buffer in the background, ensuring all subsequent lesson requests will be on the warm path.

This helps in several ways:

1. Decoupling the personalization engine from the user's actions
 - smooth out the load on the LLM services, since we get a steady predictable stream instead of a burst of requests
2. Buffering
 - Pre-fetching 20-30 lesson ids acts as a buffer, making the system resilient to temporary delays in the pipeline
3. Graceful degradations

If the LLM providers are down or taking a lot of time to respond, we can just fetch the lesson ids for the same level questions which the user hasn't seen in the last 20 lessons instead of sending the request to the orchestrator.

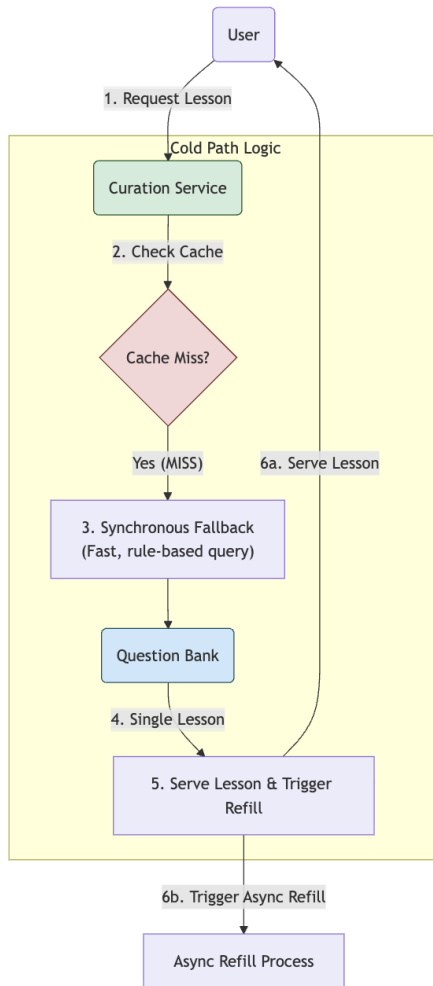


Fig 4.6: Cold path

Now, let's take a closer look at the orchestrator engine:

4.6.4. Orchestrator engine

This is responsible for making the detailed prompts for serving to LLMs.

It also decides which LLMs it should serve, based on the availability of the LLMs and the response times and the use case, whether accuracy is required or low latency is more preferred. The response goes through a safety and validation check before sending it back to the calling service (i.e. Lesson Curator Service or Lesson Generation service).

Model Router

This is an important layer responsible for reliability and cost effectiveness.

It runs a rule engine to decide which model is best suited for the job at hand. This prevents us from using an expensive, powerful model for a simple task.

Rule 1 (Simple Task): Is it a simple grammar check? Route to a small, fast, internally-hosted model.

Rule 2 (Complex Task): Is it a part of a complex conversation? Use a model like Gemini 2.5 or Claude.

Rule 3 (Premium Feature): Is it a premium user asking for a deep explanation and analysis for a question? This is where we can use a top-tier model like GPT 4.0

Prompt engineering layer

This creates the prompt based on the LLM being routed to and the user progress context. This includes guidelines and guardrails for the LLM along with the format of the response expected. This also includes examples to ensure we get a response as expected.

Resilience layer

The orchestrator is dependent on external, third-party APIs. These APIs are subject to:

- **Transient Network Failures:** Brief hiccups, TLS handshake issues, or momentary packet loss.
- **Service Overload/Rate Limits (429):** The provider is busy or usage quota is exceeded.
- **Provider Outages (5xx):** A major failure, high-latency, or full downtime of the external service.
- **High Latency/Timeouts:** The model takes too long to generate a response, exceeding the system's required response time

The orchestrator adopts several techniques to mitigate these issues:

Automatic Retries

When an initial API call fails with a **transient error** (like a 429 Rate Limit or a 503 Service Unavailable), the layer automatically re-submits the request.

- **Purpose:** To solve temporary, short-lived glitches without involving human intervention or switching models.
- **Strategy: Exponential Backoff:** Instead of retrying immediately, the orchestrator waits for an exponentially increasing amount of time between attempts. This prevents your system from overwhelming an already stressed external provider, which can lead to a Retry Storm.
- **Limitation:** Retries are typically capped at 3 attempts to avoid adding excessive latency. If all retries fail, the issue is considered non-transient, and the circuit breaker is triggered.

Circuit Breaker

The circuit breaker pattern prevents the orchestrator from sending requests to an LLM provider that is clearly and persistently failing.

State	Condition	Action in Orchestrator Context
Closed (Normal)	Everything is working.	All requests go to the primary LLM provider.

Open (Failing)	A threshold of failures (e.g., 5 consecutive 500 errors, or 20% error rate in 60 seconds) is met.	The orchestrator immediately stops sending traffic to this provider and returns an immediate failure/reroute. This prevents wasting time and tokens on a doomed request, and protects the failing service.
Half-Open (Testing)	After a timeout period (e.g., 30 seconds), the circuit breaker transitions to this state.	The orchestrator allows a single test request to the blocked provider. If the test request succeeds, the circuit closes (back to normal); if it fails, the circuit re-opens.

Rerouting

This mechanism provides continuity by routing the request to a different provider or model when the primary one is unavailable.

- **Trigger:** The circuit breaker is in the **Open** state, or the primary LLM is consistently timing out after retries.
- **Action:** The orchestrator switches the request to a **backup LLM provider** (e.g., if the primary is OpenAI, rerouted to a cheaper or alternative model from Anthropic or Google).
- **Intelligent Rerouting:**
 - **Tiered Fallback:** Reroute to a high-cost, high-performance backup first, then to a low-cost, lower-quality model for graceful degradation.
 - **Context Preservation:** The orchestrator ensures the prompt, context, and conversation history are correctly translated and formatted for the new provider's API.

This layer is responsible for reliability. Calls to external LLM providers can fail or time out. It uses **circuit breakers** to stop sending requests to a failing service and automatically **retries or reroutes** the request to a backup model if the primary choice is unavailable.

4.6.5. Safety and validation layer

This is crucial to validate the structure of the response sent back from the LLM provider. This also helps discard responses that are inappropriate or do not follow the guidelines. This checks for the structure, parsing, and content and updates the feedback loop for the response.

4.6.6. Data lake for analysis

We've built a standard analytics pipeline to learn from user interactions. Events from our services are sent to a central event streaming platform, which feeds our data lake built on cloud object storage. By using a modern open table format, we can reliably run large-scale data processing jobs on the collected data. This creates the feedback loop we need to continuously improve our AI models and content.

4.7. Database selection and data modelling

Let's look at the different data we need to store. Our architecture will use different types of databases for different jobs. This ensures we have the best performance, scalability, and reliability for each part of the system.

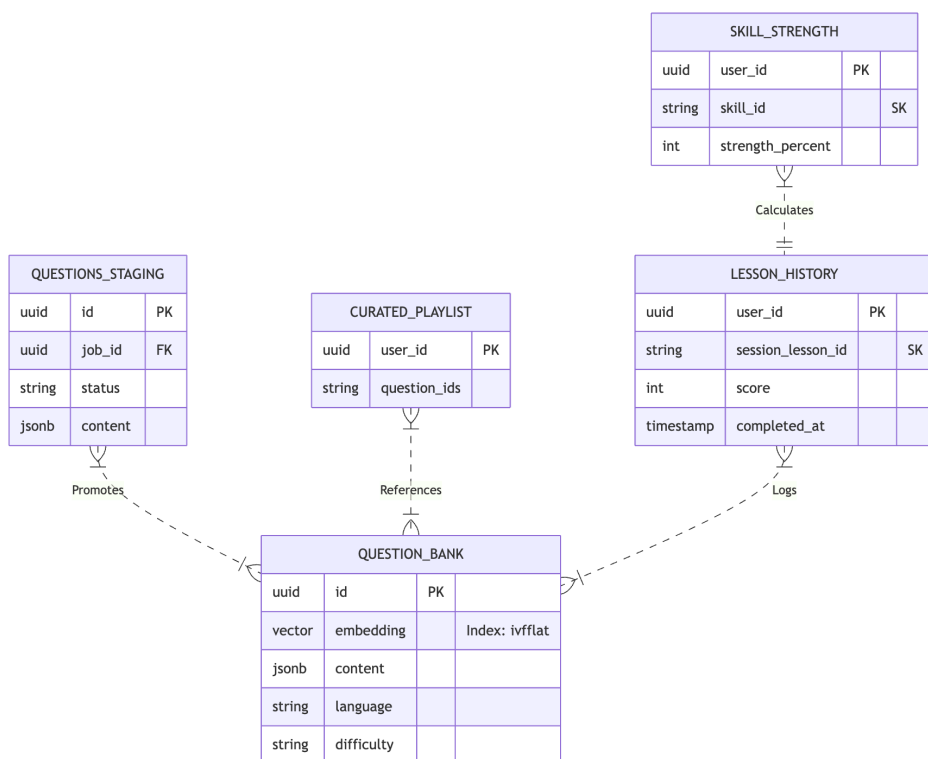


Fig 4.7: Database relations

4.7.1. User Progress data

User progress data is stored in a high-throughput NoSQL database (like Cassandra or DynamoDB) to handle the massive volume of writes from user activity as per peak load 255K RPS. It's separated into two main models to separate the raw history from the calculated summary.

Lesson History

The lesson history is an append-only log, serving as a permanent record of user activity. This data feeds the offline analytics and personalisation pipelines.

Context: An immutable, append-only log of every lesson session completed by a user.

Usage: Written to in real-time upon lesson completion; streamed via DynamoDB Streams to the Data Lake for offline model retraining.

lesson_history

Name	Type	Description
user_id	UUID (Partition Key)	UUID or unique id
session_id#lesson_id	STRING (Sort Key)	
evaluation_score	INTEGER	User's score for the lesson
completed_at	TIMESTAMP	When the session ended

However, storing an infinite append-only log of user history in a high-performance database (DynamoDB) is a financial antipattern. We utilize a lifecycle strategy:

The Hot Store (DynamoDB + TTL)

- Stores only the last 90 days of lesson history.
- Used by the Cold Path for sub-millisecond duplication checks
- TTL: Automatically deletes old records without consuming Write Capacity

The Cold Store (S3 Data Lake)

- DynamoDB Streams capture every write and delete.
- Kinesis Firehose batches these events into Parquet files on S3.
- Used by the Offline Pipeline for model retraining and long-term analytics.

Skill Strength

Context: This is a summary table, like user_skill_strength, that tracks a user's current calculated proficiency for each specific skill (e.g., "Spanish Future Tense").

Usage: Read heavily by the Lesson Curator to decide difficulty and updated asynchronously by background aggregators. This table isn't updated in real-time; instead, it's periodically recalculated by an offline processing job that analyzes the raw data from the lesson_history table.

skill_strength

Name	Type	Description
user_id	UUID (Partition Key)	UUID or unique id
skill_id	UUID (Sort Key)	course joined time
strength_percent	INTEGER	
updated_at	TIMESTAMP	

Since we need to support up to 90k writes per second, we can use a high write throughput database like Cassandra or DDB (or other NoSQL databases)

4.7.2. Curated lessons cache

We store the curated_lessons in a redis lists cache with light RDP snapshotting for faster recovery in case of failures in redis nodes. We use Redis for its millisecond latency, which is essential for the user's real-time experience.

precomputed_lesson_playlists/curated_lessons

Context: A temporary buffer containing a list of Question IDs curated specifically for a user's next session

Usage: Accessed via atomic LPOP operations when a user starts a lesson to ensure <50ms start times

Name	Type	Description
------	------	-------------

user_id	Text	UUID or unique id
question_ids	Text	A list or queue of question_ids for the user's next several lessons.

4.7.3. Admin review lessons data

questions_staging

Context: A holding area for AI-generated questions awaiting automated validation

Usage: Written to by the LLM Orchestrator, polled by the Admin Portal for review dashboards

Name	Type	Description
id	UUID	Unique question id
difficulty_level	Text	Question difficulty level
language	Text	Language the question is testing
content	JSONB	JsonB data of the actual content
question_type	Text	fill in blank, match, dialogue
concept_tags	Array	The concepts that the question is testing
score	Integer	
status	Text	New/In-ReviewAccepted/Rejected

This can be stored in a relational database. However, we might need to search for semantically similar questions, using vector embeddings. We can use a dedicated vector database like turbopuffer or Pinecone or just use PostgreSQL with the pg_vector extension. We will go with the last

approach as we're already using PostgreSQL for storing user data as well.

4.7.4. Lessons data

We need to store structured question data but also support semantic search to find similar questions during content creation. We have structured data but we need to support semantic search for similar content during lesson creation. We can use PostgreSQL with pg_vector extension which lets us handle relational data, flexible JSON, and vector embeddings in one database, avoiding the complexity of a separate vector DB.

Context: The single source of truth for all approved, live educational content, including vector embeddings

Usage: Read-heavy. Queried by ID (Hot Path) or by semantic similarity/tags (Cold Path/Content Creation)

question_bank

Name	Type	Description
id	UUID	Unique question id
difficulty_level	Text	Question difficulty level
language	Text	Language the question is testing
content	JSONB	JsonB data of the actual content
question_type	Text	fill in blank, match, dialogue
concept_tags	Array	The concepts that the question is testing
embedding	Vector(768), Index	Vector embedding of the question for semantic search queries
score	Integer	

We'll use PostgreSQL for storing this data, as it too has high read throughput. It's easy to scale reads on RDBMS like PostgreSQL using read replicas. Since we do not have too much write throughput here, we don't have that concern.

4.8. Deep dive into design

Let's quickly recap the architecture we have discussed so far.

Functional Requirement	Design Solution Summary
Lesson Generation	Offline Pipeline: Decoupled Factory model using LLMs for bulk creation. Human-in-the-Loop: Expert review portal ensures safety before ingestion
Adaptive Curation	Proactive Curation: Pre-fetches personalized lesson buffers into Redis (Warm Path). Hybrid Fallback: Uses rule-based logic for cold-start users.
High Availability	Orchestrator Engine: Implements Circuit Breakers, Retries, and Model Rerouting to handle LLM outages.
Data Persistence	PostgreSQL (Content), DynamoDB (User Progress)
Scale	<i>Next Focus:</i> Designing Async Worker Patterns to handle generation spikes and Read Replicas for the Question Bank.
Grading Accuracy	<i>Next Focus:</i> Implementing Function Calling (Tool Use) to prevent LLM math hallucinations during grading

Now, let's take a closer look at scaling some components and features.

4.8.1. Asynchronous Processing for Content Generation

For heavy tasks like 100s of question generation requests by the admin, a blocking synchronous call is a poor user experience - it would lock up the user's screen and inevitably time out.

We can use an Asynchronous Processing pattern, which uses a message queue to decouple the initial request from the actual work being done.

Let's walk through how this would work:

1. **Job Submission:** The admin initiates a request to generate 100 questions. The client calls an API endpoint, which our **API Service** handles.
2. **Lesson Generation Service:** This validates the request, creates a job entry with a unique jobId, saves it in the database, and pushes a message with the job details onto a Request Queue (`generation_jobs_queue`). It then immediately returns a 202 Accepted response to the client, along with the jobId. Subsequent workers handle up to 10 questions, so if a request comes for 100 questions, Lesson Generation Service will split those into 10 more messages in the `generation_jobs_queue`.
3. **Decoupling via Message Queue:** The `generation_jobs_queue` acts as a buffer. It decouples the **Lesson Generation Service** from the processing workers, handles backpressure during traffic spikes, and ensures that requests are not lost if a processing worker fails.

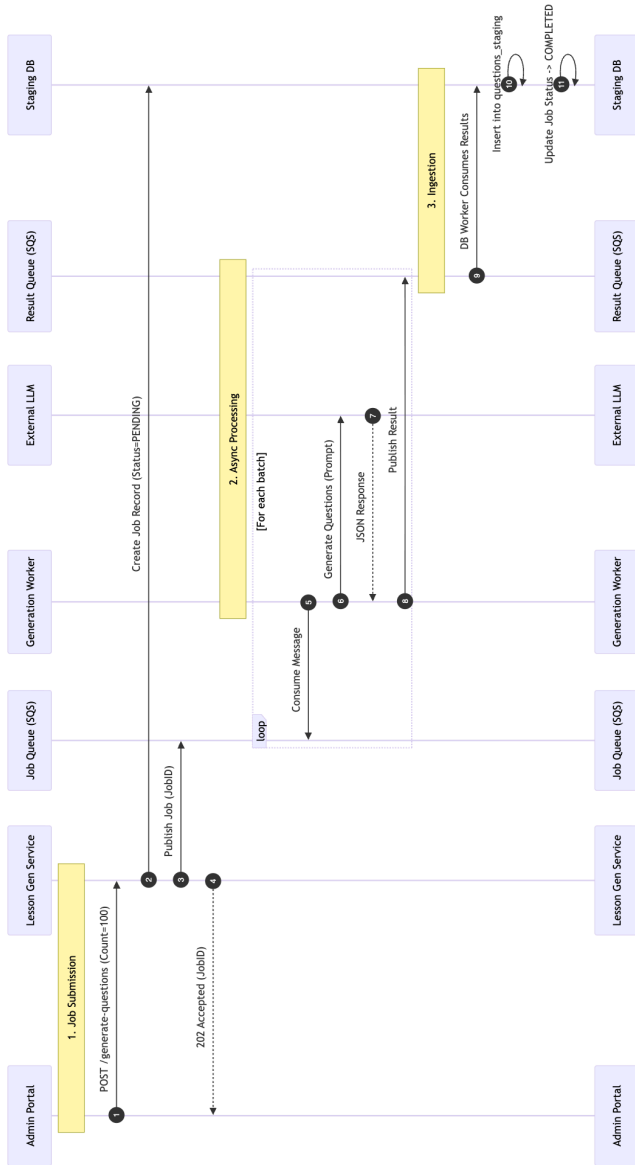


Fig 4.8: Async processing for question generation

4. **Generation Workers:** The Generation Workers within the LLM Orchestrator are consumers of the queue. They handle the long-running CPU-intensive task of generating the new questions by invoking the LLM models and pushes it to the completed questions to a `generation_results_queue`.
5. **DB Ingestion:** The DB worker within the Lesson Generation Service picks up these new questions from the `generation_results_queue` and populates the `questions_staging` database. This worker (or the Lesson Generation Service itself) also updates the database, looking for the completion of all 10 child segments before marking the parent job as `COMPLETED` and compiling the final results URL.
6. **Client Side Status check:** The admin UI, unblocked, can use the `jobId` from Step 1 to poll a status endpoint (e.g., `GET /api/v1/admin/jobs/{jobId}`). This allows for real-time status updates like "`Processing...`" or "`Completed`", improving user experience.

We also adopt a hybrid model if the admin requires immediate responses for trivial tasks. We switch to sync mode if the number of questions to be generated is less than or equal to 5. This avoids the overhead of an async model for trivial tasks and provides instant responses.

4.8.2. Scaling

Scaling our platform requires 2 sub approaches: scaling our services to handle computational load and scaling our data tier to handle the flow of information.

Service Scaling Strategy

We will employ horizontal scaling (adding more server instances) for our services.

To support the 90k rps scale, we can scale up the Curation service instances and the Orchestrator workers. Auto scaling on the orchestrator service should be done based on both pending messages on the queue and the CPU utilization. If many messages are pending on the query, the

workers should scale out. Similarly, if the service is seeing a lot of API requests, the Curation Service instances should auto-scale.

Data Tier Scaling and Caching

We will employ different scaling strategies for each database type.

The Question Bank (PostgreSQL): This is a read-heavy database. To scale it, we can cache calls to the question bank database by maintaining the exercise object per exercise id key in a cache. We will also add read replicas, so that read traffic is distributed in different replica instances.

User Progress Data (DynamoDB): User progress data is tracked in dynamodb which scales horizontally for write throughput. This database will see High write throughput as the curation service. Consumers add the exercise ids and spiky read throughput as users fetch the next exercise.

The Precomputed Lesson Cache (Redis): The `precomputed_lesson_playlists` is present in redis cache. This can be scaled by running it in Redis clusters. If our Redis cluster were to fail and lose all its data, no user would lose their progress or history. The personalization pipeline would simply run again and repopulate the cache with fresh playlists. The worst-case scenario is a brief period where lesson delivery might be slower or less personalized while the cache is being refilled.

Dependency scaling

Handling the scaling and reliability of our external LLM dependencies is the primary responsibility of our LLM Orchestrator Engine.

Multi-provider redundancy

We should integrate with multiple LLM providers to avoid a single point of failure.

We should have circuit breakers and retries and rerouting to different llm models for faster response in case the optimal one has too many requests.

Tiered LLM model

Orchestrators can route to different LLM models depending on urgency or complexity. For simple curation, it can use a simple and cheap model, reserving the use of GPT 4.0 models for conversational models.

Circuit breakers

Each LLM provider is wrapped in a circuit breaker. If a provider's latency spikes or returns too many errors, the circuit breaker trips. The orchestrator then instantly

- reroutes to a fast backup model
- or fails over to a non-AI response

4.8.3. Function Calling for Deterministic Grading

LLMs are probabilistic and can struggle with precise math verification. Instead of asking the LLM "Is this answer correct?", we use Function Calling.

When a user submits a numerical answer, the LLM is prompted to call a `verify_math_solution(user_answer, correct_formula)` tool. The tool executes the math deterministically in a sandbox (e.g., Python) and returns a binary True/False. The LLM then uses this result to generate the feedback message. This guarantees 100% grading accuracy while retaining the friendly, conversational persona of the AI.

4.9. Testing scenarios

We can apply various testing strategies for checking the resiliency of the system.

4.9.1. Performance & Load Testing

The main goal here is to make sure our auto-scaling actually works. We'll use tools like k6 or JMeter to simulate thousands of users starting conversations at the same time. By cranking up the load (10k, 50k, 100k users), we'll find the system's breaking point, measure the wait times, and make sure we have the right auto-scaling rules.

4.9.2. Resilience & Chaos Testing

This is where we verify that the system is truly fault-tolerant. We use chaos engineering principles to deliberately inject failures while the system is under load to make sure our safety nets work:

Kill a Service Instance

Randomly terminate a Conversation Service instance. The test passes if a user's active conversation seamlessly continues on another pod by picking up the session from Redis.

Simulate Network Latency

We'll introduce network delays between services and Redis to confirm that our timeout and retry logic kicks in correctly.

Degrade the LLM

Simulate the LLM provider's API becoming slow or returning errors. This is the real test of our **LLM Orchestrator** to see if it correctly trips its circuit breaker and fails over to a backup.

AI Quality & Regression Testing

LLMs can be a bit unpredictable, so we need a way to make sure the quality of our AI's answers doesn't drift over time. We maintain a dataset of hundreds of common conversational questions and their ideal responses. Any time we tweak a prompt or upgrade a model, we run this test suite to make sure the AI's responses are still accurate, safe, and have the right tone.

4.10. Monitoring and Alerting

Our whole monitoring philosophy is built around four key signals. If we watch these four things, we'll have a really good idea of the system's overall health.

P99 Lesson Serving Latency: We track the end-to-end time it takes to serve a lesson, as this is our primary measure of the user's experience.

Cache Hit Rate: This is a critical alert. A significant drop in the Redis cache hit rate means more users are being forced into the slower, synchronous fallback path, and the team needs to investigate.

Queue Depth: This is our primary alert. If the number of messages waiting in the queue grows consistently, it signals that our workers can't keep up and we need to scale them out.

End-to-End Job Duration: We track the P99 time it takes for a job to go from being enqueued to the final result being cached. This helps us spot bottlenecks in our LLM or database steps.

Worker & Dependency Health: We monitor the error rates of our consumer workers and our external dependencies. A spike in errors from our LLM provider is a key signal that our fallback logic might be kicking in.

Traffic: This tells us how much load the system is under. We're mainly watching the requests per second to downstream dependencies.

Errors: We track the HTTP 5xx error rates from all internal services and external API calls (especially the LLM provider). A sudden spike in errors is a key indicator of a problem.

Saturation: It tells us how close we are to hitting a capacity limit, like the CPU on our servers or, more importantly, the memory usage of our Redis cluster.

All of these metrics are fed into a central observability platform like Grafana or Datadog, giving the entire team a single, real-time view of the system's health.

4.11. Conclusion

So we see that LLMs are pretty powerful tools to power learning platforms in both lesson generation and content personalization flows.

4.12. References

1. [How Duolingo uses AI to create lessons faster](#)
Henry, Parker. "How Duolingo Uses AI to Create Lessons Faster." Duolingo Blog, June 22, 2023.
<https://blog.duolingo.com/large-language-model-duolingo-lessons/>
2. [Introducing Duolingo Max, a learning experience powered by GPT-5](#)
Team, Duolingo. "Duolingo Max Uses OpenAI's GPT-4 For New Learning Features." Duolingo Blog, March 14, 2023.
<https://blog.duolingo.com/duolingo-max/>
3. [Get to know the AI behind every Video Call with Lily](#)
Henry, Parker. "How Duolingo Uses AI to Create the Perfect Speaking Practice." Duolingo Blog, April 22, 2025.
<https://blog.duolingo.com/ai-and-video-call/>

5. Case Study: AI-powered search for e-commerce platforms

How to design intelligent search for e-commerce sites like Amazon, Myntra, Nykaa etc.

5.1. Introduction

Search has evolved from exact keyword and full-text matches to serving the exact needs of the user by understanding intent, and an opportunity for different product discovery.

Understanding user intent behind the search query goes beyond simple spell checks. It has evolved to understand user emotions and context. A search for “post-workout food” should be as well understood by the system as a simple query like “butter chicken.” A search for “cheeseburst pizza” should also show Coke and Pepsi, which pair well with the food.

A search for “old people home footwear” should show soft-soled slippers. We see this is one area where LLM really shines due to its vast knowledge of the world.

Let us formalise the expectations of users in a generic e-commerce application.



Chapter Spotlight

Intelligent Query Pre-processing and Real-time Hybrid Search

5.2. Functional Requirements

- **Natural Language Understanding:** The system must understand and process user queries in natural language, including complex and long-tail queries. For example, "healthy snacks for kids without nuts" or "a summer dress that's not too formal"
- **Semantic Search:** It should understand the user intent behind queries, serving relevant results with or without an exact keyword match
- **Product Recommendations:** The System should suggest relevant products, including complementary items commonly ordered by other users
- **Personalisation:** The search results should be based on the user's past behaviour, preferences, and purchase history.

5.3. Non-Functional Requirements

- **Relevance and Accuracy:** The search results must be highly relevant to the user's query to ensure good conversions
- **Low Latency -** Search results must be delivered to the user in near real-time, typically within a few hundred milliseconds. P90 latency should be <1s, P99 latency should be <5s
- **High Availability-** The smart search must be highly available and resilient to failures (eg failures in LLM providers), as it's a critical component of the e-commerce platform
- **Cost effectiveness -** LLM calls are highly expensive; the system design should be in a way to optimise this cost

- **Freshness:** The search index needs to be updated in near real-time to reflect changes in the product catalog, such as new products, price changes, and out-of-stock items

5.4. Scale estimates

Assuming a mid-large size e-commerce platform with the following statistics:

Daily Active Users (DAU): 10M

Average searches per user per day : 5

Total daily searches: $10M * 5 = 50M$

Queries per second (Avg) = $50M / (60*60*24) = \sim 579$ RPS

Peak Queries per second (assuming 5x average rps) = $(579 * 5) = \sim 2,895$ RPS

Product catalog size: 10M products

Data storage

Product Data (1KB per product): 10 GB

Vector embeddings: With 1536 dimensional embeddings (OpenAI Ada v2), and 4B per embedding = $1536*4* (10M \text{ products}) = 60$ GB

User Data : 10GB

Total storage: 80-100GB

5.5. API design

Search results endpoint

The primary entry point for the storefront; executes the Hybrid Search strategy and returns a personalized, ranked list of products.

Method	GET		
URI	/v1/search		
Query Params	Parameter	Type	Description
	q	string	The user's search query (e.g., "hill station outfits").
	userId	string	The unique ID of the user for personalization
	sessionId	string	unique ID assigned to the user in the current session
	page	int	pagination page number
	filters	string	A URL-encoded JSON string for filtering, e.g., {"brand":"Decathlon","size":"S"}
	sortBy	string	Sort order. e.g., relevance, price_asc, price_desc
Response	{ "searchId": "b4a8-f2c1-e9d0-a7b6", "correctedQuery": "hill station outfits", "results": [{		

```

        "productId": "DEC-FLC-001S",
        "title": "Quechua Women's Hiking Fleece
Jacket - MH120",
        "description": "A warm and breathable
fleece jacket, perfect for cool weather on
your mountain hikes. Made with recycled
polyester.",
        "imageUrl":
"https://example.com/images/decathlon_fleece_b
lue.jpg",
        "price": {
            "amount": 1499.00,
            "currency": "INR"
        },
        "brand": "Decathlon",
        "rating": 4.7
    },
    {
        "productId": "DEC-WJK-015S",
        "title": "Forclaz Women's Padded
Trekking Jacket - MT100",
        "description": "Compact and lightweight
padded jacket designed to keep you warm during
your hill station treks. Water-repellent.",
        "imageUrl":
"https://example.com/images/decathlon_padded_j
acket_black.jpg",
        "price": {
            "amount": 2499.00,
            "currency": "INR"
        },
        "brand": "Decathlon",
        "rating": 4.8
    }
}

```



```
],  
  "relatedSearches": [  
    "winter trekking clothes",  
    "waterproof jackets Decathlon",  
    "thermal wear for women",  
    "hiking shoes"  
  ],  
  "pagination": {  
    "currentPage": 1,  
    "totalPages": 1,  
    "totalResults": 2  
  }  
}
```

5.6. Background

Before we start with the design of an AI-powered search for e-commerce, let's discuss how the search typically works in a traditional system. At the time of restaurant or store onboarding, the merchant uploads dishes or items, which are ingested as documents in a search database like Elasticsearch. During this ingestion process, each dish or store item is thoroughly enriched with metadata attributes by our system, which helps in categorisation and classification.

e.g., when an item like a **Diet Coke 500ml can** is uploaded, its attributes include

Brand: Coca-Cola
Category: soft drink
Sub type: Diet Coke
Form: Can
Size: 500ml

Similarly, when a dish like “spicy ghee rava masala dosa” is uploaded, its attributes include

Dish: Dosa
Flavour: ghee
Grain: Rava
SubType: Masala Dosa
Spice Level: Spicy

These attributes can be added either manually or (as done by larger companies) by using automated pipelines with rules-based systems, machine learning models, and data from suppliers to extract and normalise attributes. This automated pipeline runs on new or updated products to make them fully searchable. In short, it:

- **Uses LLM** to read product info and automatically extract structured tags (e.g., colour, material).
- **Indexes for Keyword Search** by adding the product's text and new tags into Elasticsearch.
- **Indexes for Semantic Search** by creating a vector embedding of the product's meaning and storing it in a vector database.

For the rest of the chapter, we will assume this ingestion and enriching/annotating pipeline has already run. All products in our database are already indexed for both keyword and semantic search.

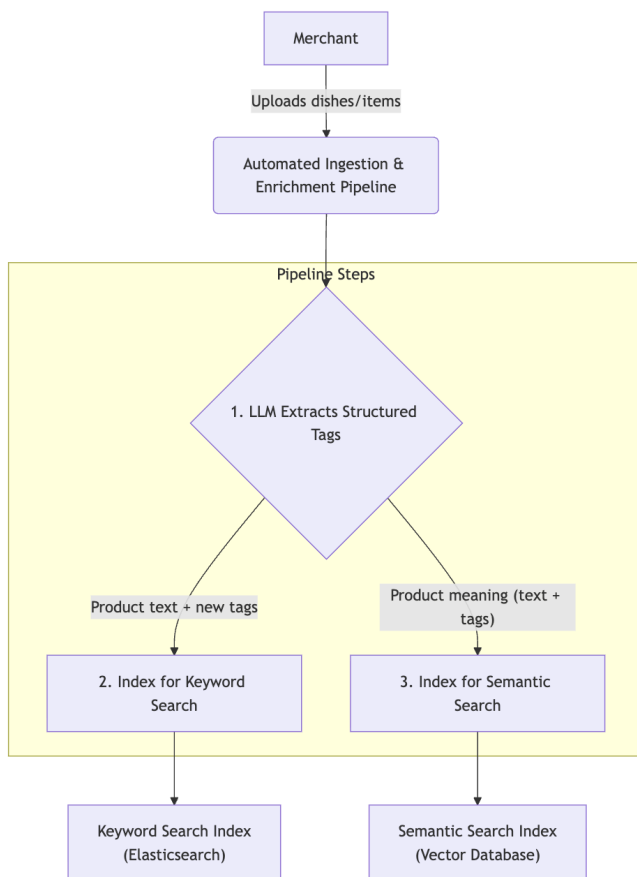


Fig 5.1: Ingestion and annotation pipeline

In a traditional search system, the user's query would be passed on to an Elasticsearch or search database where it would be used for retrieving similar documents. Traditional Elasticsearch, without additional plugins or a vector search setup, primarily uses lexical or keyword-based matching, not true semantic similarity. These documents would then be ranked by the score.

The query added by users can be at times vague and ambiguous, and does not point to a specific dish or item. Or other times, it can be a bunch

of words that make it hard to determine if it refers to one dish or more. While techniques like adding synonyms, spell correction, and autocomplete are used for improving results for ambiguous or vague queries to an extent, these are not foolproof.

LLMs can be pretty powerful in these cases to understand the intent behind the user queries.

With LLMs, we can segment the user query for a better understanding of the intent.

With the ingestion process, we already have a well-defined categorisation and classification system. The user query can be passed on to the LLM along with the classification system context in order for the LLM to determine which attributes are present in the user query. It will automatically also include spell correction and auto-complete steps. We can add in guardrails so that the LLM doesn't hallucinate and only returns the categories we're passing in the context.

We can further employ LLMs for query expansion as well, using alternatives that actually exist in our system.

For discovering new products, we send the same segmented and expanded query to the LLM. The LLM can then suggest related items and items that would pair well with the user's original search.

This process is enhanced by data from analytics pipelines that track user behaviour, like what items users buy together. These behavioural metrics, along with their conversion rates, can be fed to the LLM as additional context. These allow the LLM to make more intelligent suggestions, prioritizing related and complementary items that the users are likely to purchase.

5.7. Blueprint high-level design

We need to minimize latency while improving search intelligence. So, we separate the computationally expensive AI tasks from the real-time search queries. Our architecture will have

1. **Offline query processing pipeline:** Our strict sub-second latency requirement is achieved by decoupling the computationally-heavy AI work into an offline pipeline that populates a low-latency cache.
2. **Real-Time Search Flow:** This serves user requests in real time. We use a dual-path system designed for sub-second responses. It has a Fast path for the majority of the queries and a fallback path.
3. **Product discovery flow:** Improve product discovery and conversions by retrieving complementary items.

5.7.1. Offline Query Processing Pipeline

This runs in the background to form the intelligence that our real-time search relies on. It uses LLMs for computationally heavy tasks where latency is not a concern.

This pipeline runs daily (or more frequently as discussed in deep dive), analyzing user search behavior to continuously improve our search accuracy.

The queries searched by users in the past 1 day are collected and sent as batches to the offline query processing pipeline. For understanding the intent behind the user's search query, each past user query goes through a segmentation and expansion process.

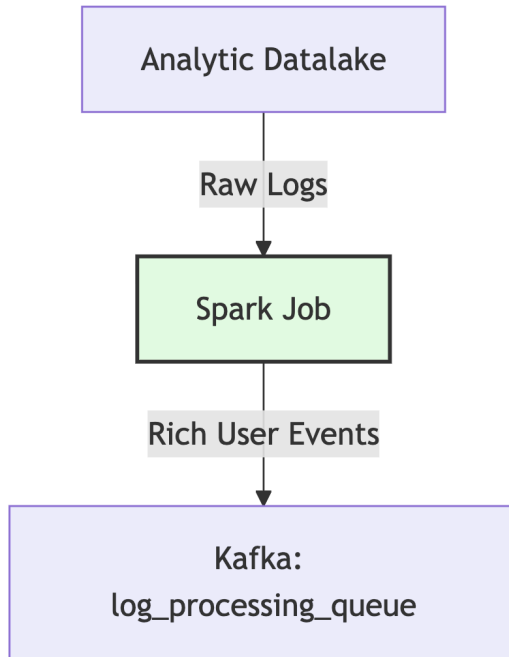


Fig 5.2: Log aggregation spark job

Log Aggregation

A spark job reads raw log files of user queries and their resulting user actions (clicks, purchases) from the analytical data lake storage, groups them by the user session and puts them into a processing queue like Kafka. It creates a rich list of events like:

(UserID: "a", Query: "sofa", Clicked: "Product #123", AddedToCart: "Product #123")
(UserID: "b", Query: "couch", Clicked: "Product #123", Purchased: "Product #123")
(UserID: "c", Query: "hot dogs", Purchased: "Product #456" AND "Product #789")

This is the raw data, connecting user queries to what they buy. During this aggregation, the system explicitly counts the query frequency. This is a key part of the aggregation step. This frequency count is then

used later to determine which queries to pre-process and where to cache them.

Parallelised Enrichment

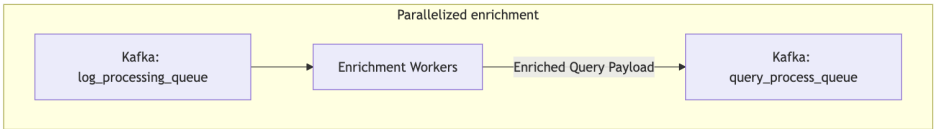


Fig 5.3: Parallelized enrichment

For processing millions of logs concurrently by distributing the workload across instances.

The Spark job takes in batches of this structured data of user queries and processes it in parallel. The log aggregation and enrichment processes are decoupled via a Kafka queue. This helps scale both workers independently, along with providing resiliency in case an enrichment worker fails temporarily.

Each structured log entry undergoes several layers of parallel enrichment:

Query Normalisation: Common spelling mistakes and typos are corrected, and case is standardised and irrelevant characters are removed.

User profile enrichment: We hydrate queries with the user's history, like past purchases, brand preferences.

Contextual enrichment: This includes users' geolocation, time of day, device data, local trends, etc, that affect purchasing decisions.

Each user query is normalised and enriched with the above details and sent as a payload to a **query_process** queue (or Kafka topic).

Query segmentation and classification service

This is responsible for decoding user intent. The query segmentation workers consume the enriched payloads from the **query_process** queue.

This enriched query payload is sent to the Gen AI service with additional details like the platform’s classification structure, etc.

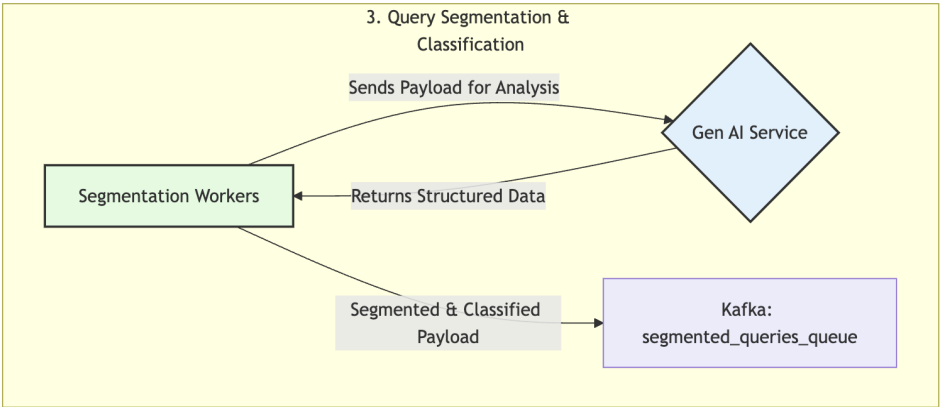


Fig 5.4: Query segmentation and classification

Segmentation: The Gen AI service performs segmentation, structuring the query according to our platform’s taxonomy. For example, the query "organic aged basmati rice" is segmented into:

organic:	Attribute/Tag
aged:	Attribute/Tag
Basmati:	Product Quality
rice:	Product Category

Classification: The model then performs classification. It takes the identified Product Category (rice) and maps it to the precise location within our internal product hierarchy, such as Groceries > Rice, Flour & Pulses > Basmati Rice

The output of this service is a new JSON payload containing this structured, mapped data. This payload, representing the system's deep

understanding of the query's intent, is then pushed to the `segmented_queries` queue.

```
{
  "original_query": "organic aged basmati rice",
  "normalized_query": "organic aged basmati rice",
  "user_profile": {
    "user_id": "user-abc-123",
    "past_purchases": ["p-987", "p-321"],
    "brand_preferences": ["OrganicIndia"]
  },
  "context": {
    "geolocation": "Mumbai",
    "time_of_day": "evening",
    "device": "mobile"
  },
  "segmentation_results": {
    "attributes": ["organic", "aged"],
    "product_category": "rice",
    "product_quality": "Basmati"
  },
  "classification_results": {
    "taxonomy_path": "Groceries > Rice, Flour & Pulses
> Basmati Rice"
  }
}
```

Linking and Query Expansion service

This bridges the gap between what the user said and their true intent, while also proactively suggesting relevant alternatives they may not have considered.

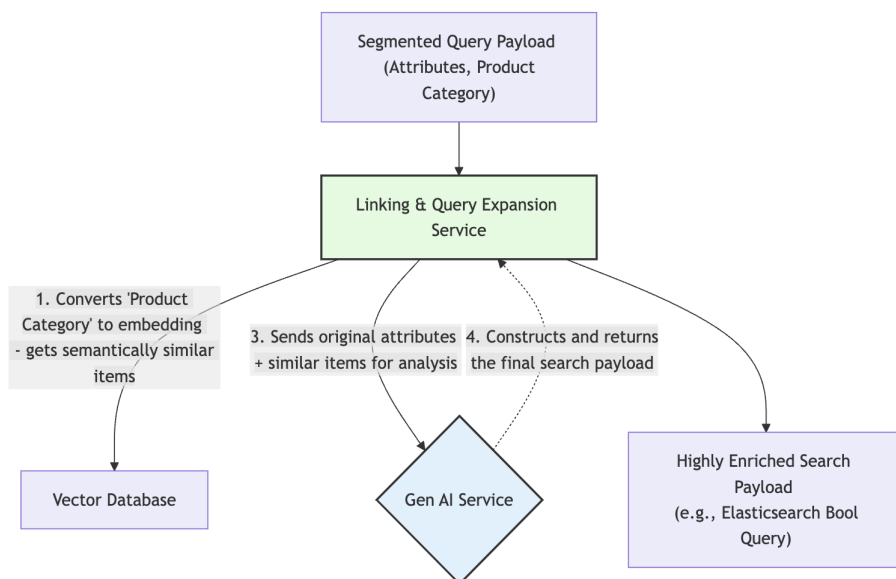


Fig 5.5: Linking and query expansion service

Now we want to link the user query to similar/alternate items in our system based on the intent of the user. The idea is to send the query as a prompt along with items from our catalogue of products to LLM so that it can determine how to best expand the query with alternatives available in our system.

LLMs have a fixed short term memory called context window which is measured in tokens. This limit includes both input prompt and the model's generated output, so a large prompt leaves little room for an answer. So there is a restriction on the number of tokens being sent to in the prompt. We have a huge catalogue of products, and it is not feasible to send the entirety along with the user query to the LLM for query expansion.

The system must first retrieve a small, relevant list of products that easily fits within the context window, allowing the LLM to intelligently refine it for query expansion.

Entity retrieval and linking: This is where vector database and semantic search come in.

The vector database called `semantic_search_index` stores the product description and tags in an embedded form along with the `product_id` to link it to.

The segmented query components (e.g., rice) are converted into vector embeddings (by calling the embedding model or cache) and used to perform a similarity search in our vector database.

Semantically similar items and products for each attribute value are searched. It will retrieve not only "rice" but also semantically similar items like "grains," "poha" and "millets".

Intelligent Query Expansion: It sends these retrieved items to the Gen AI service for the best query expansion with retrieved alternatives available in our system. The Gen AI service picks the most relevant alternatives from the ones retrieved via semantic search, and combines the original user attributes (organic, aged) with these products.

```
{
  "original_query": "organic aged basmati rice",
  "segmented_query": {
    "attributes": ["organic", "aged"],
    "product_category": "rice",
    "product_quality": "basmati"
  },
  "semantic_alternatives": [
    {"product_id": "p-987", "name": "Organic Poha"},
    {"product_id": "p-654", "name": "Barnyard
Millet"},
    {"product_id": "p-321", "name": "Brown Rice
Flakes"},
    {"product_id": "p-111", "name": "Aged Sona Masoori
Rice"}
  ],
}
```

```
"instructions": "You are a search relevance expert
for an e-commerce platform. Analyze the semantic
alternatives and select the most relevant ones for the
original query. Construct a search payload that
combines the original attributes with the base product
and its best alternatives."
}
```

The output is a highly enriched search query payload that includes synonyms and alternatives. It looks like the below:

```
{
  "search_query_payload": {
    "bool": {
      "must": [
        { "match": { "attributes": "organic" } },
        { "match": { "attributes": "aged" } }
      ],
      "should": [
        { "match": { "product_type": "basmati rice" } },
        { "match": { "product_type": "sona masoori
rice" } },
        { "match": { "product_type": "poha" } },
        { "match": { "product_type": "millet" } }
      ],
      "minimum_should_match": 1
    }
  }
}
```

The output is a highly enriched search query payload that includes synonyms and alternatives.

Ranking and Indexing worker

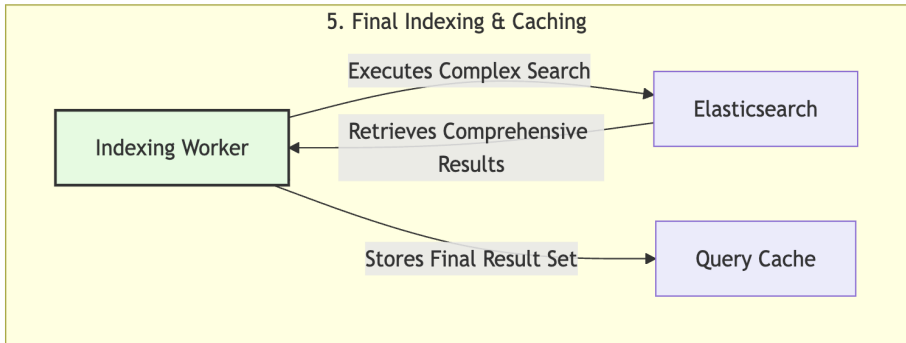


Fig 5.6: Ranking and indexing worker

The final segmented and expanded query is consumed by the Elastic Search worker, which uses it to retrieve the items from the Elasticsearch database by a keyword search.

We cache the enriched Elasticsearch (ES) query instead of the final list of product matches to solve two critical flaws: stale data and high memory usage.

Maintaining Freshness

While caching the final list of product IDs offers the fastest possible latency, it risks showing users incorrect information. If a product's price changes or it goes out of stock, a cached product list would be wrong until the entire offline pipeline runs again.

By caching the ES query instead, the search service executes this query against the live Elasticsearch index every time. This ensures that the results returned to the user always reflect real-time price, stock, and rating information, satisfying the freshness requirement.

Memory Efficiency

Storing the complete list of product IDs for millions of unique user queries would consume a significant and potentially unmanageable amount of memory.

Storing the elasticsearch query is far more memory-efficient than storing a long list of product IDs for every possible search permutation.

5.7.2. Real-Time Search Flow

This is the flow where the user interacts. It is designed for speed and uses a two-tiered approach, a fast path and a fallback path. The user enters an item or a query into the search bar, and the client app calls the server with the user's search query.

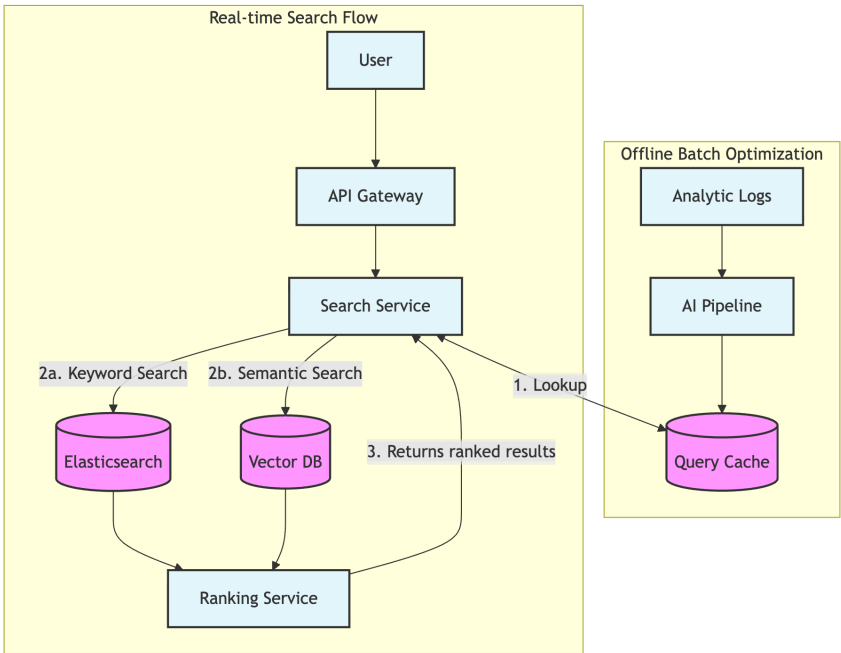


Fig 5.7: Real time search flow

API gateway

User requests are validated for authentication and rate limiting at this component. It routes the request to the search service.

Search Service

This service handles real-time search for sub-second responses. It checks the query cache first - a cache hit retrieves the ES query which is used to retrieve the set of relevant results ranked and sorted. A cache miss leads to fallback on the fast path as below.

Fast Path (Default for 99% uncached queries)

It executes a hybrid search in parallel:

Keyword Search: It calls the document search database (like Elasticsearch) with the query to retrieve the related documents.

Semantic Search: It converts the user's query into a vector embedding on the fly (using the embedding cache or making a call to the embedding model) and makes a call to the vector database to search for semantically similar products.

Re-ranking: The responses from both searches are merged, de-duplicated, and ranked with light-weight ML re-ranking models. This uses criteria like keyword score, semantic score, and product popularity to create the final most relevant list.

Response: If the results have a high confidence score, they are returned to the user immediately.

Fallback Path (for complex queries)

This path is triggered if the fast path returns 0 results or results with extremely low confidence scores.

Trigger: When the Search service detects a failed search from the fast path.

LLM for query understanding: The Search service calls the Gen AI service for query understanding. The goal is to rewrite the user's ambiguous query into system understandable terms.

Example: A user's query "stuff for my dorm party" is rewritten by the LLM to a structured query like (category: "snacks" OR category: "drinks") AND (attribute: "party size")

Retry Search: This rewritten query is taken and sent to the fast path again.

Response: The results from this second, more successful attempt is shown to the user. This ensures that difficult queries also get a good response without slowing down the majority of the queries.

Gen AI Service (LLM Gateway)

This is a centralized intelligent gateway for all interactions with Large Language Models. This encapsulates the dependencies on third-party LLM providers from the rest of the system components. It is responsible for:

Provider Abstraction

It exposes a single unified API for Gen AI tasks. Internal services like Search Service and the Offline query processors need to call only this generic endpoint. This makes it easy to enter and switch any LLM providers as required.

Dynamic Model routing

This component routes the request to LLM providers based on the kind and type of request. It selects models based on:

Cost: For high-volume requests and simpler tasks, it switches to low latency and cheaper models like Gemini 2.5 Flash. For tasks like query rewrite, it can take more optimal LLMs like GPT-5o. The router also maintains a configuration that maps specific tasks to models.

Latency: For real-time online requests, it will prioritize the fastest available model that meets the quality threshold.

Resiliency and Reliability

Since interacting with third party dependencies can be unreliable, we need a robust resiliency layer so it doesn't impact the rest of our system.

Gen AI service implements :

- **Circuit breakers:** so that if one LLM provider is down, it can be sent to the next optimal LLM provider
- **Time-outs:** i.e. if a provider doesn't respond within a threshold duration, the request is cancelled
- **Retries:** It performs intelligent retries with exponential backoff for transient errors

Prompt Templates

It also stores prompt templates for fast generation of prompts

LLM providers

These are actual third-party services or self-hosted models that execute our prompts. They can be of several types:

- **Managed AI Platforms:** They offer access to a variety of models from different companies through a single API. Example: AWS Bedrock, Google Cloud's Vertex AI etc.
- **Direct Model APIs:** These are the APIs provided directly by the AI research labs that create the models. Example: OpenAI (GPT models), Anthropic (for Claude models) etc.
- **Open-Source Models:** These are models that can be downloaded and hosted on our own infrastructure for maximum control over privacy and cost. Example: Llama series, Hugging Face etc.

5.7.3. Product Discovery flow

We want to return related items or items that pair with the items searched by the user. This helps in new product discovery and conversions.

For example, a search for "brownie" should suggest "ice cream" as a complementary item. Our strict sub-second latency requirement is achieved by decoupling the computationally-heavy AI work into an offline pipeline that populates a low-latency cache.

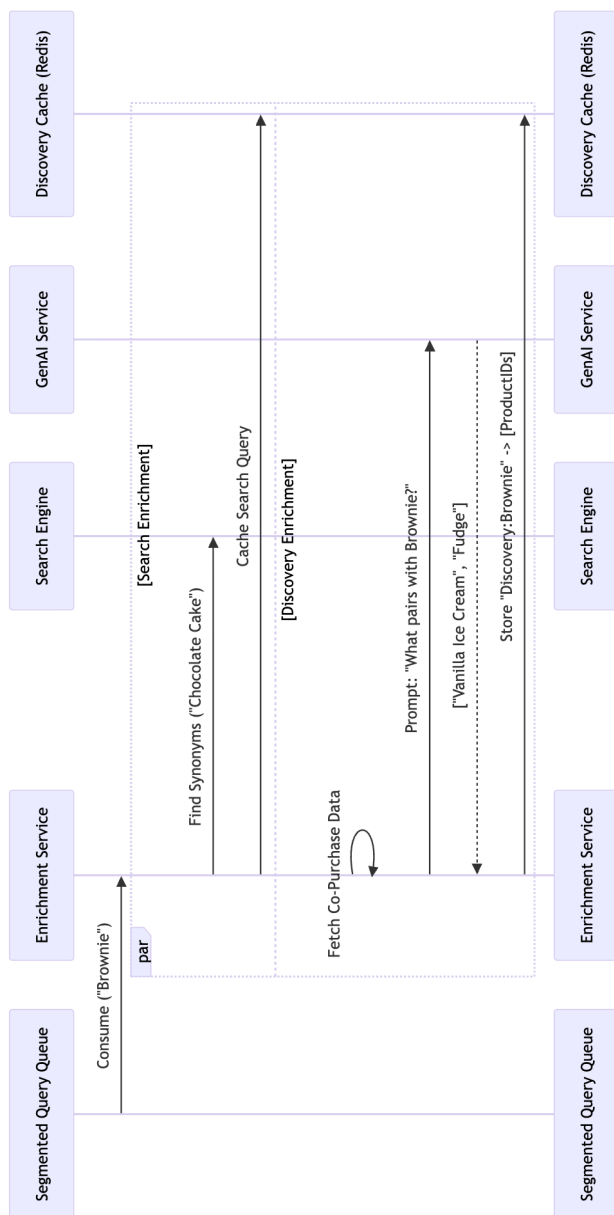


Fig 5.7a: Product Discovery flow

Offline Flow

We discussed how the Query segmentation and classification helps in getting a clear mapping of the product entity and its attributes according to the platform's taxonomy (e.g., product: "brownie").

Our offline pipeline is triggered after the Query segmentation and classification service produces a message in the `segmented_queries` queue. The Enrichment Service consumes this message and performs both search and discovery tasks in parallel:

Search Enrichment:

1. The service performs the linking and query expansion flow and caches the resulting ES query in the `smart_query_cache` as discussed.
2. **Discovery Enrichment:** The service fetches co-purchase analytics for the product. The primary product, along with its rich analytics context, is sent to a Gen AI Service. The prompt in the Gen AI service would look like:

You are an expert e-commerce merchandiser. Given a primary product and data on what customers often buy with it, generate a list of 3-5 complementary products that would enhance the customer's experience. Return the output as a JSON array.

****Input:****

```
{
  "primary_product": "brownie",
  "co_purchase_data": ["chocolate chips", "eggs", "vanilla extract",
    "vanilla ice cream", "chocolate sauce", "whipped cream"]
}
```

****Output:****

```
["vanilla ice cream", "chocolate sauce", "whipped cream"]
```

The resulting list of items is saved in Redis in a **discovery_products** cache with the primary product name as the key.

Online flow

When the user searches for “brownie”, the primary search for brownie products happens as usual. The system makes a parallel call to the Discovery cache to get a list of related products. This list is displayed to the user in a dedicated section, like “Pairs well with” or “Users also bought”.

5.8. Data modelling

There is a variety of data in our system. The storage and access patterns for each differs widely from others. Let’s take a look at all the kinds of data we touched upon in the system design blueprint.

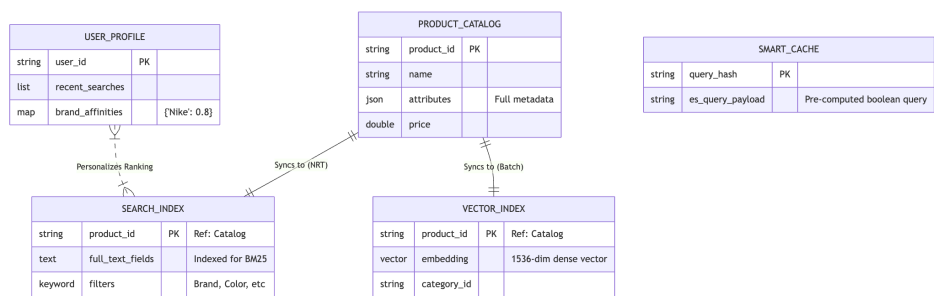


Fig 5.7b: Table relationships

5.8.1. Product Catalog

This handles all product details.

product_catalog

Context: The authoritative source of truth for all product data (pricing, descriptions, stock).

Usage: High Read/Write. Updated by merchant ingestion pipelines; read by the Checkout service and Search Indexers to ensure data consistency.

Column	Type	Description
productId	String (partition key)	Unique identifier
name	String	Product name
description	String	Detailed product description
attributes	JSON object	Key-value pairs of product attributes (e.g., {"brand": "Nike", "color": "blue"})
price	Double (Sort Key)	Price of the product
imageUrl	String	URL of the product image
category	String	Product category
GSI (Global Secondary Index)	category_id(Partition Key) price (Sort Key)	To allow efficient non-search retrieval like "Show me all items in 'Shoes' sorted by Price" without scanning the whole table

This product catalogue will be populated during document ingestion i.e. when new merchants are uploading stores or new restaurants are onboarding etc. This sees a high write throughput when the ingestion pipeline runs. This also sees high read throughput as it is accessed by our search system during both online and offline flows. We can use a NoSQL database like DynamoDB for fast access in this case. The partition key is productId. This ensures all information for a single product is stored together.

5.8.2. Smart Search Cache

This is the cache that the search service queries first when a user query comes in.

Context: A specialized semantic cache that maps a user's natural language query (e.g., "warm coat") to a pre-computed database query.

Usage: Extremely High Read. Hit by the Search Service before any logic is executed. Maps User Query Hash -> Elasticsearch JSON Payload

smart_query cache

Key	user query
Value	ES query

For the persistent tier stored in DynamoDB, the partition key would be the user_query itself.

5.8.3. Product Search Index (for Keyword Search & Filtering)

This is a fast, traditional keyword-based search and handles all structured filtering.

It uses an inverted index to quickly look up products based on keywords in their title, description, and attributes. It's also highly efficient at applying filters (e.g., price < 100, brand = "Nike", color = "blue").

product_search_index

Context: An inverted index optimized for fast keyword matching, filtering, and faceting (counts).

Usage: This is the queried in the Fast Path. Executes logic (e.g., brand="Nike" AND price < 50) in sub seconds.

Field	Type	Description
product_id	Keyword	The unique identifier for the product.

		Mapped as a keyword for exact-match lookups.
name	Text	These are the primary fields for full-text search
description	Text	These are the primary fields for full-text search
brand	keyword	For filtering, sorting
category_path	keyword	
price	float	Allows for numeric range queries (e.g., price < 100) and sorting
in_stock	boolean	true/false
rating	float	
review_count	integer	
attributes	nested	Preserves the relationship between key and value for accurate filtering on multiple attributes at once
attributes.key	keyword	For exact-match filtering on the attribute's name (e.g., "color")
attributes.value	keyword	For exact-match filtering on the attribute's value (e.g., "blue")

imageUrl	String	URL of the product image
created_at	date	for date based queries

We can use Elasticsearch or OpenSearch for querying in sub seconds.

5.8.4. Vector Database (for Semantic Search)

For products that are semantically similar to the user's query, even if they don't share any keywords, semantic search is used. A vector database stores numerical representations (embeddings) of each product. For each product, several fields from the product catalogue are combined into one semantic document before sending it to the embedding model.

For a product with these fields:

name: "Men's Terra Running Shoe"
brand: "Olympus Athletics"
category_path: "Apparel/Footwear/Running Shoes"
attributes: ["trail running", "waterproof", "breathable"]

The semantic document would be:

"Men's Terra Running Shoe by Olympus Athletics. Category: Footwear, Running Shoes. A waterproof and breathable shoe for trail running."

semantic_search_index

Context: Stores the high-dimensional vector embeddings of products to enable conceptual matching.

Usage: Queried during the Hybrid Search phase to retrieve items that match the *intent* but not necessarily the *keywords* of the query.

Column	Type
productId	To link back to the product data
embedding	The high-dimensional vector representation of the product
category_id	String
metadata	other metadata relevant to the product

When a query comes in, its embedding is calculated, and the database performs an Approximate Nearest Neighbour (ANN) search to find the closest product vectors in a high-dimensional space.

This vector database is used in the **online search path for fast search**. It sees a very **high read throughput**. During the offline enrichment for search pipeline, this will see fairly **high write throughput** as well. We can choose to go with a serverless option for vector databases like Turbopuffer to autoscale during load.

5.8.5. Product Discovery Cache

This is the cache that the discovery service queries for fetching related and complementary items from the user query.

product_discovery

Key	user query
Value	ES query to retrieve these product ids

Analytic Logs Store (Datalake)

Context: An immutable append-only ledger of every search event, click, and purchase.

Usage: Write-heavy (Streaming). Consumed by Spark jobs to train the ranking models and populate the Hot Queries list.

Column	Type	Description
event_id	STRING	Unique identifier (UUID) for each individual log entry.
event_timestamp		Precise UTC timestamp when the event occurred. Used for time-series analysis.
user_id	STRING	The unique identifier for the user who performed the action.
session_id	STRING	The identifier for the user's session, used to analyse a single user journey.
raw_query	STRING	The exact, unaltered search query as typed by the user.
filters_applied	MAP<STRING, STRING>	Key-value pairs of all filters applied to the search (e.g., {"brand": "Nike", "price_max": "100"}).
search_type	STRING	The type of search executed (e.g., 'hybrid', 'keyword_only', 'semantic_fallback').
displayed_product_ids	ARRAY<STRING>	An ordered list of the product IDs (SKUs) that were displayed to the user.
clicked_product_id	STRING	The product ID the user clicked. NULL if no click occurred.

added_to_cart_product_id	STRING	Product ID added to cart from search results. NULL if none.
device_type	STRING	The type of client used (e.g., 'web', 'mobile_app_ios', 'mobile_app_android').

5.8.6. User Profile

This is used by the search service to determine the user’s buying preferences and history.

user_profile

Column	Type	Description
userId	partition key	Unique identifier
searchHistory		A list of recent search queries
purchaseHistory		A list of purchased product IDs
preferences		User preferences (e.g., favourite brands, sizes)

A NoSQL database like DynamoDB for fast access. The partition key is userId. This groups all data for a single user in one location for fast retrieval.

5.9. System Design Deep Dive

Let’s recap what we’ve discussed so far.

Functional Requirement	Design Solution Summary
------------------------	-------------------------

Natural Language Understanding	Query Segmentation: LLM breaks queries into structured attributes (e.g., "red" -> Color, "dress" -> Category).
Semantic Search	Hybrid Retrieval: Parallel execution of Keyword Search (Elasticsearch) and Vector Search (HNSW) to capture both exact matches and intent.
Low Latency (<1s)	Tiered Caching: L1 (Local), L2 (Redis), and Warm Tier (Vector Cache) to serve 80% of traffic without hitting the LLM.
Personalization	Re-Ranking Service: Lightweight model re-orders the top 50 results based on user affinity (Brand/Price) in real-time.
Product Discovery	Offline Enrichment: Batch jobs pre-calculate "Pairs Well With" suggestions to enable zero-latency recommendations.
High Scale	Next Focus: Lambda Architecture for handling real-time trends and HNSW Graph optimization for vector scale.

For sub-second search results, we decoupled the online path from the offline intelligence-building path. Let’s explore how that offline pipeline and our caching strategy work to balance powerful, AI-driven results with low latency.

5.9.1. How often should the offline pipeline run?

Frequency depends on business needs and query patterns. While a daily run is standard, this frequency can be increased to hourly during high-traffic events like festivals or flash sales, when user query patterns change rapidly.

However what if a query becomes a viral trend within a few minutes? Running the entire pipeline too often will become computationally expensive.

Near Real time path

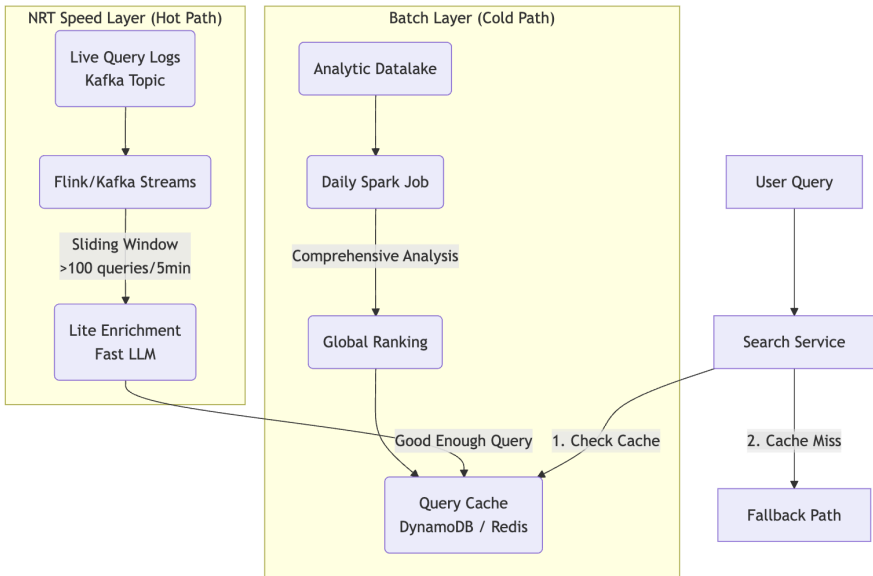


Fig 5.8: Near real time path

This is where a Near Real time path ([Lambda architecture](#)) could be useful.

We keep the daily job for deep, comprehensive analysis and global ranking and additionally:

1. Monitoring Trends:

A separate, lightweight streaming path monitors query logs in real-time by consuming directly from the **query_logs** Kafka topic. We use a stream processing engine (like Kafka Streams or Apache Flink) to perform real-time aggregation.

To detect a trending query, the stream processor applies a sliding time window (e.g., a 5-minute window that slides every 10 seconds). It counts the occurrences of each unique **normalized_query**.

2. **Lite Enrichment Pipeline:** When this NRT path detects a new, trending query (e.g., >100 searches in 5 minutes), it sends just that query through a faster, lightweight version of the segmentation/expansion pipeline. It skips the slow, expensive steps:
 - a. It does NOT perform the full RAG-based query expansion (which involves vector lookups and complex LLM reasoning)
 - b. It DOES perform a simple, fast segmentation. It calls a fast, cheap LLM model with a tight prompt (e.g., "Extract product attributes from this query: ...") to get a moderately good structured query.
3. **Populating the Warm Tier:** This good enough enriched query is immediately written to our Warm Tier cache

So, a query that was new five minutes ago now gets a pre-computed result in the cache. The next user who searches for it gets a warm cache hit, and our Search Service can instantly serve relevant results instead of failing.

4. **Handling Data Overwrites:** When the batch job runs, its writes must always overwrite any data previously written by the NRT speed layer. This ensures that the good enough temporary results are eventually replaced by the perfect globally-ranked results, maintaining the high quality of our search cache over time.

This ensures capture of any quick trends by the streaming pipeline and maintains the comprehensive analysis using the batch pipeline.

5.9.2. Handling caching

A core challenge is deciding what to cache (the value). Caching the final product list offers the fastest possible latency and reduces database load for common queries.

We will primarily cache the Elasticsearch query (the output from the LLM expansion). This ensures that any query run against the live index reflects real-time price, stock and rating information.

Storing the JSON query object is also far more memory-efficient than storing a long list of product IDs.

The trade-off is that this requires a round-trip call to Elasticsearch, which is slower than a direct cache hit and increases load on the search database.

Therefore, we use a hybrid approach:

- **Default:** Cache the Elasticsearch query for all pre-computed long-tail and warm queries.
- **Optimization:** For a small subset of hot generic queries with stable results, we will cache the final product list with a short TTL. This optimization layer shields our database, but the query cache remains our core strategy.

5.9.3. Cache Tiers: Hybrid Strategy for Speed and Scale

We cannot pre-process and cache every unique query in expensive in-memory stores. We use the 80/20 rule, where 20% of queries generate 80% of traffic.

So, we use a tiered approach based on [query frequency, which we got earlier during the Log Aggregation](#) step. Our architecture uses two different cache types.

L1 and L2 Caches (K-V Lookups for Hot Queries)

During [log aggregation, we count query frequency](#) to determine which queries to pre-process and where to cache them:

- L1 Cache (Top 5%): Stored in-memory (in-JVM) for the lowest latency
- L2 Cache (Top 20%): Stored in a distributed cache like Redis

For these caches, the key is a simple, fast hash of the normalized user query. This is perfect for high-frequency, exact-match queries like "iphone" or "running shoes".

Warm Tier (Semantic Cache for the Long Tail)

For the long tail queries (e.g., "winter wear for women"), it is unlikely two users will type the exact same query. A hash key is useless here.

We need a semantic cache. The Warm Tier is a vector database (like OpenSearch with the k-NN plugin, or a dedicated vector store) that stores the pre-processed ES queries.

The lookup flow is:

1. On an L1/L2 cache miss, the Search Service creates a vector embedding of the user's query.
2. It performs a semantic search against the Warm Tier vector store.

If a semantically similar, pre-processed query is found, the service retrieves its cached ES query payload.

This semantic cache lookup is far faster and cheaper than running the full Fallback Path (which involves a real-time LLM call) for every unique long-tail query.

5.9.4. Cache warming

The application instance can't just guess the top 5% of queries when it starts. It needs to warm its cache by fetching this list from a central, persistent source.

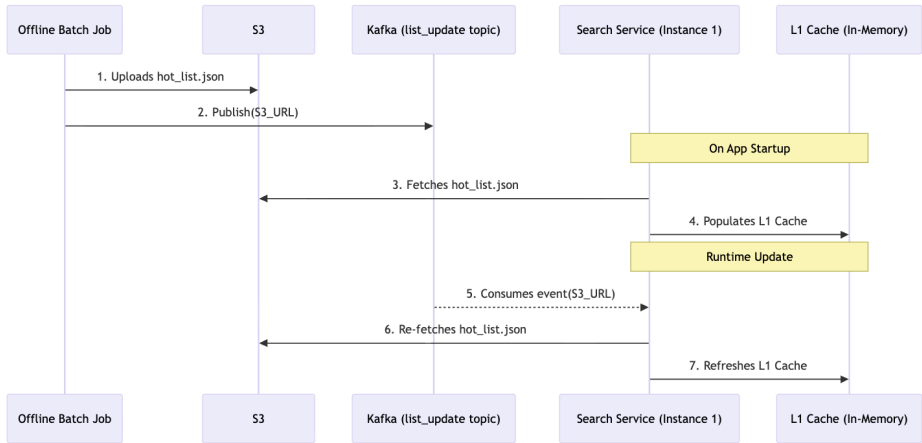


Fig 5.9: Cache warming

This is how it works:

1. Offline job: The [log aggregation pipeline](#) tracks the most frequent queries.
2. Generate hot list: As a last step of the pipeline, it creates a JSON/CSV file containing the top 5% queries and their corresponding pre-computed elasticsearch queries. This list is uploaded to a S3 bucket.
3. Store hot list: It also sends a corresponding notification message (containing the bucket url) to a [list_update](#) topic which is read by the Search service application.
4. Application startup: At search service startup time, it fetches this file from S3.
5. Populate cache: The service then parses this file and loads all the key value pairs into a local in memory store (like ConcurrentHashMap).

6. Runtime updates: All Search service instances are subscribed to the `list_update` topic. Upon receiving the event, they trigger their internal logic to re-fetch and refresh their L1 cache.

This ensures the in-memory cache stays fresh and always represents the current top 5% of queries, making the system highly adaptive to changing user trends.

Cache Invalidation



Fig 5.10: Cache invalidation

Our hybrid strategy requires two different approaches:

1. **For the Query Cache (L2/Warm Tier):** Invalidation is simple. The pre-computed query is just data; it only needs to be updated when the offline pipeline runs and generates a better query for that search term
2. **For the Results Cache (L1/Hot Tier):** This cache risks staleness. We will use a short Time-To-Live (TTL) (e.g., 5-10 minutes) as our primary invalidation strategy. A more robust, event-based invalidation (e.g., updates to the product catalog publish an event to invalidate cache entries) could be added for critical items, but TTL provides the best balance of simplicity and freshness

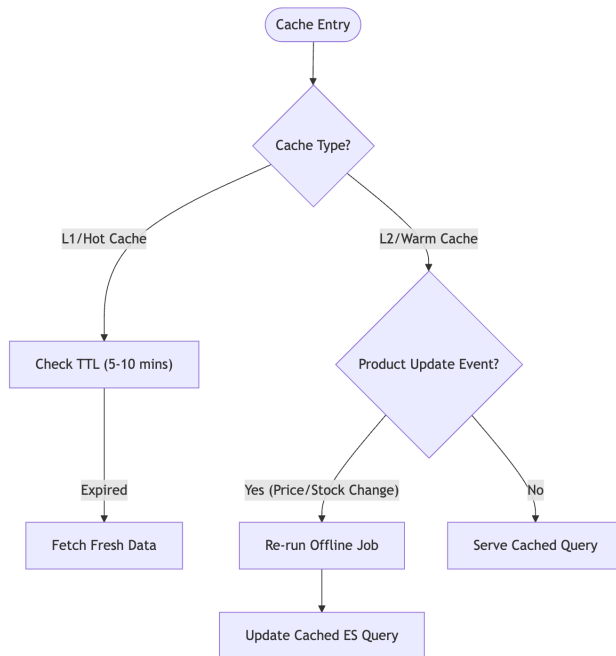


Fig 5.10b: Cache update and expiry

5.9.6. Latency and the P99 Trade-off

Our general P90 latency SLI (Service Level Indicator) should be well under a second for the fast and cached paths.

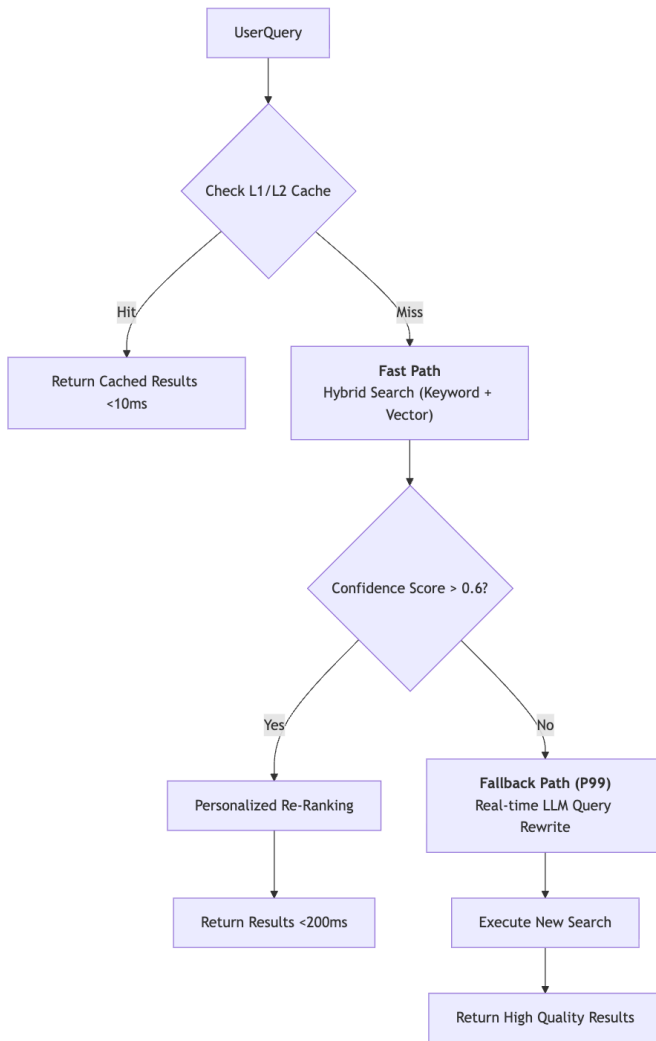


Fig 5.11: Long tail queries served by LLM

For complex, long-tail queries that fail to match any cache tier or semantic search, the system triggers the **Fallback Path** (Real-time LLM Rewrite).

At this specific percentile (P99), we face a critical architectural trade-off: **Utility vs. Latency**.

We must choose between two outcomes:

1. **Be Fast:** Fail immediately and return a "No results found" page in under 200ms.
2. **Show Results:** Execute a real-time LLM chain to understand the intent and rewrite the query, ensuring the user finds what they need, even if it takes longer.

In this design, we prioritize showing results. It is a conscious decision to sacrifice speed to avoid a dead-end customer experience. While the standard search path (P90) is strictly optimized for sub-second latency, the Fallback Path operates on the principle that a slower, successful search is infinitely more valuable to the business and the user than a fast failure.

So, for complex, long-tail queries that are not present in any cache tier (and do not match semantically as well), we must use the Fallback Path (real-time LLM call). In our scenario, returning a good result is more important than a fast "no results found" response. We can therefore accept a higher latency budget (e.g., < 3-5 seconds) for this P99 percentile case to avoid returning zero or low-confidence results.

This ensures that while 90% of users get instant results, the remaining 1% with difficult queries still convert, rather than bouncing off the platform due to a lack of results.

5.9.7. Optimised vector search: ANN

We have different algorithms for vector search using the approximate nearest neighbours.

We need to look at 4 key metrics while deciding on what algorithm to use:

1. High recall or high accuracy
2. QPS supported on a single core for throughput

- 3. Low memory footprint
- 4. Low latency

The algorithm we use also depends on where we store these vectors i.e. in SSD storage or cloud storage, in memory or in secondary durable memory.



Fig 5.12: Optimized vector search techniques

We look at two different algorithms.

HNSW(Hierarchical Navigable Small Worlds) using graph-based partitioning

This creates a multi-layered graph structure using interconnected vectors. Searches start at the top sparse layer to locate the right neighbourhood and then move to the compact dense bottom layers. This uses this graph to traverse nearby vectors. This has high recall and high query performance. This has a large memory footprint since it doesn't compress the vectors. It consumes a lot of RAM, and index build times are high. This is suitable for low latency, high recall when we have a catalogue of 1M to 10M items.

Inverted Partitioning

This groups vectors into clusters and then compresses the vectors in each cluster. This is useful when we need to reduce the memory footprint by a lot. So for nearest vectors, it locates the nearby best clusters to search and then into the compressed vectors inside them. This would be useful if our catalogue had 1B+ items. This has higher latency than the HNSW algorithm and lower recall. This is preferred when memory cost is a constraint.

Algorithm Family	How it Works	Best For	Key Weakness
------------------	--------------	----------	--------------

Graph-based (HNSW)	Connects vectors in a multi-layered graph for efficient navigation	High accuracy & speed at moderate scale (millions)	High memory usage
Inverted Partitioning	Group and compress vectors into clusters.	Massive scale (billions) where memory is the constraint.	Lower accuracy (recall)

Based on the above analysis, we decide to go with HNSW based partitioning since our catalogue size is within limits for a medium memory footprint. It ensures a highly accurate and low-latency experience for our users.

5.10. Efficient ranking

To deliver highly relevant and personalised results with low latency, we apply ranking twice. We perform a generalised ranking in the offline search path to establish a baseline, and a lightweight, personalised re-ranking online to tailor the results for each specific user.

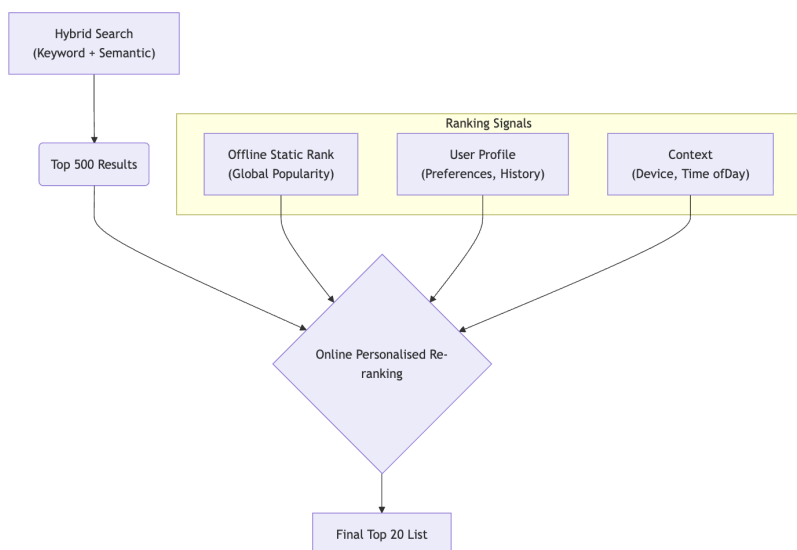


Fig 5.13: Personalized Re-ranking

5.10.1. Offline Static Ranking (Global Relevance)

This assigns a ranking to every product-query pair based on the collective behaviour of all users. This doesn't fit any particular user.

Key signals used to calculate this score include:

Collective User Behaviour: We analyse historical data to see which products perform best for certain queries. A purchase is given more weightage than an add-to-cart, which in turn is weighted more than a click.

Textual Relevance: The semantic and keyword match between the query and the product's title, description, and attributes.

Product Popularity: A product's overall sales velocity, number of views, and ratings, independent of any specific query.

The output of this phase is a global relevance score that is stored alongside the product in our search indexes and cache.

5.10.2. Online Personalised Re-ranking

This reorders the retrieved list based on the user who is searching.

This lightweight model uses real-time user data to re-rank:

Short-Term Intent: If the user has recently seen a brand's shoes, then we show more items from the same brand. So items of the same brand are scored more.

Long-Term Preferences: For historical data of users' past history and conversions, we give more score to the kind of items users have bought i.e. “organic” items etc.

Context: Other data like the user's device, location, or time of day, can also be used to fine-tune the final ranking.

This final, personalised list is what the user sees, ensuring that the most relevant products for them are always at the top.

5.11. Ensuring Robustness: Availability, Scalability, and Resilience

5.11.1. High availability

High availability ensures our system remains available even if individual components of the system or entire data centres fail.

We use the following methods to make the system highly available:

Multi-AZ Deployments: All critical components of our online path—including the application services, Elasticsearch clusters, vector databases, and Redis cache and DynamoDB—are deployed across

multiple Availability Zones (AZs). This eliminates a single point of failure in case the data centres of one AZ go down.

5.11.2. Scalability

Scalability is the ability of the system to handle growing amounts of load.

Horizontal Scaling of Services: Our microservices (e.g., Search Service, Ranking Service) are stateless. This means any instance can handle any request, allowing us to simply add more instances to handle more traffic. We use Horizontal Pod Autoscalers (HPA) in Kubernetes to automatically scale the number of instances based on real-time CPU and memory usage.

Scalable Datastores: We choose datastores built for scale. DynamoDB and our serverless Vector Database automatically handle sharding and scaling to accommodate massive throughput. DynamoDB uses consistent hashing to scale efficiently.

Managed sharding

As data grows, it is not feasible to store it all in one server. The database automatically splits the data into shards and stores them in different servers.

Managed throughput

As the database load increases, the database needs to increase its compute power.

DynamoDB does this by automatically scaling up and down to match the traffic (in on demand mode). In turbopuffer, it automatically spins up the necessary compute resources to handle the load and then spins them down, so you only pay for what you use.

Vector database scaling

A serverless vector DB generally scales using a decoupled storage and compute model.

Storage: The vector data and its index (like the HNSW graph) are placed in a highly durable, scalable object store (like S3). This storage can grow to petabytes without any manual management.

Compute: When a query is sent, Turbopuffer's serverless architecture activates the necessary compute resources.

Parallelism: The query is sent to many stateless compute nodes in parallel. Each node loads only the part of the index it needs from the object store, searches its portion, and returns its top results.

Aggregation: A final service aggregates the results from all the parallel nodes to find the absolute nearest neighbors and returns them.

It scales because it can dynamically add more compute nodes as query volume increases, and we only pay for the compute-seconds we actually use for each query.

Asynchronous Processing: The offline pipeline is designed for scalability through decoupling. By using message queues like Kafka, we separate the Spark jobs for log aggregation from the enrichment workers and the Gen AI services. All these individual components can be scaled individually and independently. We can scale them according to cpu and memory along with queue size.

5.11.3. Resilience and Fault Tolerance

The resiliency of a system determines how it handles partial failures gracefully.

Load balancing

A load balancer sits before each service. It distributes traffic evenly across healthy instances for scalability. It also automatically redirects traffic away from any instance that becomes unresponsive, thus isolating failures.

Database Failover

For stateful systems like our Redis cache, we operate in a primary-replica configuration across AZs. We have backup clusters for failover. If the primary node fails, a replica is automatically promoted to become the new primary with minimal disruption.

Kafka: Failover is handled by replication. For each partition, there is one leader broker and multiple follower brokers. Followers constantly copy data from the leader. If the leader broker fails, Kafka's controller (running in Zookeeper or KRaft) automatically promotes one of the in-sync followers to be the new leader. This happens in seconds, and producers/consumers are redirected to the new leader.

DynamoDB: Failover is automatic and managed by AWS. DynamoDB is deployed in a Multi-AZ configuration. This means your data is synchronously replicated to at least three different physical datacenters (Availability Zones). If one entire datacenter fails, DynamoDB automatically routes all traffic to one of the healthy, replicated copies in another AZ with no data loss and minimal disruption.

Turbopuffer (Serverless Vector DB)

Durable Storage: The vector data is replicated across multiple AZs in an object store.

Stateless Compute: The query-handling compute instances are stateless. If one fails, the load balancer simply removes it and routes traffic to healthy ones.

This combination ensures both data durability (no data is lost) and high availability (queries can almost always be served).

Circuit Breakers

Interacting with third-party LLM providers is inherently unreliable. The gen AI service has circuit breakers for each LLM model. The circuit opens and

traffic is re-routed to a fallback model in case the primary model encounters timeouts or failures. This prevents a single failing dependency from freezing our entire offline pipeline.

5.12. Monitoring and alerting

We need to monitor and alert our services, Spark jobs and other consumers along with our data stores.

5.12.1. Key metrics to monitor

We need to proactively understand the health of our system from both a technical and a user-centric perspective. We look at the health of our machines and the pulse of our users.

Latency for the search API

We track the **p95 and p99 latency** of our search API. This tells us the experience of the slowest 5% and 1% of our users.

Error Rate

We monitor the percentage of server-side errors (5xx). A sudden spike is our earliest warning that a service or dependency is failing.

Throughput

Measured in Queries Per Second (QPS), this helps us with capacity planning and identifying unusual traffic patterns, like a flash sale or a potential DDoS attack.

CPU and memory utilisation

These are fundamental for cost optimization and triggering our autoscaling rules. Consistently high usage tells us it's time to scale up.

Click-through rate for discovery

This measures the percentage of searches where a user clicks on a result. It's a direct signal of relevance: "Are we showing people things they find interesting?"

Conversion rate for search and discovery

This tracks the percentage of searches that lead to a purchase. This is our ultimate business metric, telling us if search is successfully driving revenue.

5.13. Alerting

We trigger alerts on anomalies that require human intervention:

Critical Errors

A sharp increase in the API error rate.

Performance Degradation

When p99 latency exceeds our defined threshold (e.g., 500ms).

Resource Exhaustion

When CPU or memory utilization maxes out on our services or databases, indicating a scaling issue.

Business Metric Drops

A sudden, unexplained drop in CTR or conversion rate, which could signal a problem with a new algorithm deployment.

5.14. Testing and Evaluation

5.14.1. LLM as a judge

We use a powerful Large Language Model (like GPT-5) as an impartial expert. We present the LLM with a query and the search results from both the old and new algorithms. We then ask it: "Which set of results is more relevant and helpful for the user's query? Explain your reasoning." This gives us a large-scale, qualitative signal on whether our changes are directionally correct, helping us iterate faster. We use Golden datasets to have a baseline for what the ideal responses should look like.

Judge Model: GPT-4o or Claude 3.5 Sonnet (High reasoning capability).

Scale: 1-5 (1 = Irrelevant, 5 = Perfect Match).

Cost Control: Run this on a random sample of 500 daily queries, not live traffic.

Implementation Code

```
from pydantic import BaseModel
from openai import OpenAI

client = OpenAI()

class RelevanceScore(BaseModel):
    score: int
    reasoning: str

def evaluate_relevance(user_query, product_title,
product_desc):
    """
    Uses GPT-4o to act as an impartial judge of search
    quality.
    """
```



```

prompt = f"""
    You are a Search Quality Rater. Rate the relevance
    of the product to the query.

    Query: "{user_query}"
    Product: "{product_title}" - {product_desc}

    Scale:
    1: Completely irrelevant (Query: "Phone", Result:
    "Socks")
    3: Tangentially relevant (Query: "Nike Shoes",
    Result: "Adidas Shoes")
    5: Perfect match (Query: "iPhone 15", Result:
    "Apple iPhone 15 Pro")

    Return JSON with 'score' (1-5) and short
    'reasoning'.
    """

completion = client.beta.chat.completions.parse(
    model="gpt-4o-2024-08-06",
    messages=[{"role": "user", "content": prompt}],
    response_format=RelevanceScore,
)

return completion.choices[0].message.parsed

# Usage Example
# query = "winter hiking jacket women"
# product = "Quechua MH100 Fleece" (from our search
# engine)
# result = evaluate_relevance(query, product.title,
# product.desc)
# if result.score < 3:

```

```
# flag_for_human_review(query, product)
```

5.14.2. Golden Set Regression Suite

Before any code hits production, it must pass the Golden Set. These are non-negotiable, high-value queries where failure is not an option (e.g., "iphone" must return Apple products).

Implementation: Use pytest to block deployments if core queries degrade.

```
# test_search_quality.py
import pytest
from search_client import get_search_results

# The "Golden Set": Queries that MUST return specific
# brands/categories
GOLDEN_CASES = [
    ("iphone", "Apple", "brand"),
    ("running shoes", "Footwear", "category"),
    ("protein powder", "Supplements", "category"),
    ("sony headphones", "Sony", "brand"),
]

@pytest.mark.parametrize("query, expected_value, field",
                           GOLDEN_CASES)
def test_golden_set_accuracy(query, expected_value,
                              field):
    """
    Blocks deployment if top 3 results don't match the
    golden expectation.
    """
    results = get_search_results(query, limit=3)
```

```

# Fail if 0 results - instant blocker
assert len(results) > 0, f"Critical: No results for
golden query '{query}'"

# Check if the top result matches our expectation
top_result = results[0]
assert expected_value in top_result[field], \
    f"Regression: '{query}' returned
{top_result[field]}, expected {expected_value}"

```

5.14.3. A/B testing

We roll out any significant change, like a new ranking algorithm, to a small subset of users (e.g., 5%). This group's behavior is then compared against the control group using the existing system. We closely monitor key metrics like CTR, conversion rate, and revenue per user. Only changes that show a statistically significant improvement are gradually rolled out to all users.

Control (A): Keyword Search + Simple Vector (Baseline).

Treatment (B): The new Hybrid + Re-ranking pipeline.

Metric to Watch: Revenue Per Session (RPS)

Implementation

```

import hashlib

def get_experiment_bucket(user_id,
salt="experiment_v2"):
    """
    Deterministic bucketing. User X is ALWAYS in the
    same bucket
    for the duration of the test.
    """

```

```

hash_input = f"{user_id}-{salt}".encode('utf-8')
# Hex digest to integer (0-100)
bucket_val =
int(hashlib.sha256(hash_input).hexdigest(), 16) % 100

if bucket_val < 50:
    return "CONTROL" # 50% traffic
else:
    return "TREATMENT" # 50% traffic

# In the API Handler
def handle_search_request(request):
    bucket = get_experiment_bucket(request.user_id)

    if bucket == "TREATMENT":
        return run_new_ai_search(request.query) # The
new fancy system
    else:
        return run_legacy_search(request.query) # The
old dependable system

```

5.14.4. Model Routing & Fallback Logic

We do not send every query to an expensive LLM. We use a Router to decide the path.

Fast Path (P90): Elasticsearch + Cached Vectors

Slow Path (P99): Real-time LLM Query Rewrite

Routing Strategy

Exact Match: If query exists in L1_Cache -> Return immediately.

High Confidence: If Vector Search returns scores > 0.85 -> Return immediately.

Fallback: If top result score < 0.5 -> Trigger LLM.

Implementation

```
def search_orchestrator(query, user_id):
    # 1. Check Cache (Speed: < 10ms)
    cached = cache.get(query)
    if cached: return cached

    # 2. Fast Path: Hybrid Search (Speed: < 200ms)
    # Uses cheap embeddings (e.g., OpenAI
    text-embedding-3-small)
    results = hybrid_search_engine.search(query)

    top_score = results[0].score if results else 0

    # 3. Decision Gate: Is the result garbage?
    if top_score > 0.65:
        return results # Good enough, don't waste
money

    # 4. Fallback Path: The "P99" Rescue (Speed: 2-3s)
    # Trigger expensive model only when necessary
    print(f"Low confidence ({top_score}) for '{query}'.
Triggering LLM rewrite.")

    rewritten_query = llm_rewrite_query(query) # Uses
an SLM (cheaper, fast)
    rescue_results =
hybrid_search_engine.search(rewritten_query)

    return rescue_results
```

5.14.5. Human evaluation

We have a team of human evaluators who review results for a curated set of challenging and ambiguous queries. They look for subtleties that automated systems can't—like contextual appropriateness, serendipity, and fairness. This human-in-the-loop feedback is invaluable for catching edge cases and ensuring our search experience feels truly intelligent.

Human-in-the-Loop for NRT Trends

The Near Real-Time (NRT) pipeline detects viral trends. However, automatically promoting a trending query to the cache using a cheap LLM creates a Viral Hallucination risk.

- **Guardrail:** Output from the NRT pipeline should go to a Staging Cache.
- **Review:** High-velocity trends trigger a Slack alert to a Merchandising Manager who must click Approve to push the AI-generated search results to the public Live Cache.

5.15. Conclusion

Building an AI-powered search for e-commerce is a complex challenge. By leveraging the power of LLMs and vector search, we can create an intelligent and personalized discovery experience for our users. The future of e-commerce search is not just about finding products; it's about understanding customers and helping them discover things they'll love.

5.16. References

1. [Enhancing Search Retrieval with Large Language Models \(LLMs\)](https://blog.picnic.nl/enhancing-search-retrieval-with-large-language-models-llms-7c3748b26d72)
Sukel, Maarten. "Enhancing Search Retrieval with Large Language Models (LLMs)." Picnic Engineering, May 14, 2024.
<https://blog.picnic.nl/enhancing-search-retrieval-with-large-language-models-llms-7c3748b26d72>

2. [How DoorDash leverages LLMs for better search retrieval](#)
Martinez, Eduardo. "How DoorDash Leverages LLMs for Better Search Retrieval." DoorDash, November 19, 2024.
<https://careersatdoordash.com/blog/how-doordash-leverages-llms-for-better-search-retrieval/>
3. [How Instacart Uses Embeddings to Improve Search Relevance](#)
Xiao, Xiao. "How Instacart Uses Embeddings to Improve Search Relevance." Tech-at-Instacart, September 15, 2022.
<https://tech.instacart.com/how-instacart-uses-embeddings-to-improve-search-relevance-e569839c3c36>
4. [Grab - LLM-assisted vector similarity search](#)
Grab Tech. "LLM-Assisted Vector Similarity Search." Accessed December 5, 2025.
<https://engineering.grab.com/llm-assisted-vector-similarity-search>

6. Case Study: AI-powered Customer Support Agent

Companies can receive several thousand customer support tickets that executives need to handle, creating a massive operational load. Relying just on human agents leads to huge turn around time (TAT) for the tickets which can create a poor user experience. To help customer support executives handle their loads, there are automated support agents.

Traditional support agents have predefined workflows and fail for queries that deviate even slightly from expected, typical categories or simple FAQs. Furthermore, a significant architectural constraint for these systems is the huge amount of data that companies maintain. They struggle to efficiently traverse a large, complex, and constantly evolving knowledge base like deep-diving into a massive documentation portal, to pinpoint a specific, context-relevant paragraph within a long article. We can leverage LLMs in these use cases that drastically reduce load on human agents and create robust support flows.

For this chapter, we will design an AI-powered customer support agent for MongoDB.

Developers engaging with MongoDB documentation face a vast array of issues, from nuanced queries about feature functionality to frustrating connection troubleshooting. The AI agent should be smart enough to automatically handle queries that it can, and redirect to human agents otherwise. It should also summarize the chat history between the customer and the bot to help the human agent get a concise, actionable overview.

Footnote: We choose the MongoDB ecosystem as a case study due to its immense feature breadth and the massive scale of its official and community knowledge base,

which perfectly illustrates the limitations of traditional support systems and the need for advanced GraphRAG solutions.



Chapter Spotlight

GraphRAG & Hybrid Retrieval When Vector Search is not enough

6.1. The Standard RAG Pattern

As established in Chapter 2, our foundation is a standard Retrieval-Augmented Generation (RAG) pipeline. We ingest raw MongoDB documentation and historical support tickets, chunking them into semantic units. These chunks are embedded using `text-embedding-ada-002` and stored in our Vector Database (OpenSearch) to enable high-speed semantic retrieval.

Under normal circumstances, this allows us to retrieve content that is semantically similar to a user's query to ground the LLM and reduce hallucinations.

However, in a complex technical support domain, semantic similarity is not the same as relevance. A vector search might find 10 documents that seem like error E11000, but fail to find the one document that explains the root cause because the phrasing is different. To solve this, we must evolve from Standard RAG to GraphRAG.

6.1.1. Why is naive RAG not enough in this case?

A naive RAG often falls into two critical limitations.

Similarity vs true relevance

Vector search excels at finding similarity but that is not always strictly relevant.

Several documents may be framed similarly, which leads to the retrieval of several semantically similar chunks which may be related to the query. So the LLM receives redundant, overwhelming context, which makes it difficult to pinpoint the exact solution which is needed.

Inefficient retrieval due to poor chunking

Documents are often organized hierarchically (Title->Section->Sub section)

If a user query matches the title of a document, the system may retrieve the high-level summary but miss the critical resolution located five sections down. The core piece of information the user needs is missed because the embedding model didn't recognize the connection between the high-level match and the low-level detail.

6.1.2. Utilizing a knowledge graph

To mitigate the limitations of semantic-only retrieval, we introduce a Knowledge Graph (KG). A KG is a structured representation of data defining relationships between entities. Graph databases like Neo4J or Amazon Neptune can be used for this. A KG makes it easier to retrieve a particular entity based on its relationship with other entities.

Instead of relying on word similarity, the KG understands relationships such as:

Document A -[is_part_of]-> Section 3, or
Ticket X -[resolved_by]-> Code Commit Y.

This allows the retrieval engine to traverse complex, logical paths to find the definitive solution set.

6.1.3. Adopting the graphRAG Pattern

We utilize the GraphRAG pattern to traverse complex relationships between entities, to get relevant, inter-connected info for navigating structured and domain-specific data.

6.2. Functional Requirements

- **Query Intent Detection:** The agent must accurately understand the intent and context of user queries expressed in natural language. It should remember the conversation history to handle back-and-forth dialogue.
- **Accurately Retrieve Solutions:** The system must use semantic search across the Knowledge Base and past tickets to retrieve relevant solutions and answer the customer accurately.
- **Intelligent Escalation and Handoff:** The agent must reliably detect when a query is unanswerable (i.e., low confidence score). In such cases it should seamlessly escalate/handoff to a human agent including a summarized context of the chat.
- **Maintaining Freshness:** The Knowledge Base ingestion pipeline must ensure the system's training and retrieval data is kept fresh and synchronized with the company's documentation repo, latest patch notes etc. within a definite period

6.3. Non-functional requirements

- **TAT for replying on tickets:** Upto 3 seconds for the first reply on ticket. Upto 1 min for follow up / case resolution or escalation to human agents.
- **Accuracy and Hallucination Control:** Should be able to solve user queries accurately (avoid hallucinations)
- **Reliability and Fault Tolerance:** should be able to answer queries reliably even when parts of the system are down
- **Scalability:** The architecture must be capable of processing up to N concurrent sessions and handling projected peak load increases

- Data Security and Privacy:** The system must follow all relevant data privacy regulations (e.g., GDPR, CCPA) and implement masking or redaction protocols for any Personally Identifiable Information (PII) before it is processed or stored in internal logs.

6.4. Scale estimates

We must quantify the expected load to properly size the infrastructure for the Non-Functional Requirements (NFRs), particularly Latency and Scalability.

6.4.1. Input metrics

Metric	Baseline Estimate	Description
Peak Support Tickets/Day	3000	
Agent Containment Rate (Goal)	70%	Automate most basic and intermediate support tickets
Peak Concurrent Chat Sessions	200	Peak hour traffic will be 5-10% of the daily total

6.4.2. Source document volume

Source Component	Estimated Count / Volume	Description
Official MongoDB Documentation	5,000–7000 core pages/articles	Documentation, guides, tutorials, etc.

Internal Support Tickets (Historical)	50k resolved tickets	Closed tickets with RCA and resolution logs
Community Forums / Stack Overflow / Github Issues/ Reddit	100k threads/posts	high-volume source of common issues
TOTAL RAW SOURCE ITEMS	≈160k	Amount of data that the ingestion pipeline must manage

6.4.3. Vector database volume

Metric	Estimate	Description
Average Chunk Size	256-512 tokens	Balance between retaining context and preventing the LLM's context window from being overwhelmed
Chunking Ratio	10 chunks per item	Chunk counts vary widely, but we take 10 as average
Total Vector Chunks	~1.6 Million vectors	160k total source * 10 chunk per source
Vector Dimensionality	768 to 1536 dimensions	Depends on the chosen embedding model
Estimated Storage Size	~9.8 GB + metadata = 20GB	1.6 Million vectors * 1536 dimensions * 4 bytes for float32

- Total size of chat message data in database (assuming 3k customer support tickets per day and 20 messages per chat on avg)

Per day data assuming 500B per chat message

$$3k * 20 * 500B \approx 30MB$$

- Daily number of documents ingested
~2k (1% of the 160k total items are updated daily)

6.5. High level design

Here's what a high level overview of the system would look like. We will discuss this further in the System Design Blueprint section.

Here are some common components:

Chat Service (Web/Slack): Manages user session, state and chat history

Gen AI Service: A simple, resilient client (with circuit breakers) that only calls LLM APIs (like Bedrock)

RAG Orchestrator: The backend brain that executes the tiered query logic

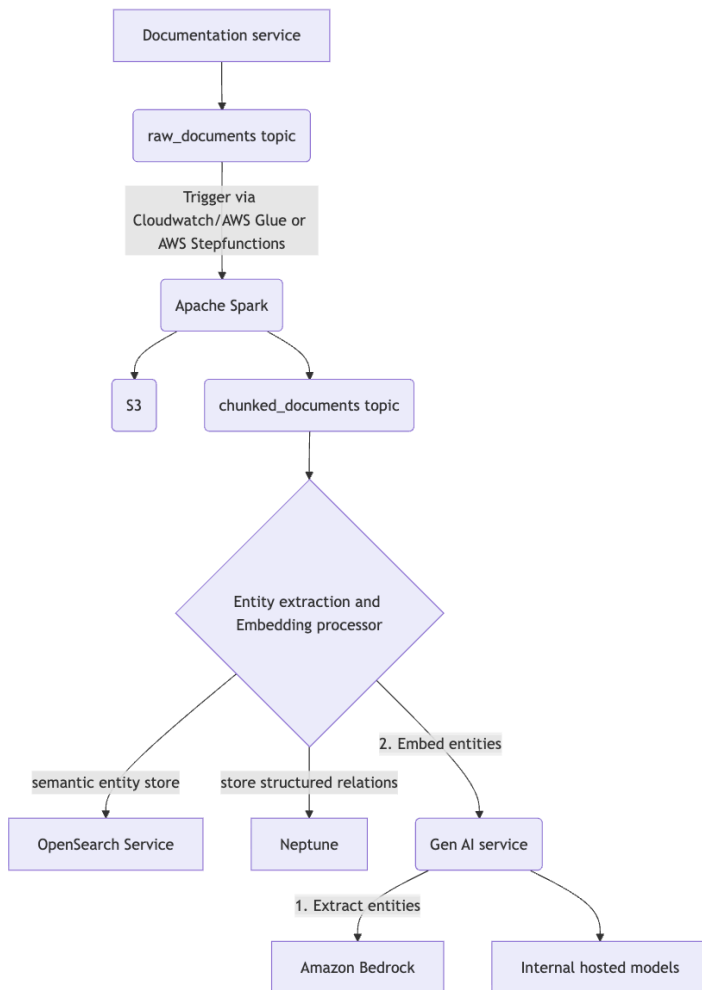


Fig 6.1: High level design

6.6. API design

6.6.1. Real time source webhooks

This API's job is to write to the *raw_documents* Topic whenever any documentation is updated, an issue or ticket is closed.

URI	/api/ingest/collect
Method	POST
Body	<pre>{"source_id": "TICKET-1990", "url": "...", "type": "TICKET"}</pre>
Response	<pre>{"status": "QUEUED", "message": "Source item added to raw_documents buffer."}</pre>

The Spark job is automatically scaled up (from 0 to N) when the Kafka *raw_documents* topic lag exceeds 200 messages, ensuring immediate freshness during spikes, and is otherwise guaranteed to run daily at 2:00 AM (via AWS Step Functions) for cost-efficient nightly cleanup.

6.6.2. User chats with customer support via Slack/Web

The service maintains a session state for each user session.

URI	/api/v1/chat
Method	POST
Body	<pre>{ "initial_query": "how to connect to db driver?", }</pre>

	<pre>"channel":{ "ctype" : "slack", "cid": "s3289710" }, "is_faq": false, "faq_id": null }</pre>
Response	<pre>{ "session_id": "s1", "response": "Here is the relevant documentation to help you out..", "confidence_score": 7.2, "channel": { "ctype": "slack", "cid": "s1382ry3" , "sent": true } }</pre>

6.6.3. Embed Chunked Documents (Internal API)

Accepts a text chunk and returns the high-dimensional vector

URI	/internal/ingest/chunk/embed
Method	POST
Body	<pre>{ "chunk_id": "c12", "content": "this is a sample chunk...", "embedding_model": "text-embedding-ada-002" }</pre>

Response	<pre>{ "chunk_id": "c12", "vector" :[-1219,32432,35434,-4353,43534], "dimensions": 1536, "model_used": "text-embedding-ada-002", "processing_time_ms": "100" // time taken for vectorization }</pre>
----------	--

6.7. System design blueprint

Let’s discuss the architecture for the system during the user query flow (i.e.chat or ticket creation by the user).

6.7.1. User Query flow

This section details the architecture of the User Query Flow, focusing on the decision-making and Hybrid Retrieval-Augmented Generation (RAG) engine.

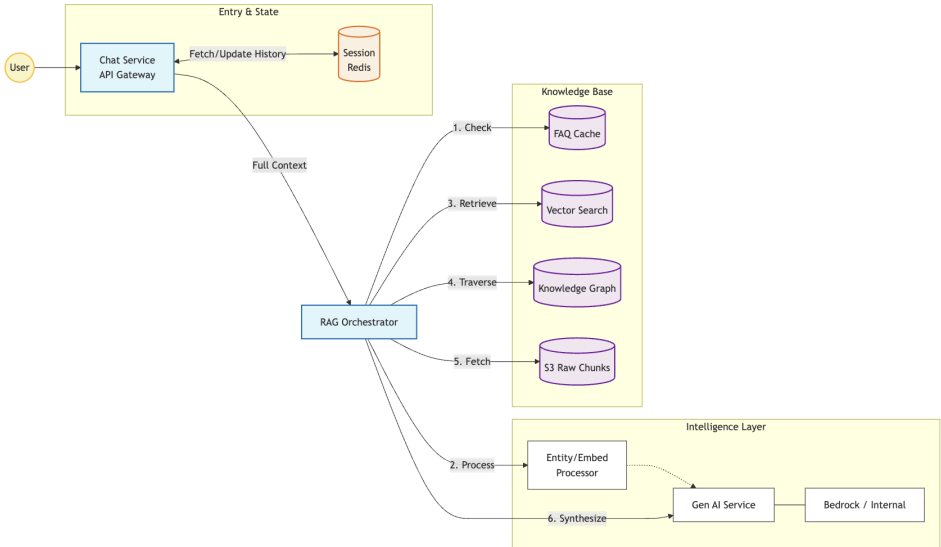


Fig 6.2: User query flow

6.7.2. Chat Service and State Management

Before a query is processed, the Chat Service (which handles the user-facing `/api/v1/chat` endpoint) must manage the conversational history. This state management is critical because we require analyzing the entire history of the chat, not just the single, most recent message. The system uses a `session_id` (returned in the initial chat response) to manage this. The `session_id` acts as a key to retrieve the full conversation history, which is stored in a dedicated, low-latency cache.

This way we avoid the inefficiency of passing the entire, potentially large, conversation history in the API call body for every single turn. This keeps the API request payload small. When a user sends a new message:

1. The Chat Service receives the new message and its `session_id`.
2. It sends a response back on the ticket stating it is working on resolving the query.

3. It retrieves the full chat history from the cache using the `session_id`.
4. It appends the new message to this history.
5. This complete, updated history is then passed to the orchestration layer for intent detection and RAG processing.

The history in the cache is updated with the new message and the subsequent response.

RAG Orchestrator Service

The Chat Service acts as a lightweight, user-facing API gateway. It is responsible for handling user authentication and session management (as described above). Once the user's query and full chat history are prepared, the Chat Service passes the request to the RAG Orchestrator service. The RAG Orchestrator is responsible for all downstream calls: checking the FAQ cache, performing vector search, executing graph traversals, and calling the Gen AI Service for final synthesis.

Tiered Orchestration and Intent Routing

The system prioritizes speed and cost-efficiency through a tiered approach managed by the RAG Orchestrator.

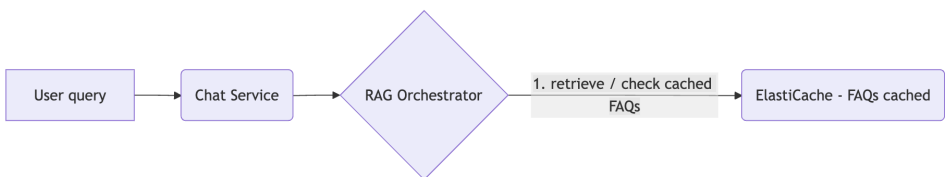


Fig 6.3: Rag orchestrator flow

Predefined Workflow/FAQ Redirect

When the RAG Orchestrator receives a request from the Chat Service, it first executes a fast, low-latency check. It checks a high-speed cache for an exact match against known, simple FAQs.

Why do we need the entire history of the chat?

In a multi-turn support chat, the user's latest message is often incomplete on its own.

Turn 1 (User): "My app isn't connecting."

Turn 2 (Bot): "What error code do you see?"

Turn 3 (User): "E11000"

If we simply vectorize E11000, the search engine lacks context (Is it a billing error? A connection timeout?). It effectively searches for a random alphanumeric string.

So, before performing vector search, the RAG Orchestrator sends the full chat history to the Gen AI Service.

Entity Extraction and Contextual Query Rewriting

In case of a cache-miss, the RAG Orchestrator sends the entire chat history to the Gen AI Service to analyze the context and extract the primary entities and intent of the user query. It generates a concise, context-aware internal search query and structured entities

{Component: 'Auth', ErrorCode: 'E11000'}

Hybrid RAG approach

The RAG Orchestrator takes this response and begins the hybrid retrieval.

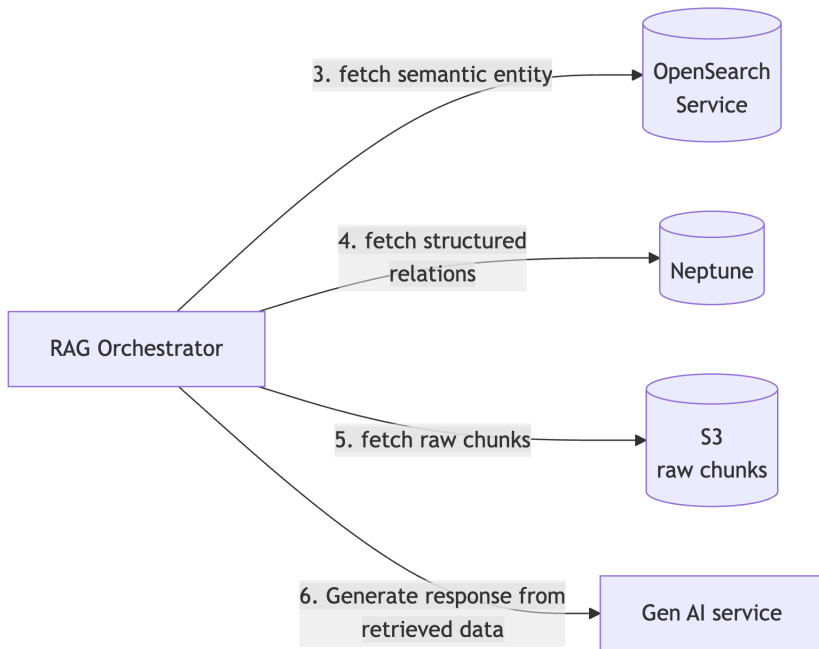


Fig 6.4: Hybrid RAG approach

Vector Search: It first semantically searches the vector database (OpenSearch) for similar chunks, identifying the top-K relevant documentations and support cases. The related documentation_ids/case_ids are scored according to their semantic similarity and relevance. The vector scores are summed up per document to identify the top-K relevant documentations, support cases etc.

Graph Traversal: The RAG Orchestrator then consults the Knowledge Graph (Neptune) to find structured paths (like Root Causes and Resolution Steps) connected to these retrieved items. The system then extracts their complete, structured solutions directly from the KG.

Chunk Retrieval: It then retrieves the raw text chunks for the final context from S3 (since the s3 chunk urls are attached to the nodes in the graph database). The raw text chunks for the resolution are retrieved then from S3

Final Synthesis: This comprehensive, grounded context is combined with the chat history and finally sent to the Gen AI Service to synthesize a highly accurate response.

6.7.3. Ingestion Pipeline

We build the Ingestion pipeline as a multi stage event driven architecture to ensure the knowledge base is constantly refreshed without impacting real time latency.

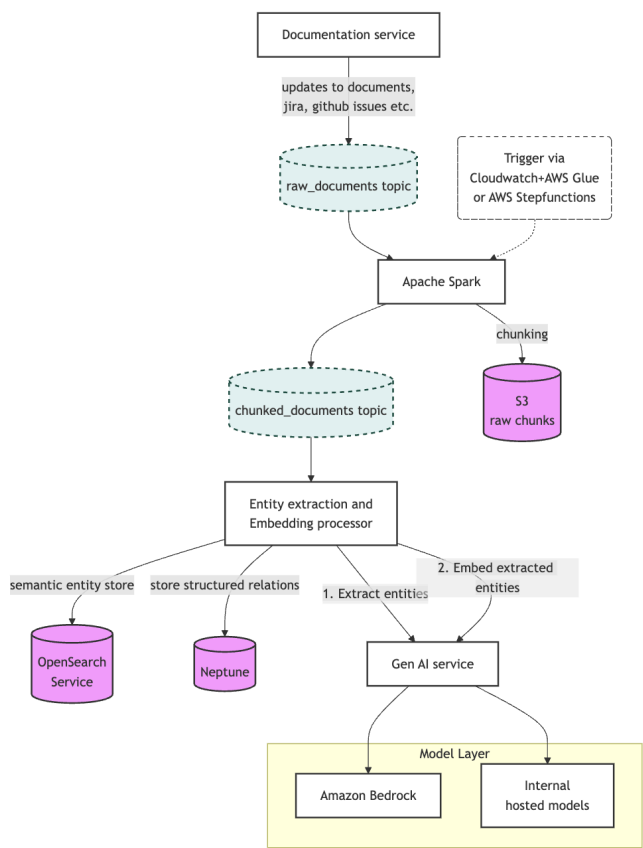


Fig 6.5: Ingestion pipeline

Asynchronous Triggering and Elastic Scale

All updates start with a single event. When documentation is updated, or a support ticket/GitHub issue is closed, the event metadata is captured and published to the Kafka *raw_documents* Topic. This topic acts as a durable queue buffer for all pending work.

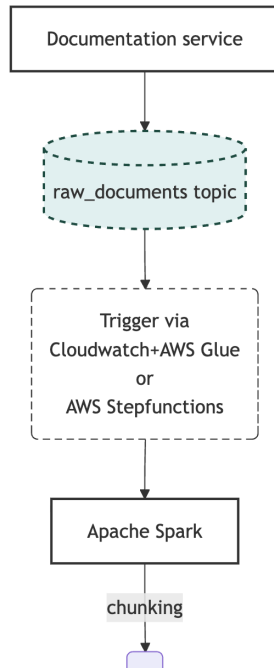


Fig 6.6: Async triggering

The load on the *raw_documents* Topic will be bursty. To handle this load, we use a hybrid triggering model:

Load Based Trigger

The Spark streaming job is automatically scaled up (from 0 to N pods) by cloudwatch monitoring when the *raw_documents* Topic lag exceeds message threshold (say 200). This ensures immediate processing during traffic spikes and optimises resources by scaling to zero when idle.

Time based Trigger

An AWS Step Function job is guaranteed to run daily at 2:00 AM for a cost-efficient nightly cleanup, ensuring any remaining small backlog is processed outside of peak hours.

Data Ingestion and Chunking (Spark)

The Spark streaming job consumes the backlog from the *raw_documents* Topic. It reads the links for the updated documents, closed tickets etc. This ensures processing is decoupled from source systems.

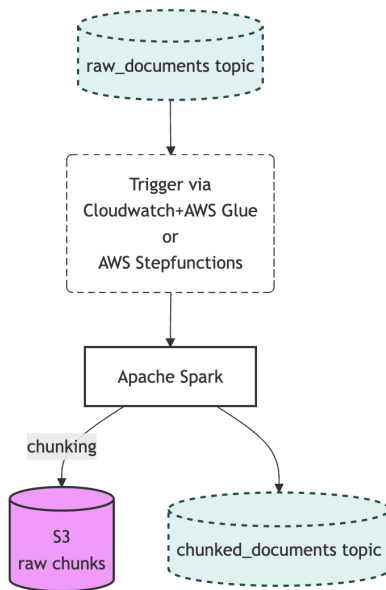


Fig 6.7: Data ingestion and chunking

The embedding model has a restriction on the size of the data being embedded. So we cannot just send an entire document (like the MongoDB manual) to be embedded by the model, it will be inefficient even if it is a smaller document.

So, the Spark job chunks the document into sections, or the JIRA Ticket into components.

Chunking respects semantic boundaries (sections, paragraphs, JIRA components).

This is crucial for retaining context and splitting content based on structure (sections, paragraphs etc. of a document or components of a ticket). Each chunk is assigned a unique id and retains a reference to the original document/ticket Id.

PII Redaction in the Spark Job

Before any document chunk is vectorized or stored in S3, it must pass through a PII Redaction filter. Since once PII is embedded in a vector, it is extremely difficult to selectively delete without re-indexing the entire corpus.

The Spark job then stores the raw chunks in blob storage like S3 Raw Storage for durability.

The link to S3 chunk, along with metadata, is then published to another Kafka topic (*chunked_documents* topic) to signal downstream readiness.

Entity Extraction and Embedding processor

The Entity Extraction and Embedding Processor consumes the events from the *chunked_documents* topic. It downloads the raw S3 chunk and initiates calls to internal models for extracting entities and relations.

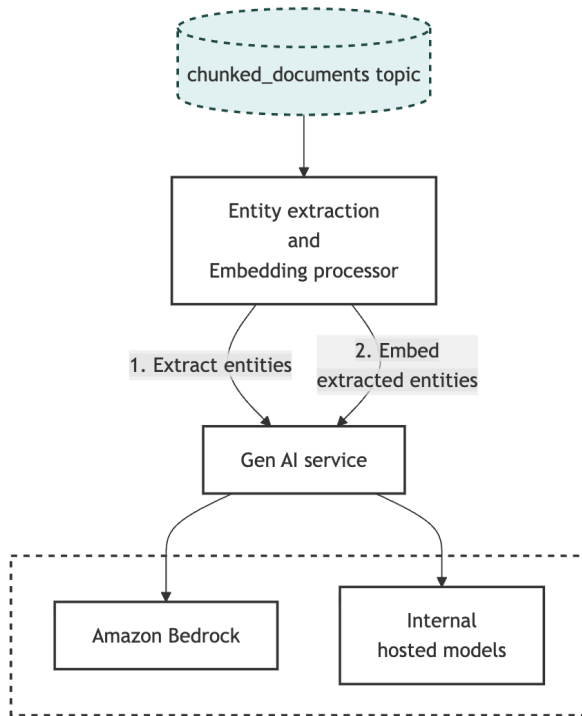


Fig 6.8: Entity extraction and embedding processor

Extract entities and relations

The processor sends the chunk to an internally hosted (and fine tuned) model to capture the structural relationships (entities and relations) for further processing.

Say we have a sentence

"The MongoDB find() method returns documents from a collection" it would extract:

Entities: find(), method, document, collection.

Relationships: find() -[is_a]-> method, method -[returns]-> documents, documents -[are_in]-> collection

Vectorize

Creating the high dimensional vector embedding for storage in the vector database, using the chunk id as the metadata. It adds the extracted entities and relations within the chunk before embedding. It uses an internally hosted model for generating embeddings.

Vector and Graph Persistence

The chunk's metadata is persisted in both the vector database and in the knowledge graph for the hybrid RAG retrieval.

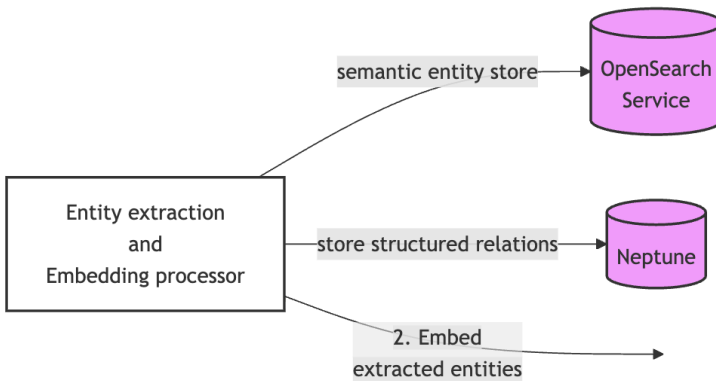


Fig 6.9: Vector and graph persistence

Vector Database (Semantic Search)

Stores high dimensional vector embedding along with metadata for fast semantic retrieval. Note that we also add the extracted entities and relations within the chunk before embedding.

An optimization for storage that is done here is that we don't embed the entire raw chunk in the vector database. Embed only dense, searchable content. We don't embed the raw text. The vector DB's job is semantic indexing, not raw data storage. In fact, the extracted entities, relations and their reference to the parent document are what we need here. So only this metadata can be embedded. The metadata also has information about the title of the document or Jira, etc.

This helps in better semantic search and better retrieval.

The actual chunk, which was stored in blob storage, like S3 is referenced in the knowledge graph and the vector database against the embedded metadata.

Knowledge Graph (Structured relations)

Once the main entities and their relationships are extracted, they are used to build the knowledge graph.

Intra-Connections (Within Item)

Define the internal structure of a single item.

Every documentation article can have different subsections discussing different points - Title, Sub-Heading, Section 1.1, etc. The subsections are represented as nodes connected by hierarchical relations, i.e. [Section 1.1] -[is_part_of]-> [Section 1].

For a JIRA ticket (title, description, resolution), the internal components form the connections.

Inter-Connections (Between Items)

Capture explicit links and structural dependencies across the knowledge base. This is vital for complex technical issues, such as capturing explicit links.

[Article A] -[references]-> [Article B]

or mapping component usage:

[Ticket A] -[uses_component]-> [MongoDB Sharding]

Gen AI service

The Gen AI service interacts with LLM vendors. It crafts prompts from templates and the context provided to it by the Spark job. Its primary purpose is to remain resilient in case of failures in a particular LLM

provider and reroute it to back up LLM providers. It also implements circuit breakers and exponential retries for this purpose. We will discuss how to expand this service in deep dive sections.

OpenAI text-embedding-ada-002 model

These are embedding models provided by OpenAI. These take the data and convert it into vector format for semantic comparisons.

6.8. Data Modelling

Existing tables in a customer agent system:

tickets

Context: The central ledger tracking the lifecycle of a support issue from creation to resolution.

Usage: High Concurrency. Updated frequently by both the AI (adding tags/summary) and Human Agents (changing status).

messages

Context: An immutable log of the conversation transcript, including metadata like who sent it (Bot vs. Human).

Usage: Write-Once, Read-Many. Used to build the Prompt Context for the AI and the Chat History UI for the user.

6.8.1. Blob storage for chunked data

The S3 Raw Storage serves as the immutable source of truth for all knowledge used by the RAG system.

Storage Choice and Durability

We leverage a cloud-based Blob Storage solution, such as Amazon S3 (or equivalent Azure Blob/GCP Cloud Storage), specifically utilizing a low-cost, high-durability tier (e.g., S3 Standard or Infrequently Accessed) for cost efficiency and reliability.

We store the finalized, raw text chunks generated by the Spark processing job in the S3 storage.

Access Patterns and load

High-Throughput Batch Writes: The system experiences high write throughput during the nightly (or threshold-triggered) batch execution of the Spark chunking job. The write access pattern is simple: append new files to the current bucket (date).

Low-Volume, Highly Specific Reads: The primary retrieval path is through the Vector DB/KG, not S3. Reads only occur after the retrieval layer has identified the top-K relevant chunk_ids. The read access will be through a direct S3 immutable file path.

S3 bucket path

The path is defined by the globally unique chunk_id and is organized by the date of ingestion for efficient governance and lifecycle management.

Path

s3://[bucket-name]/[source_type]/[year]/[month]/[day]/[chunk_id].txt

Example

s3://kb-storage/DOC/2025/10/01/c123456789.txt

6.8.2. Vector database

The Vector Database is the highly scalable, low-latency engine for **semantic search**. It is optimized for efficient index searching and metadata filtering, not raw text storage.

Technology Choice

We require a solution built for high-dimensional, Approximate Nearest Neighbor (ANN) search, managed service like Opensearch. We use AWS

Opensearch that can store and search vector embeddings for entity metadata of chunked articles and tickets.

Workload

High-Throughput Batch Writes: The database will see significant ingestion bursts when the Spark job processes the backlog, requiring the platform to scale its write capacity and indexing in real-time.

High-Velocity ANN Search: The primary operation is ANN search (or Cosine Similarity) on the vector embeddings to quickly find the top-K semantically relevant chunk_ids in sub-second time. This search is often combined with metadata filtering (e.g., filter by source_type).

The Spark-driven processor calls the /internal/ingest/kb/vector/add API, creating a new index entry per chunk, associated with its embedding and metadata.

knowledge_chunks

Context: Stores vector embeddings of documentation, parsed into small, semantic snippets.

Usage: Read-Heavy. Queried via vector similarity search on *every* user message to retrieve relevant context.

Field	Type	Required	Description
id	String	Primary Key	The unique identifier of the chunk (chunk_id). Used to retrieve the vector and its metadata
vector	Array of Floats	Required	The high-dimensional vector embedding obtained from the /embed service
source_metadata	Object	Required	Key metadata for search filtering and RAG grounding
doc_id	String	Required	The ID of the original document, ticket, or issue
source_type	String	Required	The type of knowledge: ("DOC", "TICKET", "GITHUB")
update_timestamp	String (ISO 8601)	Required	Timestamp of the last update; used for freshness checks

security_level	String	Optional	e.g., "PUBLIC", "INTERNAL"—crucial for filtering responses based on the customer's clearance level
chunk_url	String	Required	S3 path of the chunk

6.8.3. Knowledge graph

The Knowledge Graph (KG) provides the structural context necessary for the Hybrid RAG approach with verifiable, interconnected entities.

Technology Choice

We will utilize a dedicated, scalable Graph Database service such as Amazon Neptune or Neo4j, or a graph-optimized layer within our core data platform.

Amazon Neptune has a managed, serverless architecture, which automatically handles scaling and capacity provisioning. This removes the operational overhead of manually configuring instances according to capacity.

Workload

Heavy write throughput: Writes are based on adding new entity_ids and their relations (nodes and edges). These occur asynchronously during the ingestion pipeline's scheduled runs.

Targeted Traversal: We retrieve the intra-relationships and structure for specific IDs in the RAG prompt.

The KG is composed of two primary entities: Nodes (Entities) and Edges (Relationships).

1. Node Structure (Entities)

Nodes represent discrete, identifiable concepts (within the MongoDB ecosystem as per our example).

Field	Data Type	Constraint	Description	Example Value
id	String	Primary Key	Unique identifier for the entity(can be doc id or jira id etc + type)	E-5f72a4c
name	String	Required	The name of the entity	"MongoDB Node.js Driver"
labels	Array of Strings	Required	type(s) of the entity, critical for graph traversal filtering.	["Driver", "Component", "Software"]
attributes	Object	Optional	Key-value pairs containing entity properties.	{"version": "4.5.0", "language": "JavaScript"}
chunk_id_ref	String	Conditionally Required	References the chunk_id in the Vector DB/S3	c12345

2. Edge Structure (Relationships)

Edges connect nodes and define the semantic meaning between them, allowing the system to reason across documents.

Field	Data Type	Constraint	Description	Example Value
source_id	String	Required	The id of the starting node.	E-5f72a4c
target_id	String	Required	The id of the ending node.	E-9b3d1e0

relationType	String	Required	The type of relationship required for defining traversal paths.	"DEPENDS_ON"
attributes	Object	Optional	Properties of the relationship itself.	{"weight": 0.8, "verified": true}
source_context	String	Required	The ID of the document/chunk that explicitly stated this relationship. Required for RAG traceability.	c12345

3. Key RAG-Optimized Relationship Types

This ensures that when the RAG engine executes its graph traversal step, it retrieves not just text, but structured answers that are easier for the final LLM to synthesize accurately.

Relationship Type	Nodes Connected	Purpose in RAG Retrieval
IS_PART_OF	[Section] → [Article/Ticket]	Used to retrieve the entire parent context when a section is matched.
REFERENCES	[Article] → [Article]	Expands the retrieval search to highly relevant, related documents.

HAS_ROOT_CAUSE	[Ticket] → [Concept/Entity]	If a user mentions a concept, the RAG can traverse back to a ticket with the same root cause.
USES_COMPONENT	[Ticket] → [Driver/Service]	Filters retrieval to ensure the answer is relevant to the customer's stated version/driver (e.g., "Node.js").
RESOLVES	[Resolution Chunk] → [Problem/Error]	Allows the RAG engine to retrieve the canonical "Resolution Step" for a known error ID.

6.9. Deep Dive

Let's recap what we have discussed so far.

Functional Requirement	Design Solution Summary
Knowledge Retrieval	RAG Pipeline: Vector search on documentation chunks + Hybrid Keyword search
Context Awareness	Conversation Buffer: Injecting the last N turns into the prompt so the bot remembers "My server version is 5.0".
Hallucination Control	Citation Enforcement: The UI only renders an answer if the LLM cites a specific Chunk ID from the Knowledge Base.
Ticket Management	State Machine: Tracks ticket lifecycle (Open -> Bot_Active -> Escalated -> Closed) in PostgreSQL.
Smart Escalation	Next Focus: Sentiment Analysis to auto-detect angry users and Handoff for real-time agent takeover.
Action Execution	Next Focus: Tool Use (Function Calling) to allow the bot to actually <i>do</i> things (e.g., "Restart Cluster") safely.

Let's look closely at the non functional requirements of the design now.

6.9.1. Scalability

Scaling focuses on ensuring that the high volume ingestion path and the real time query path remain bottle neck free.

Spark Ingestion Jobs

Scaling is handled dynamically by cloudwatch monitoring the Kafka topic lag in the AWS MSK. This allows the AWS Glue to manage the automated scaling of Spark cluster from 0 to N worker pods only when processing is required, providing immediate throughput increases during ingestion spikes. The scaling is driven by data volume, ensuring cost optimization.

OpenSearch Serverless Auto-Scaling

We utilize Amazon OpenSearch Serverless, which automatically scales capacity based on the workload's demands. This eliminates the need for manual sharding/instance management and guarantees that the high write throughput from the ingestion pipeline and high read concurrency during peak RAG queries are both handled elastically.

S3

Similarly, S3 is an AWS managed service and a thread pool can be used to manage connections to S3.

Graph Database

We deploy Amazon Neptune Serverless for the KG. It adjusts compute resources (ACUs) based on the changing requirements of the complex traversal queries. It allows handling graph workloads at scale without the need for managing or optimizing database capacity.

Gen AI Service

The Gen AI Service can become a bottleneck if not scaled properly. When facing a huge load of requests, it needs to actively send the request to the

LLM providers which are available, and trip the ones that are having delays. This can be done using circuit breaker configuration in the Gen AI Service. It monitors latency from external LLM providers, and trips the circuit for slow endpoints, automatically rerouting the request to available backup LLM models to maintain user-facing performance.

6.9.2. Reducing latency

Low latency is crucial for an AI powered customer service agent to ensure a smooth user experience.

When the user creates a ticket, the AI powered agent should start working on it and be able to respond with resolutions within a few seconds. The customer, who typically waits for a manual agent, would traditionally expect responses in hours. In the case of an enterprise agent, with dedicated support, it would still take minutes for manual response.

The overall system must meet the <60 seconds NFR by optimizing the sequential and parallel steps in the RAG pipeline.

Retrieval optimization

The retrieval phase runs Vector Search (OpenSearch) first to identify the relevant IDs, followed by the specific ID-based lookups in the Neptune KG. The latency is minimized since expensive Neptune traversal is constrained to only the handful of verified, relevant document IDs retrieved from OpenSearch.

SLMs for Pre-Processing

Smaller, fine-tuned models are used for low-cost, low-latency tasks like Intent Detection and Entity Extraction, reserving the larger, slower Generative LLM for the final answer synthesis.

This reduces the time spent on the initial query analysis.

Response and Prompt Caching

The LLM Orchestrator implements two caching layers:

Response Cache

Caches the final LLM response using the Hash of the Contextualized Query + LLM Model ID as the key.

Prompt Cache

Caches static prompt templates using the Prompt ID as the key.

Co-location and Connection Pooling

All core services (Orchestrator, OpenSearch, Neptune, and LLM Providers' regional endpoints) must be co-located within the same AWS region. Connection pooling is used for all external LLM provider APIs to minimize HTTP overhead.

Coalesce Caching (Request Coalescing)

Thousands of users often rush to support asking the exact same question simultaneously e.g., in case of a partial outage. A standard cache will miss the first request for everyone, causing a thundering herd that can crash your LLM gateway.

We implement Coalesce Caching (or Request Collapsing) in the API Gateway. The gateway identifies identical in-flight requests. It pauses all subsequent requests, waits for the first LLM response to complete, and then serves that single response to all 10,000 waiting users at once. This reduces the load on the LLM provider by orders of magnitude during critical incidents.

6.10. Accuracy - Avoiding hallucinations

For better customer satisfaction in handling tickets, we need to ensure our system returns the correct and relevant responses. Ensuring the system returns verifiable and correct responses is paramount to achieving a high Customer Satisfaction (CSAT) score. This relies on strategic guardrails within the Hybrid RAG flow.

6.10.1. Explicit Prompt Guardrails

We implement strong, explicit guardrails within the LLM's system prompt. This instruction set mandates that the LLM must not hallucinate and should reply with a standardized fallback message (e.g., "I cannot find the relevant information...") if the retrieved context from the Knowledge Graph or Vector Database is insufficient.

6.10.2. Validation and Evaluation Suite

Golden Dataset Evaluation

We maintain a dedicated golden dataset of verified queries and correct answers. Our system is continuously evaluated against this set.

Example 1: Simple FAQ / Intent Detection

Query: "how to connect to db driver?"

Golden Answer (Factual): "You can connect to a MongoDB database using a connection string (URI) and a language-specific driver. For example, in Node.js, you would use the MongoClient class. Here is a basic code snippet: [Code Snippet]"

This tests the system's ability to handle simple, common questions and provide accurate, helpful code.

Example 2: Complex Query / RAG Retrieval

Query: "I'm trying to optimize read performance for a large collection, but my find() query with a sort() is slow. I already have an index on the sort field."

Golden Answer (Synthesized): "A slow find() with a sort()"

on an indexed field often indicates that the query is not using the index efficiently or that the index doesn't cover the entire query.

Check the Index: Ensure the index also includes the fields in your query's filter, not just the sort field.

Covered Query: For best performance, create a 'covered index' that includes all fields used in both the query filter and the sort operation.

Use `.explain()`: Run `db.collection.find(...).sort(...).explain('executionStats')` to verify that the query is using your index (look for `IXSCAN`) and not performing an in-memory sort (`SORT` stage)."

This tests the Hybrid RAG engine's ability to retrieve, synthesize, and relate multiple concepts (find, sort, indexes, explain) to solve a nuanced performance issue.

Example 3: Error Code / GraphRAG Traversal

Query: "My app is crashing, I see an 'E11000' error in the logs."

Golden Answer (Contextual): "An 'E11000 duplicate key error' means you are trying to insert or update a document that violates a unique index. This commonly happens when you try to insert a new document with a `_id` that already exists or a value in another field (like 'username') that is defined as unique. Please check the document you are trying to insert and ensure its unique fields have new values."

This tests the system's ability to map a common entity (the error code) to its specific root cause and resolution steps, a key feature of the knowledge graph.

Example 4: Unanswerable / Intelligent Escalation

Query: "I think my billing is wrong, you charged me twice this month."

Golden Answer (Escalation): "I see you have a question about billing. I am not equipped to handle billing information or access your account details for privacy reasons. I will escalate this to a human agent who can securely assist you."

This tests that the agent correctly identifies a query it has low confidence on (or is out-of-scope) and hands off appropriately.

LLM as a Judge (LLM-Judge)

We employ a separate, powerful LLM instance configured as a Judge Model to score the system's live performance against the golden dataset or against real-life user queries. This provides a quantifiable metric (according to the Judge LLM) on the quality and helpfulness of the response.

Here are a few practical examples of how this LLM-Judge would be used in the context of the MongoDB support agent:

Example 1: Scoring for Accuracy and Correctness

This example tests if the agent's answer is factually correct by comparing it to a known-good golden answer.

Internal Prompt to LLM-Judge:

User Query: "My app is crashing, I see an 'E11000' error in the logs."

Agent Response: "An 'E11000' error is a duplicate key error. This means you are trying to insert a document with an `_id` that already exists. You should check your code."

Golden Answer: "An 'E11000 duplicate key error' means you are trying to insert or update a document that violates a unique index. This could be the `_id` field, or another field you defined as unique (e.g., 'username'). You should check the document you are trying to insert."

On a scale of 1-5, how factually accurate is the Agent Response compared to the Golden Answer? Explain your reasoning.

LLM-Judge Output:

Score: 3/5

Reasoning: The agent's response is partially correct but incomplete. It correctly identifies the 'duplicate key error' but incorrectly assumes it only applies to the `_id` field. The golden answer is more accurate because it correctly states that the error is due to any unique index, which is a critical distinction for a developer.

Example 2: Scoring for Groundedness (Detecting Hallucination)

This example tests if the agent's answer is grounded in the factual context provided by the RAG system, or if it's hallucinating (making up) information.

Internal Prompt to LLM-Judge:

Retrieved Context: "[chunk_1]: ...For optimizing 'find()', the most important tool is the index. Use '.explain('executionStats')'... [chunk_2]: ...If your query filters on 'fieldA' and sorts on 'fieldB', a compound index on '{ fieldA: 1, fieldB: 1 }' is most effective..."

Agent Response: "To optimize a slow 'find()' query, you should use an index. A good first step is to run '.explain()'. You can also try using the 'setQueryOptimizer()' function, which often helps."

Is the Agent Response fully grounded in the Retrieved Context? Are there any statements in the response that are NOT supported by the context?

LLM-Judge Output:

Grounded: No (Partially)

Reasoning: The agent's statements about "index" and ".explain()" are grounded in the context. However, the suggestion to use setQueryOptimizer() is not present anywhere in the provided context. This is a potential hallucination that could mislead the user. The Grounding Score for this response would be low.

Example 3: Scoring for Helpfulness and Tone (Open-Ended)

This example evaluates the agent's performance on a real-world user query where there is no golden answer.

Internal Prompt to LLM-Judge:

User Query: "this stupid driver won't connect i'm just getting a timeout error"

Agent Response: "I understand you're frustrated. A timeout error when connecting often means... [full response text] ..."

On a scale of 1-5, rate the quality and helpfulness of the Agent Response to the User Query. Consider:

1. Did it address the user's technical problem (timeout error)?
2. Did it handle the user's frustrated tone appropriately?
3. Was the answer actionable?

LLM-Judge Output:

Score: 5/5

Reasoning: The agent performed excellently.

Tone: It immediately acknowledged the user's frustration ("I understand you're frustrated...") before becoming technical.

Technical: It correctly identified the likely class of problem (reachability) and provided the three most common, actionable solutions (Firewall, IP Whitelisting,

Connection String). This is a highly helpful and high-quality response.

Human-in-the-Loop

A dedicated team of domain experts constantly reviews cases flagged by low system confidence scores or negative user feedback. This feedback loop is essential for quickly identifying drift and retraining/refining the knowledge base.

This includes evaluating the performance of different models with different prompts and rate which one works best.

This also includes system evaluation for real life user queries and evaluating how the system fares.

6.10.3. Smart Chunking and Metadata

Accuracy starts at ingestion. We enforce Smart Chunking by including necessary continuity context (parent section, title) in every chunk's metadata. This ensures that the Vector DB retrieval provides context that is not just semantically similar, but well structured too, leading to a higher quality fusion step.

6.10.4. Reliability

The system needs to be reliable so that even though dependencies like LLM provider models may be down, the whole system should still be functional. This is handled by the Gen AI Service, by implementing circuit breakers. The LLM provider models which are not performing well, are slow, or unavailable, trip the circuit. The Gen AI Service then reroutes the prompt requests to backup models which are available.

6.10.5. Privacy

We enforce strict data isolation to ensure that confidential or internal corporate knowledge never leaks into public responses, ensuring privacy.

Ingestion Isolation

The initial ingestion pipeline configuration (Spark job/data source connectors) must be strictly defined to only crawl and process documents explicitly tagged as "PUBLICLY AVAILABLE" (e.g., MongoDB Documentation, public GitHub issues)

Query-Time Filtering

The Vector Database (OpenSearch/Neptune) is configured to filter search results based on the `security_level` metadata associated with the user's session, guaranteeing that even if internal data were present (e.g., for internal-facing agents), it would not be exposed to a public user.

Middleware PII Masking

We cannot trust the LLM to ignore sensitive data. We implement a middleware layer before the prompt construction that detects patterns (credit card numbers, SSNs) and replaces them with tokens (e.g., `<CREDIT_CARD_MASKED>`). This ensures sensitive user data never enters the context window or the third-party provider logs.

6.11. Monitoring and Alerting

We set up monitoring and alerting for different aspects of the system to identify their performance. A robust monitoring strategy is essential for maintaining the Reliability and Performance (<5 sec NFR) of the Hybrid RAG system. We define a tiered approach using observability platforms (e.g., CloudWatch, Prometheus/Grafana) to cover the entire pipeline, from ingestion to user response.

6.11.1. Ingestion Pipeline Metrics (Freshness and Cost Control)

These metrics track the health and efficiency of the asynchronous knowledge update process, which runs on Spark and Kafka.

Kafka raw_documents topic

Alert: Lag >200 messages (trigger autoscaling via cloudwatch and AWS glue) and >5000 messages (high-priority alert)

Ingestion Slowdown

Indicates source systems are producing data faster than the Spark cluster can consume. Triggers AWS Glue to scale the Spark cluster.

End-to-End Ingestion Latency

Alert: Time to ingest a single document/ticket >30 minutes (from source event to Vector DB write)

Stale Knowledge: The system is indexing old data. Requires investigating the Kafka-Spark connection or the Gen AI Service API health.

Vectorization Latency

Alert: P95 latency for /internal/ingest/chunk/embed>500 ms

Embedding Bottleneck: Indicates the dedicated embedding model inference endpoint is underprovisioned or congested. Triggers auto-scaling of the embedder service.

6.11.2. Query Path Metrics (User Experience and Performance)

We evaluate the query path performance by capturing different metrics as below.

End-to-End RAG Latency

Alert: P90 latency for /api/v1/chat/message>5 seconds

NFR Breach: Immediate, critical alert. Triggers scaling review for LLM Orchestrator and core retrieval services

Retrieval Latency (Total)

Alert: P90 time for Vector DB+KG Lookup>1.2 seconds

Retrieval Bottleneck: Indicates the OpenSearch index or Neptune read replicas are saturated. Triggers automated scaling of read replicas/serverless capacity

LLM Inference Latency

Alert: Individual LLM call time (Time to Last Token) >4 seconds or Time to First Token (TTFT)>1 second

Provider Performance Degradation: Triggers the Circuit Breaker in the LLM Orchestrator to route traffic to a healthy backup provider

6.11.3. Financial and Quality Metrics

We track the cost incurred due to LLM and other resources.

LLM Token Usage & Cost

Alert: Daily cost exceeds X (e.g., 150% of baseline)

Monitor input/output token counts per query against the provider's API usage dashboard

Detects inefficient prompts, infinite conversation loops etc.

Accuracy / Grounding Score

Alert: Grounding score <90% (or user feedback is >5% negative)

LLM-Judge metric, Human Expert Verification (via HITL), and User Feedback (Thumbs Up/Down)

This indicates that the RAG pipeline is not fetching the relevant context.

Review of chunking strategy, vector database is necessary

Escalation Rate

Alert: Escalation rate spikes >20% above baseline

Tracks rate of redirects from system agent to human agent

Indicates the agent is failing to solve queries, possibly due to failure in intent detection model

6.11.4. System Health

We monitor and track the system metrics to understand the health of our system at any point.

Resource Utilization

Alert: CPU utilization >85% or Memory usage >90% for any service instance

Triggers horizontal scaling or capacity upgrade

Error Rate

Alert: 5xx error rate >1% of total requests

Indicates a service is unhealthy

Dependency Health

Alert: Continuous failures of connections to dependencies like S3 etc.

Indicates network isolation issue or service failures

6.12. Failure Mode Analysis (FMA)

We must design for the inevitable failure of the non-deterministic components.

Component	Failure Mode	Impact	Mitigation Strategy
LLM Provider	Hallucination	The agent gives confident but incorrect legal/technical advice.	Grounding & Citation: The UI only shows answers if they cite a specific chunk ID from the Knowledge Base. No chunk = No answer.

LLM Provider	Latency Spike (>10s)	User abandons chat; Cascading timeouts in API Gateway.	Streaming & Fallback: Stream tokens immediately to show activity. If TTFT > 5s, auto-failover to a less smart but faster local model.
---------------------	--------------------------------	---	--

6.13. Conclusion

Using intelligent integrations with LLMs we can help reduce human agent workload so that they can focus on escalated and complex issues, leaving the simple ones to the LLM customer agents.

6.14. References

1. [Path to high-quality LLM-based Dasher support automation](#)
DoorDash. “Path to High-Quality LLM-Based Dasher Support Automation.” September 17, 2024.
<https://careersatdoordash.com/blog/large-language-modules-based-dasher-support-automation>
2. [Genie: Uber’s Gen AI On-Call Copilot](#)
Chothani, Paarth. “Genie: Uber’s Gen AI On-Call Copilot.” Uber Blog, October 10, 2024.
https://www.uber.com/en-HR/blog/genie-ubers-gen-ai-on-call-copilot/?uclid_id=92508acc-3a86-4fcc-bc5f-ba1799e3055e
3. [Retrieval-Augmented Generation \(RAG\) with MongoDB](#)
MongoDB Docs. “Retrieval-Augmented Generation (RAG) with MongoDB - Atlas.” Accessed December 5, 2025.
<https://www.mongodb.com/docs/atlas/atlas-vector-search/rag/>

4. [Better Customer Support Using Retrieval-Augmented Generation \(RAG\) at Thomson Reuters](https://medium.com/tr-labs-ml-engineering-blog/better-customer-support-using-retrieval-augmented-generation-rag-at-thomson-reuters-4d140a6044c3)
Unni, Keshav. "Better Customer Support Using Retrieval-Augmented Generation (RAG) at Thomson Reuters." Thomson Reuters Labs, August 8, 2023.
<https://medium.com/tr-labs-ml-engineering-blog/better-customer-support-using-retrieval-augmented-generation-rag-at-thomson-reuters-4d140a6044c3>

7. Appendix

A

- **Abstract Syntax Tree (AST):** A structured, tree-like representation of source code that captures the hierarchical relationship between classes, functions, and methods. Used in system design for semantic code chunking.
- **Agentic AI:** An autonomous system that uses an LLM to reason, plan, and execute a multi-step workflow (Reason + Act loop) to achieve a high-level goal without a pre-defined path.
- **Artificial Intelligence (AI):** A broad branch of computer science focused on building smart machines capable of performing tasks that typically require human intelligence, such as learning and problem-solving.

C

- **Circuit Breaker:** A resilience pattern that detects when an external provider (like an LLM API) is failing. It temporarily stops sending requests (opens the circuit) to prevent cascading failures, often rerouting traffic to a fallback model.
- **Coalesce Caching:** A concurrency pattern where middleware identifies identical in-flight requests during traffic spikes, pauses subsequent ones, and serves a single LLM response to all waiting users.
- **Context Engineering:** The strategic design of the data pipeline that assembles relevant information (history, RAG context, instructions) to fit within the model's context window at inference time.
- **Context Window:** The short-term memory of an LLM. It is the hard limit on the amount of text (measured in tokens) the model

can process at one time, including both the prompt and the generated response.

D

- **Deep Learning:** An advanced subset of Machine Learning using multi-layered neural networks to learn complex patterns from unstructured data like images, audio, and text.
- **Deterministic:** A system behavior where the same input always produces the exact same output. Standard software is deterministic; LLMs are not.

E

- **Embeddings:** High-dimensional numerical vectors that represent the semantic meaning of text or data. They allow computers to compare concepts based on meaning rather than keywords.

F

- **Fan-Out:** The pattern of taking one large task (Generate 100 questions) and splitting it into many small tasks (10 workers generating 10 questions each) to run in parallel.
- **Fine-Tuning:** The process of taking a pre-trained generalist model and training it further on a specific, smaller dataset to make it an expert in a particular domain (e.g., medical or legal).
- **Function Calling:** A capability where an LLM outputs a structured request (like JSON) to execute a specific tool or API function (like `verify_math(2+2)`), rather than generating text.

G

- **Generative AI (Gen AI):** A type of AI capable of creating new, original content (text, images, code) rather than just analyzing existing data.

- **Golden Dataset:** A manually curated set of inputs and ideal outputs used as the ground truth for testing and benchmarking LLM performance.
- **GraphRAG:** An advanced retrieval technique that combines Knowledge Graphs with LLMs, allowing the system to traverse relationships between entities rather than relying solely on vector similarity.

H

- **Hallucination:** A failure mode where an LLM confidently generates false or nonsensical information because it is statistically plausible based on its training data.
- **HNSW (Hierarchical Navigable Small World):** An algorithm used by vector databases (like `pg_vector`) to find nearest neighbors (similar questions) extremely fast, without scanning the whole database.
- **Hybrid RAG:** A retrieval architecture that queries both a Vector Database (for semantic similarity) and a Knowledge Graph (for structured relationships) to build a richer context for the LLM.

I

- **Ingestion Pipeline:** An asynchronous data pipeline that cleans, chunks, embeds, and stores documents into a vector database to prepare them for RAG retrieval.

K

- **Knowledge Graph (KG):** A structured database that stores data as entities (nodes) and relationships (edges), enabling the retrieval of information based on logical connections.

L

- **Lambda Architecture:** A big data processing pattern that combines two data processing paths: a batch layer for accurate, historical analysis and a speed layer for real-time insights, merged in a serving layer to provide comprehensive, fault-tolerant, and scalable data views for complex applications like recommendation engines.
- **Large Language Model (LLM):** A massive AI model trained on a vast dataset (internet scale) designed to understand and generate human language for general-purpose tasks.
- **LLM Gateway:** A centralized microservice pattern that acts as the single entry point for all LLM calls, handling routing, auth, rate limiting, and failover.
- **LLM-as-a-Judge:** An evaluation pattern where a highly capable LLM (like GPT-4) is used to score the quality of responses generated by another model against a Golden Dataset.
- **LPOP:** Operation in redis, Removes and returns the first elements of the list stored at key

M

- **Machine Learning (ML):** A subset of AI where systems learn patterns from data rather than being explicitly programmed with rules.
- **Modality:** The type or format of data an AI model can process, such as text, images, audio, or video.
- **Model Router:** A logic component within the LLM Gateway that dynamically directs requests to the most appropriate model (e.g., cheap vs. powerful) based on task complexity or cost constraints.

N

- **Natural Language Processing (NLP):** A branch of AI focused on giving computers the ability to understand, interpret, and generate human language.

P

- **PII:** Personally Identifiable Information is any data that can be used to identify a specific individual, either directly, like a name or Social Security Number or indirectly, like an IP address or date of birth when combined with other info
- **Pre-Training:** The initial, computationally expensive phase of training an LLM on a massive dataset to teach it the fundamentals of language and world knowledge.
- **Prompt:** The input instruction or question given to an AI model to trigger a response.
- **Prompt Engineering:** The practice of structuring and optimizing input prompts (using context, personas, and constraints) to improve the quality of the LLM's output.
- **Prompt Injection:** A security vulnerability where malicious inputs trick an LLM into ignoring its system instructions and performing unauthorized actions.

R

- **Reasoning:** The ability of an LLM to process information logically and generate a conclusion, often simulated via Chain-of-Thought processing.
- **Retrieval-Augmented Generation (RAG):** An architectural pattern that retrieves relevant, private data from an external source and injects it into the context window, grounding the LLM's response in fact.

S

- **Small Language Model (SLM):** A compact AI model trained for efficiency and specific domains, capable of running on local devices with lower latency and cost than LLMs.
- **Stochastic:** Involving randomness. LLMs are stochastic because they select the next token based on probability, meaning the same input can yield different outputs.

- **Stop Word Removal:** Stop word removal is a text preprocessing step in Natural Language Processing (NLP) where common, grammatically essential words (like "the," "a," "in," "is") that carry little semantic meaning are filtered out.

T

- **Temperature:** A parameter that controls the randomness of an LLM's output. Low temperature (0.1) makes it deterministic/focused; high temperature (0.8) makes it creative/random.
- **Time To First Token (TTFT):** A latency metric measuring the time elapsed between the user sending a prompt and seeing the first character of the response.
- **Time Per Output Token (TPOT):** A metric measuring generation latency (time per single token generated). High TPOT indicates a slow model or overloaded provider.
- **TTL (Time To Live):** A feature (used in Redis etc.) that automatically deletes data after a set time (e.g., 5 minutes)
- **Token:** The fundamental unit of text (part of a word) that an LLM processes. Usage and billing are calculated per token.

V

- **Vector Search:** A search technique that uses embeddings to find data that is semantically similar to a query, even if it shares no keywords (e.g., matching "pay" with "salary")